



北京大学
PEKING UNIVERSITY

智能硬件体系结构

第十二讲：存算一体简介

主讲：陶耀宇

2026年春季

- 课程作业情况

- 作业时间

第二次作业时间：5.11-5.25

第三次作业时间：5.28-6.13

- 第1次lab时间：4月13日-5月28日 **(因CLAB故障延期)**
- 第2次lab时间：5月23日-6月18日 **(因CLAB故障延期，已适当简化)**
- 文献调研与报告（6月1日、6月8日）（形式待定，可以录视频）

主讲：陶耀宇、李萌

- 自行选择器件 (IEDM等)、电路 (ISSCC、VLSI等) 或架构 (MICRO、ISCA等) 方面的论文, 或Nature/Science系列相关论文, **进行3-5篇文献阅读**
- **占总成绩20%**, 分组 (2-3人一组) 深入论文技术细节, 做**10分钟汇报**

AI加速器芯片

- 传统AI加速器: Fused-layer cnn accelerators、Eyeries, Google TPU等
- 新兴AI加速器 (大模型Transformer、Neural ODE、MANN/DNC、PINN等)

GPGPU芯片

- 流式多处理器 (Multithreaded Streaming Multiprocessors, CUDA的来源)

FPGA芯片等 (可编程逻辑块、可编程路由等)

安全与通信领域处理器芯片

- 各类密钥编码 (AES、RSA)、视频编码 (MPEG等)、通信编码 (LDPC、Polar等)

传统CPU芯片

- 优化Branch Predictor、Load-Store、缓存预读取、众核缓存一致性等

新兴智能计算芯片

- 存算一体/感存算一体、量子计算、生物信息处理、高维NoC、区块链
- 基于后摩尔非CMOS器件的架构 (模拟计算架构、动力学计算架构等)

目 录

CONTENTS



- 01. 典型AI芯片架构**
- 02. AI芯片软硬件协同设计**
- 03. AI大模型芯片设计**
- 04. 存算一体AI芯片架构**

目录

- 研究背景
- 模型级加速
 - 模型压缩
 - 稀疏性处理
- 模块级加速
 - 注意力模块加速
 - 前馈神经网络加速
- 算子级加速
 - 线性算子加速
 - 非线性算子加速
- 总结及展望

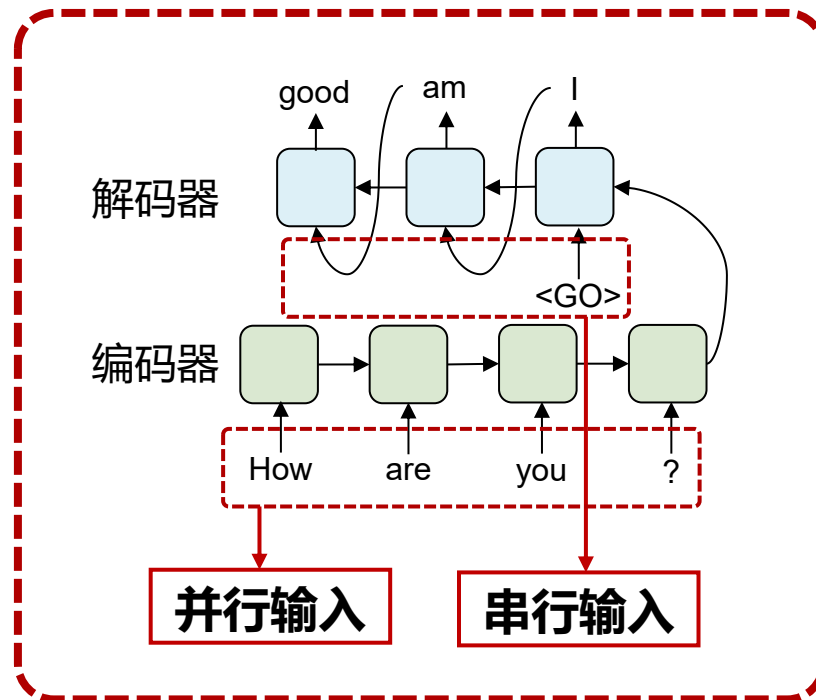
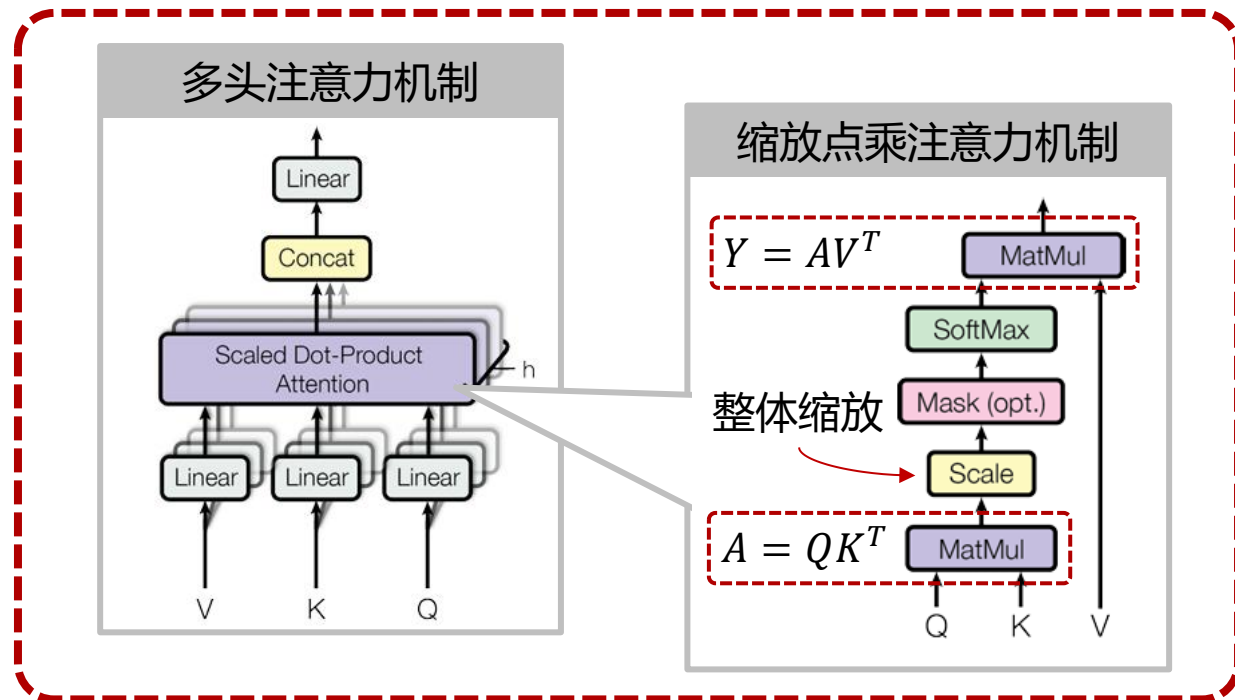
模块级加速：注意力模块加速

- 注意力模块结构分析

- 数据调度主要难点：MatMul运算需要等待所有token的线性层运算结束

- 架构加速需求：

- 流水并行设计、分阶段并行设计



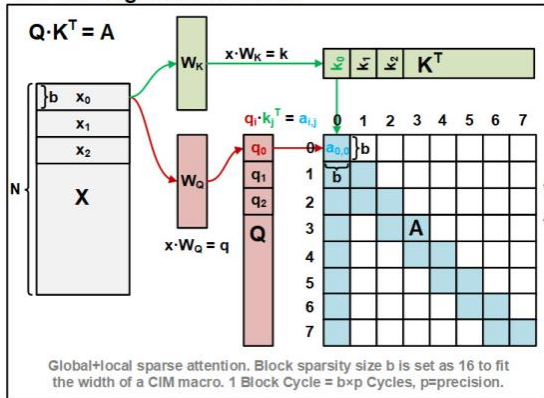
模块级加速：注意力模块加速

• 流水并行设计

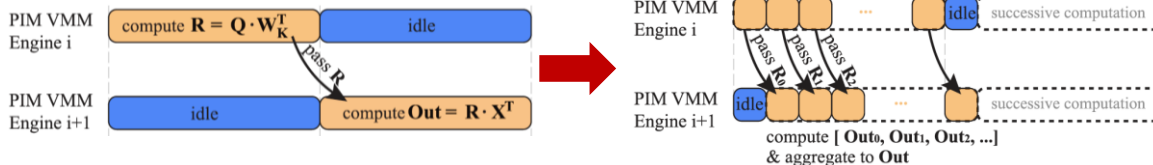
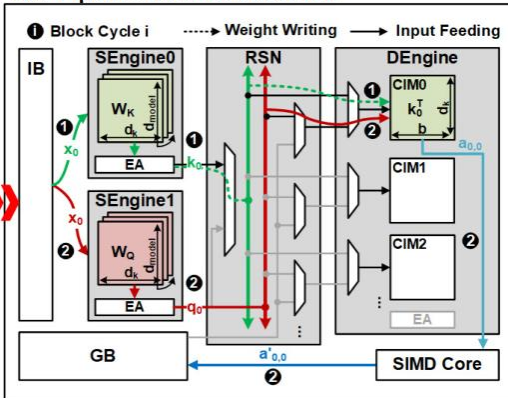
- 将输入拆分为**多块切片**提高计算核心利用率
- Q/K/V线性层**运算错位**以并行进行写入权重和计算

输入切片

QK^T-MM Algorithm Dataflow

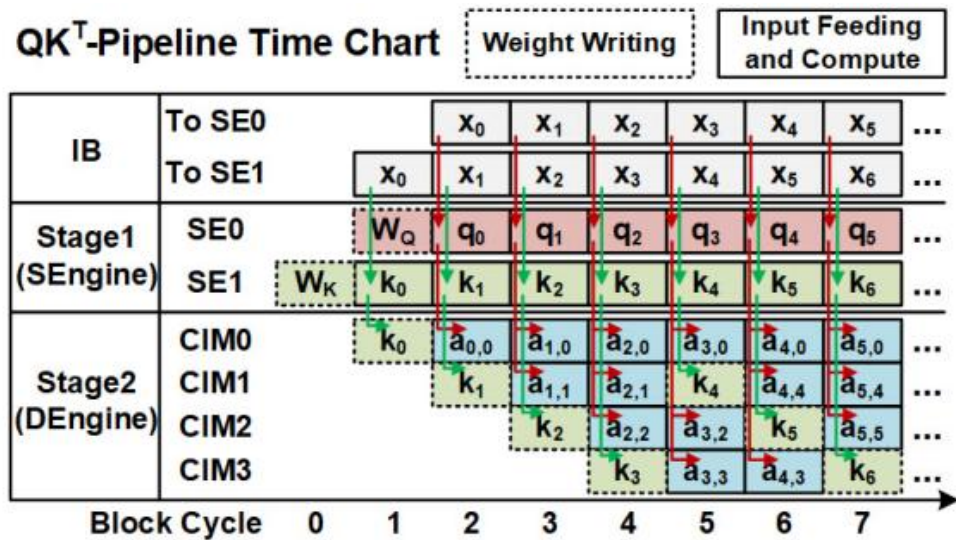


QK^T-Pipeline Hardware Dataflow



流水设计

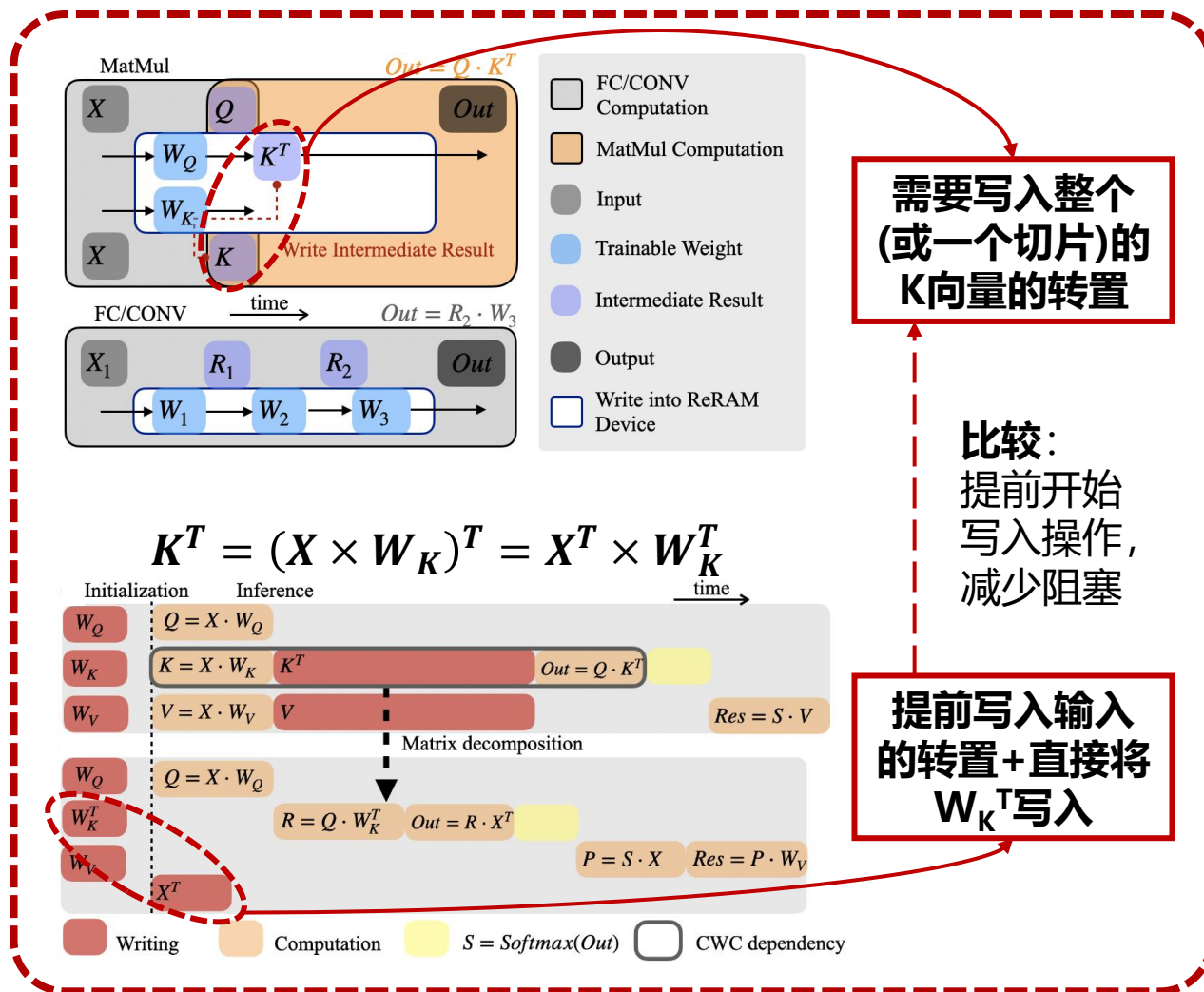
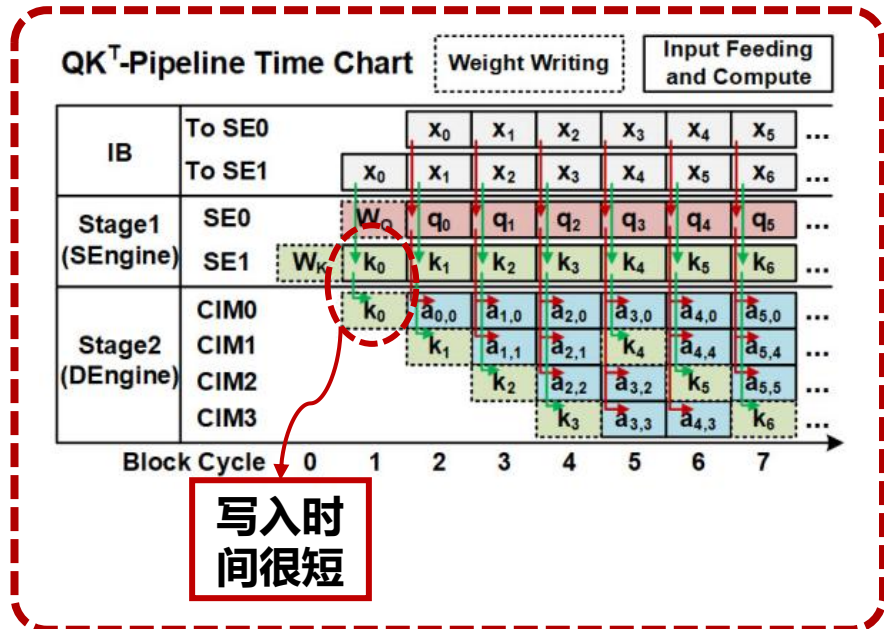
QK^T-Pipeline Time Chart



模块级加速：注意力模块加速

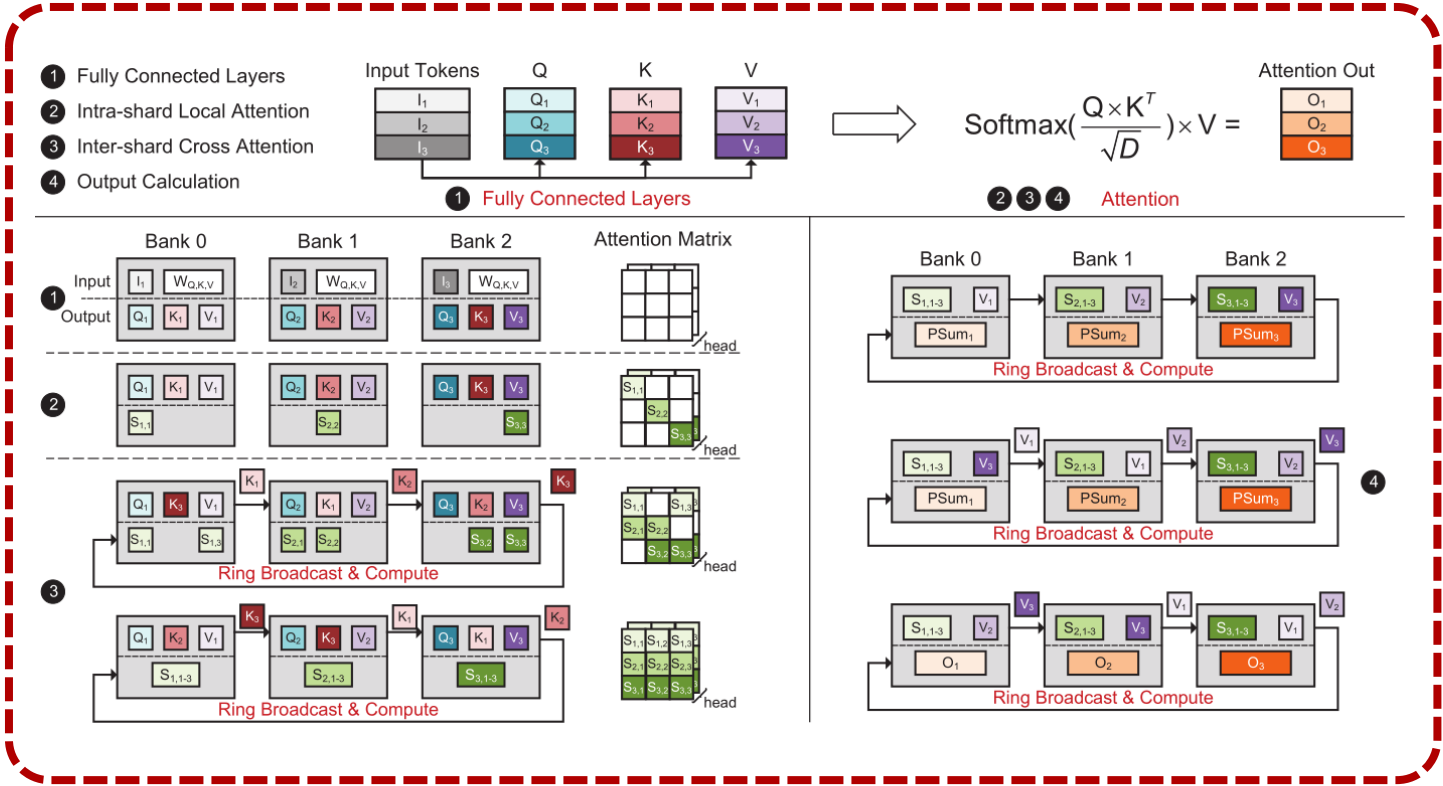
• 流水并行设计

- 通过拆分运算以提前写入



模块级加速：注意力模块加速

- 分阶段并行设计
 - 通过**环形广播**和所有token进行注意力计算



Bank 0

Bank 1

Bank 2

$S_{1,1,3}$

V_1

$P\text{Sum}_1$

$S_{2,1,3}$

V_2

$P\text{Sum}_2$

$S_{3,1,3}$

V_3

$P\text{Sum}_3$

Ring Broadcast & Compute

Bank 0

Bank 1

Bank 2

$S_{1,1,3}$

V_2

$P\text{Sum}_1$

$S_{2,1,3}$

V_1

$P\text{Sum}_2$

$S_{3,1,3}$

V_2

$P\text{Sum}_3$

Ring Broadcast & Compute

Bank 0

Bank 1

Bank 2

$S_{1,1,3}$

V_3

$P\text{Sum}_1$

$S_{2,1,3}$

V_1

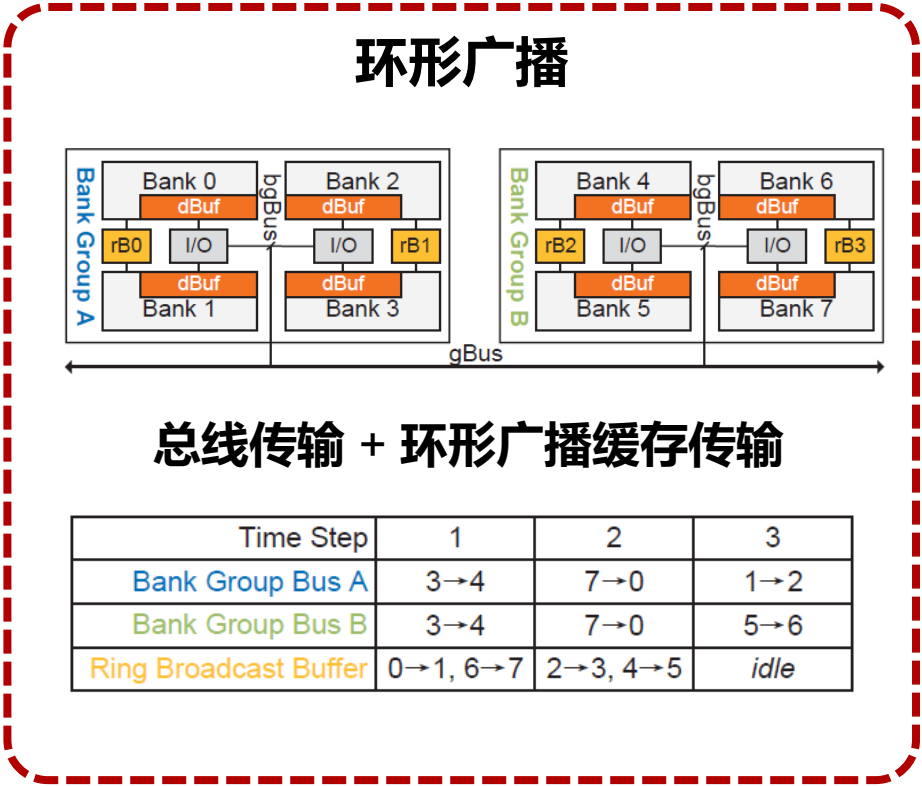
$P\text{Sum}_2$

$S_{3,1,3}$

V_3

$P\text{Sum}_3$

Ring Broadcast & Compute



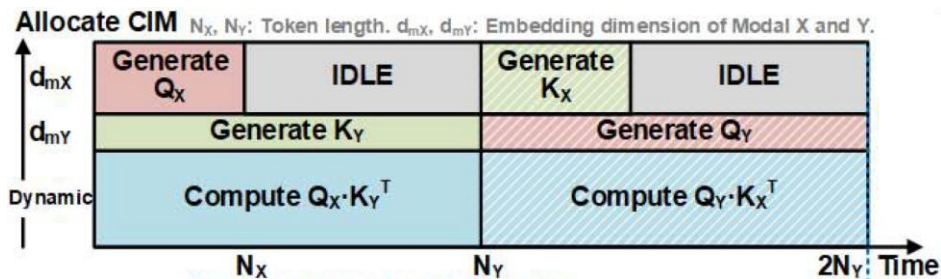
模块级加速：注意力模块加速

• 分阶段并行设计

- 针对**多模态**输入规划权重片上分布
- 提高**计算核心利用率**，降低推断延时

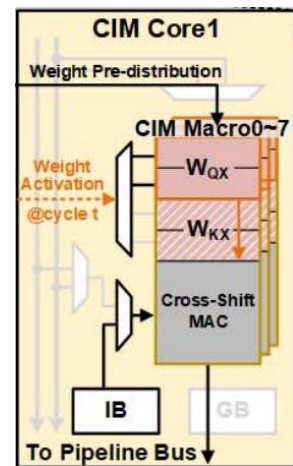
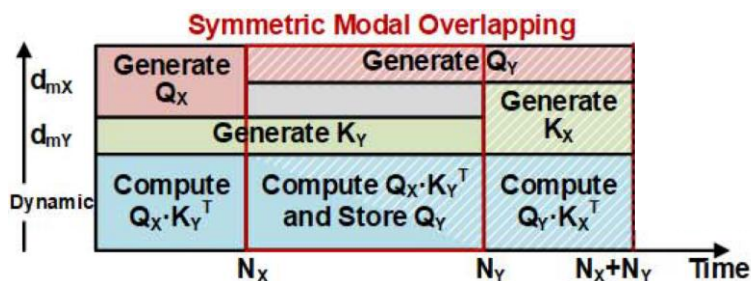
多模态输入的问题

- ❑ **传统权重分布**：不同步骤在不同计算核心实现
- ❑ **多模态输入**：不同阶段运算延时不同
- ❑ 导致多模态输入不匹配时产生**额外延时**



解决方案

- ❑ 计算核心**存储多份权重矩阵**
- ❑ 利用**模态对称性**重叠运算
- ❑ 分配负载提高硬件利用率



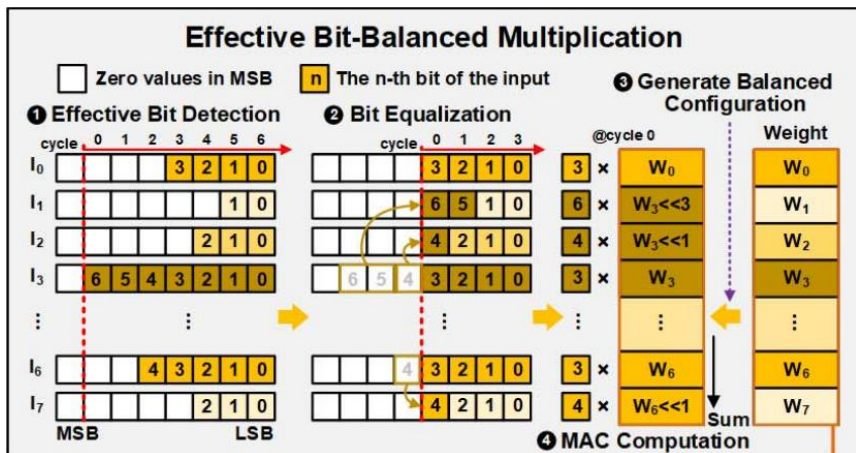
目录

- 研究背景
- 模型级加速
 - 模型压缩
 - 稀疏性处理
- 模块级加速
 - 注意力模块加速
 - 前馈神经网络加速
- 算子级加速
 - 线性算子加速
 - 非线性算子加速
- 总结及展望

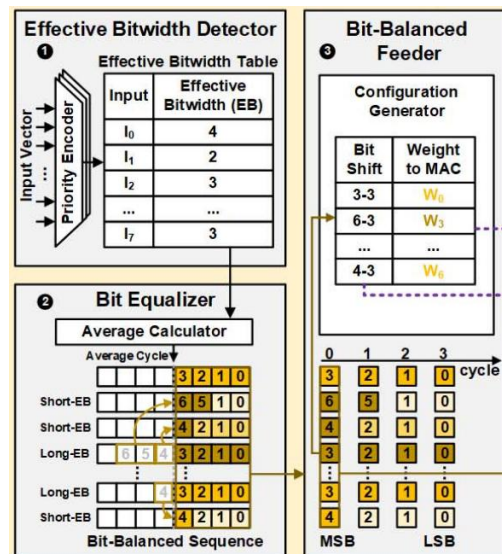
模块级加速：前馈神经网络加速

- **输入特征**——token之间的值差距较大
 - **注意力机制**本身设计目的是使模型更关注关键位置
 - **softmax**函数会放大较大输出，负值等较小输出会接近于0
- 输入前进行**有效位宽均衡**操作

有效位宽均衡



- 输入检测**有效位宽**
- 均衡有效位宽
- **等有效位宽序列**输入
- 根据**均衡表格**配置权重
- 根据均衡表格接收输出



目录

- 研究背景
- 模型级加速
 - 模型压缩
 - 稀疏性处理
- 模块级加速
 - 注意力模块加速
 - 前馈神经网络加速
- 算子级加速
 - 线性算子加速
 - 非线性算子加速
- 总结及展望

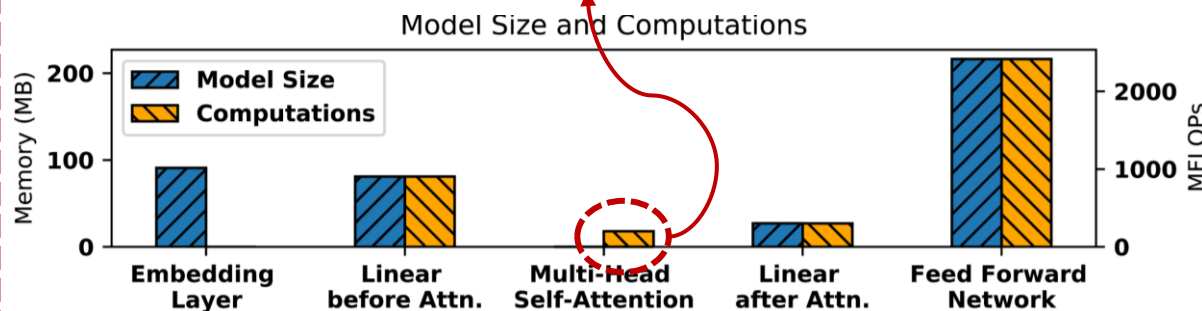
算子级加速：线性算子加速

• 线性算子加速

- 全连接层、注意力机制、ViT的卷积运算、残差等涉及常规乘加操作
- 以**MVM**算子和**MatMul**算子为主
- MVM算子：**权重矩阵固定**，输入向量可变
- MatMul算子：两个输入矩阵都不固定，每次计算前需要**加载权重矩阵**

算子计算量分布

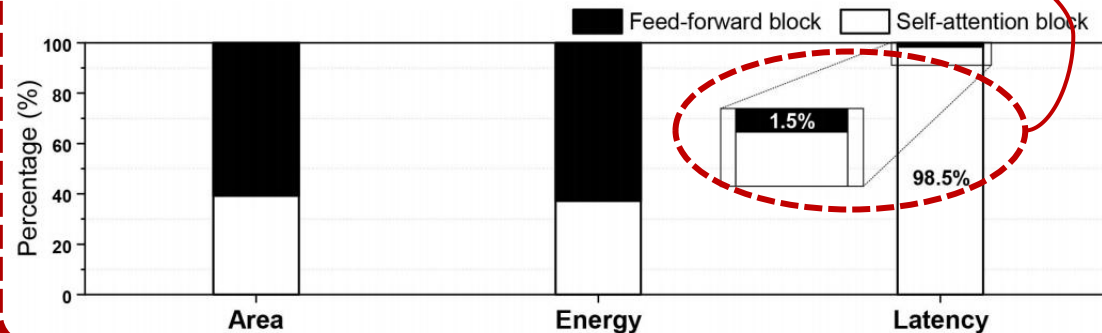
MatMul算子仅占0.5%左右



Ganesh, Prakhar, et al. *TACL* 2021

算子计算量分布

矩阵加载可能占据很高的延时



Kang, Myeonggu, Hyein Shin, and Lee-Sup Kim. *TCAD* 2021

算子级加速：线性算子加速

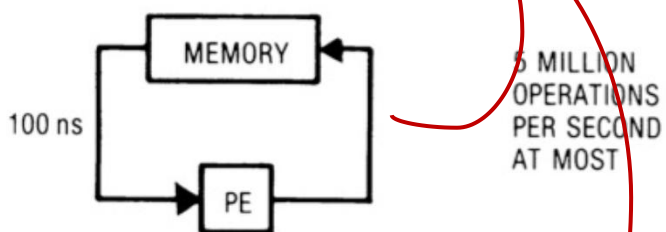
- 存算分离

- 脉动阵列加速乘法累加运算

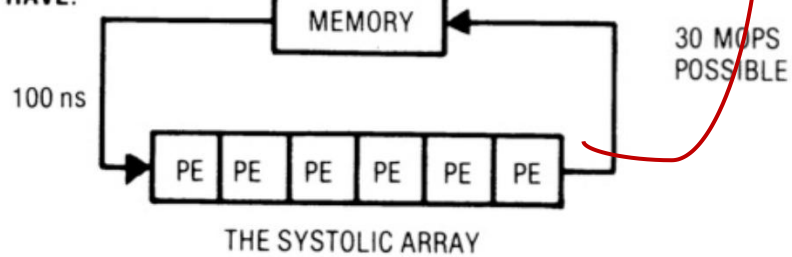
脉动阵列概念

减少存储器调度时间

INSTEAD OF:



WE HAVE:

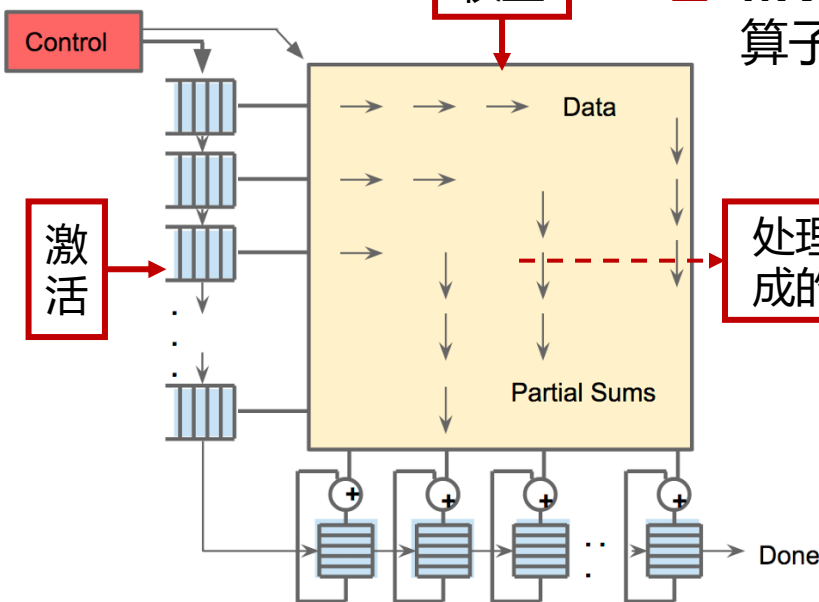


Kung, Hsiang-Tsung. *Computer* 1982

脉动阵列应用

权重

□ MVM/Matmul
算子都适配



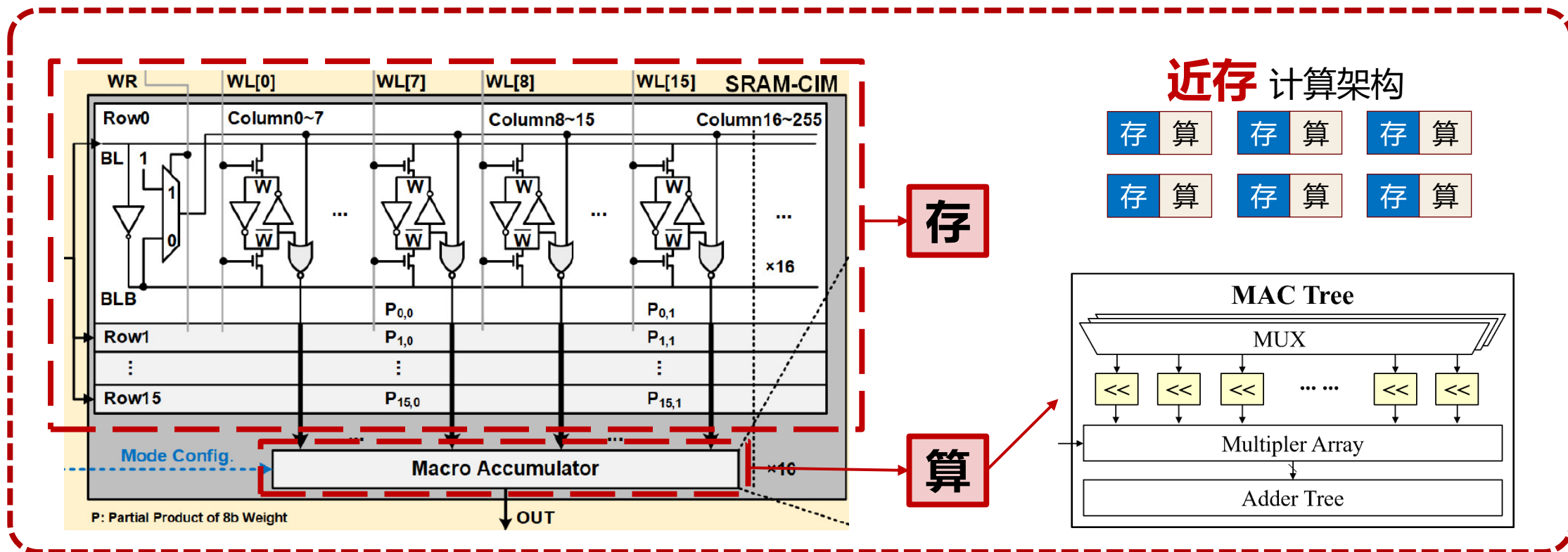
处理器单元构成的二维网格

Jouppi, Norman P., et al. *ISCA* 2017

算子级加速：线性算子加速

• 近存计算

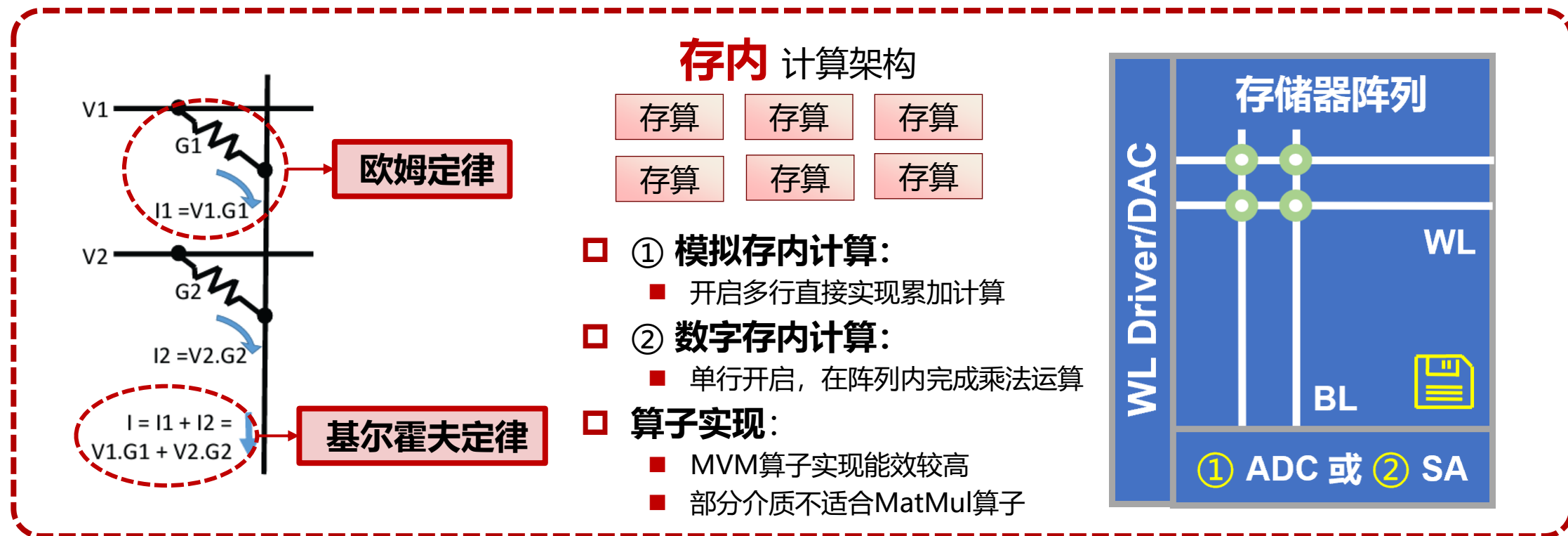
- 存储阵列输出处增加逻辑单元，**减少传输**；DRAM、SRAM等存储介质
- 乘法累加树——写入时间较长时影响MatMul算子实现



算子级加速：线性算子加速

• 存内计算

- 存储阵列内完成乘法累加运算，提高**吞吐**和**能效**；忆阻器、SRAM等存储介质
- 利用交叉阵列实现——写入时间较长时影响MatMul算子实现



目录

- 研究背景
- 模型级加速
 - 模型压缩
 - 稀疏性处理
- 模块级加速
 - 注意力模块加速
 - 前馈神经网络加速
- 算子级加速
 - 线性算子加速
 - 非线性算子加速
- 总结及展望

算子级加速：非线性算子加速

• 非线性算子加速

- 分为需要/不需要统计输入特征的部分

层级	模块级	主要线性算子	非线性算子
embedding	注意力模块	矩阵向量乘法(MVM)	GELU
编码器	前馈神经网络	矩阵矩阵乘法(MatMul)	LayerNorm
解码器	残差模块	残差相加	Softmax

输入特征统计加速

非线性函数加速

GELU

不需要统计输入特征

$$GELU(x) = 0.5x \left(1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$$
$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2) dt$$

LayerNorm

需要统计输入特征

$$\operatorname{LayerNorm}(x) = \frac{x - \mu}{\sigma}$$
$$\mu = \frac{1}{C} \sum_{i=1}^C x_i, \sigma = \sqrt{\frac{1}{C} \sum_{i=1}^C (x_i - \mu)^2}$$

Softmax

需要统计输入特征

$$\operatorname{Softmax}(x) = \frac{\exp(x_i)}{\sum_{j=1}^K \exp(x_j)}$$
$$x = [x_1, \dots, x_K]$$

算子级加速：非线性算子加速

• 非线性算子加速：输入特征统计

- 均值统计/方差统计/指数统计

□ 均值统计

- 记录累和值和数量
- 每次数据流经过时更新

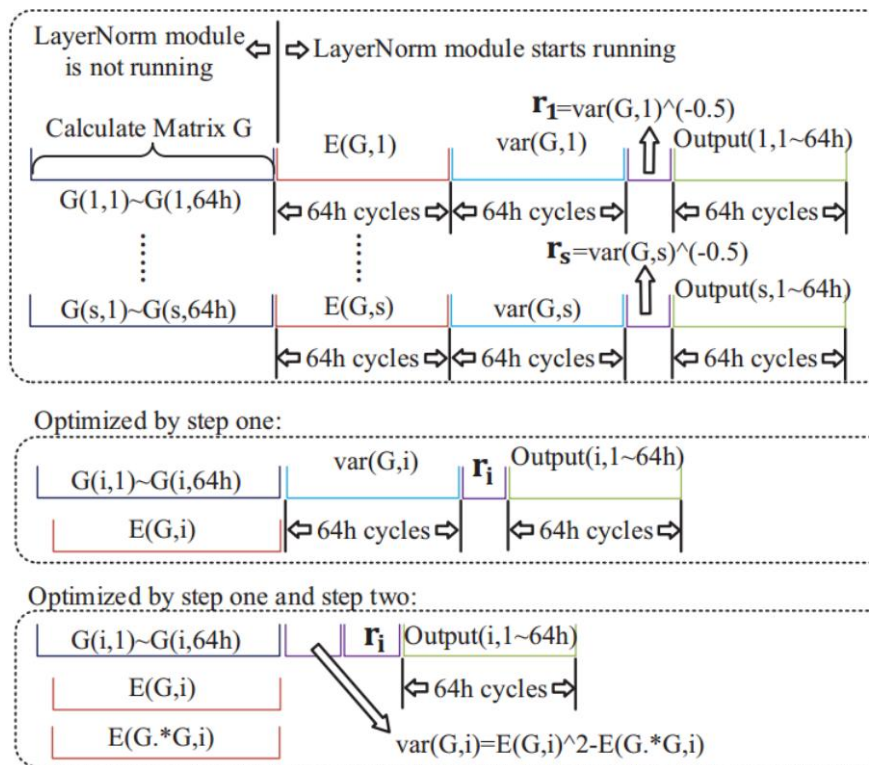
□ 方差统计

- 数学重构为 $var = \mu^2 - \frac{1}{C} \sum_{i=1}^C x_i^2$
- 记录平方值和数量
- 每次数据流经过时更新

□ 指数统计

- 需要记录所有数据的指数
- 在数据输出时完成

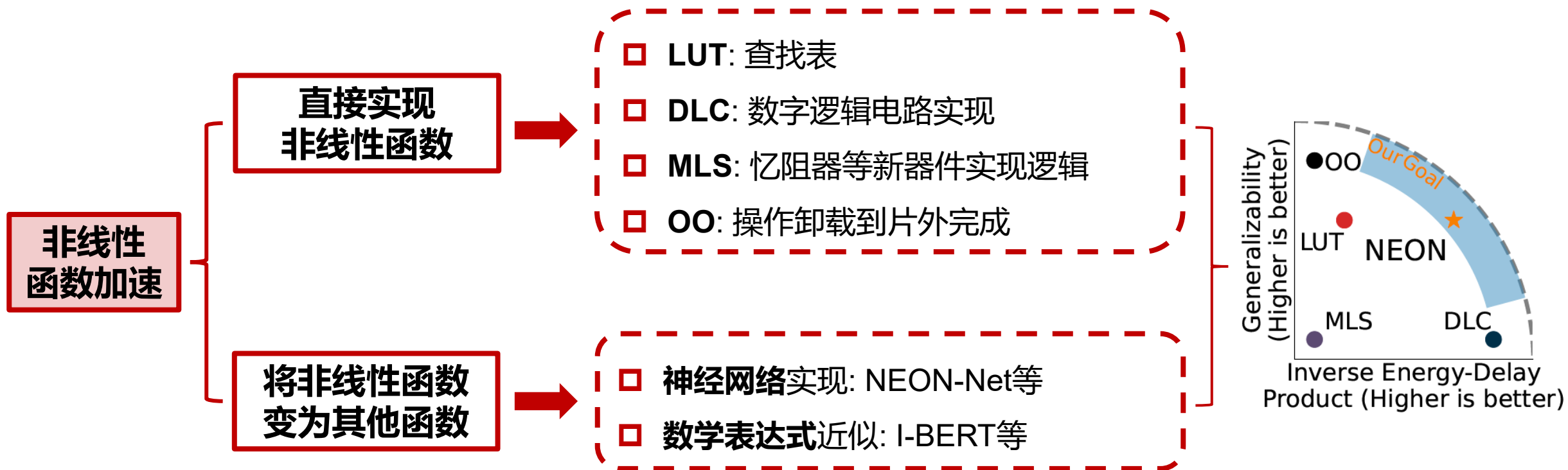
数据流上
进行加速



算子级加速：非线性算子加速

• 非线性算子加速：非线性函数加速

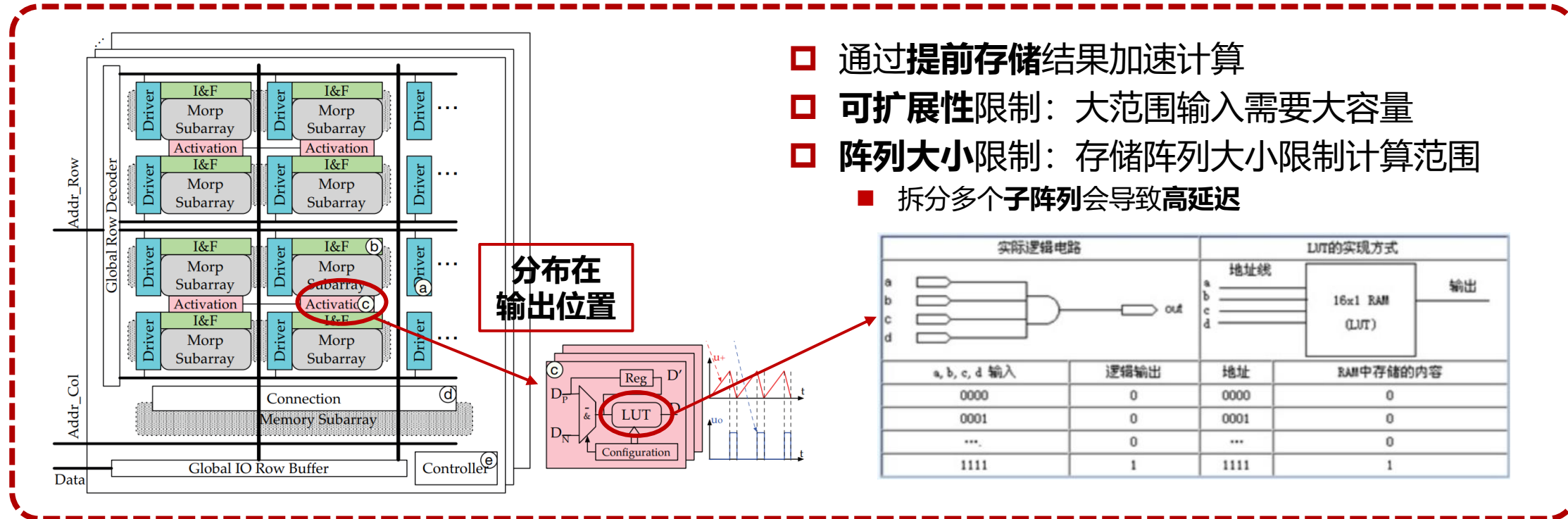
- 两种思路：将非线性函数变为线性函数/直接实现非线性函数



算子级加速：非线性算子加速

• 非线性函数实现：LUT(查找表)

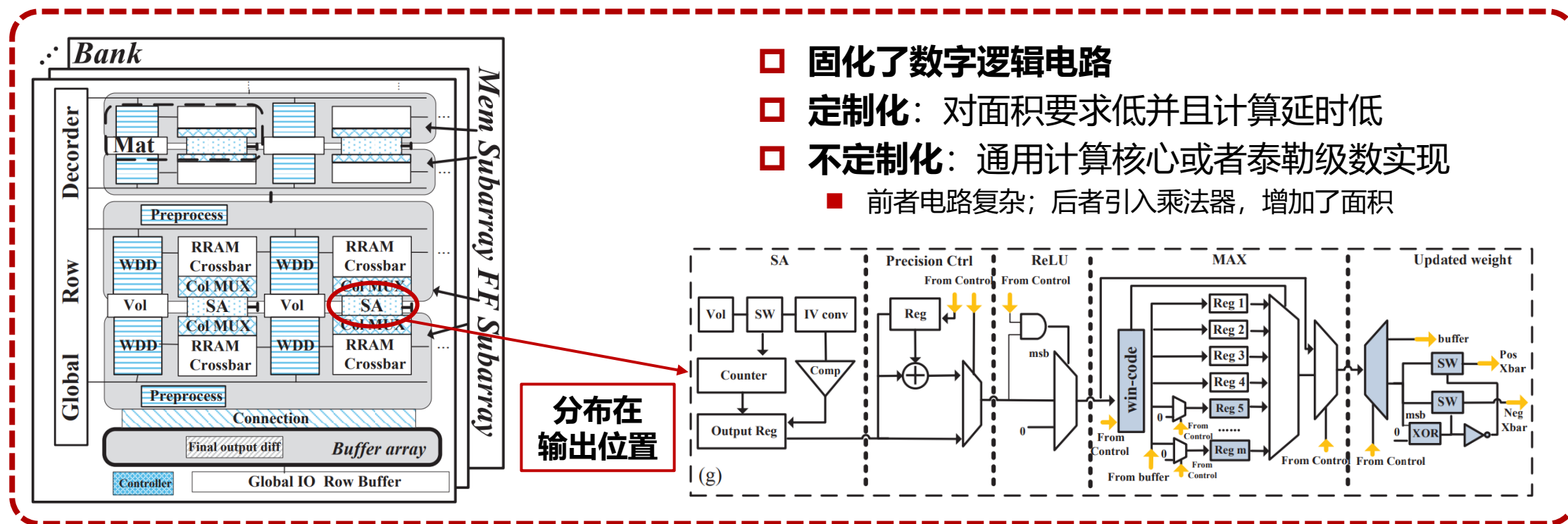
- 优点：同样的电路结构可以用于实现不同的非线性函数，具有一定的灵活性
- 缺点：大范围输入需要大容量存储阵列



算子级加速：非线性算子加速

- 非线性函数实现：DLC(数字逻辑电路实现)

- 优点：计算过程延迟低+使用成熟CMOS工艺对面积要求较低
- 缺点：固化数字电路导致灵活度低+数字设计需要考虑静态功耗

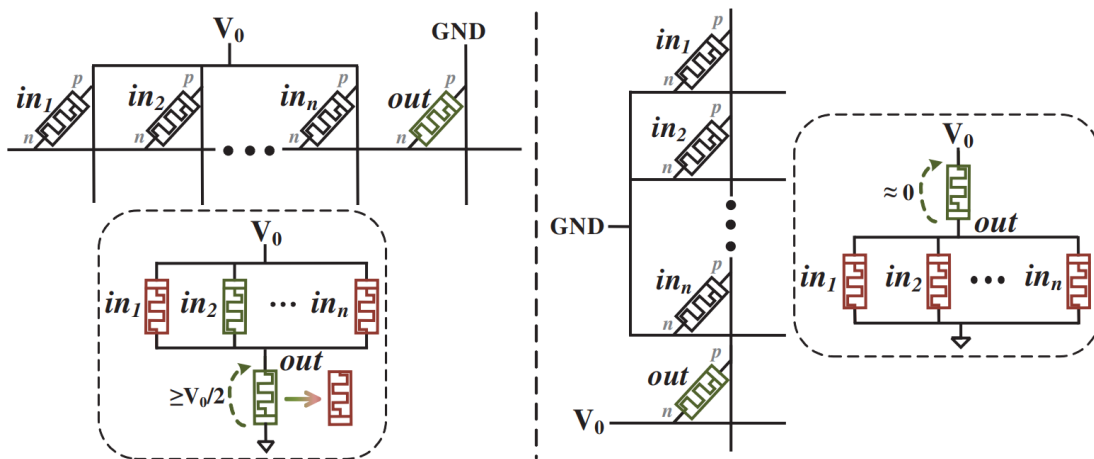


算子级加速：非线性算子加速

• 非线性函数实现：MLS&OO

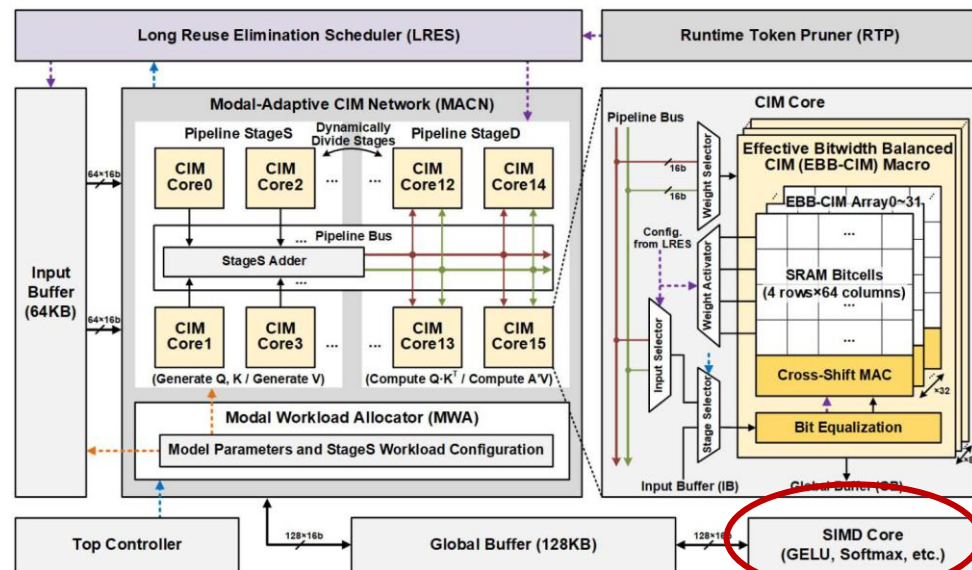
MLS(新器件实现逻辑)

- ❑ 通过**器件分压**执行**写入**操作
- ❑ **优点**：**静态功耗**低+**计算单元面积**小
- ❑ **缺点**：**写入能耗**高+**定制化灵活度**低+需要很多器件单元**成本高**



〇〇(操作片外卸载)

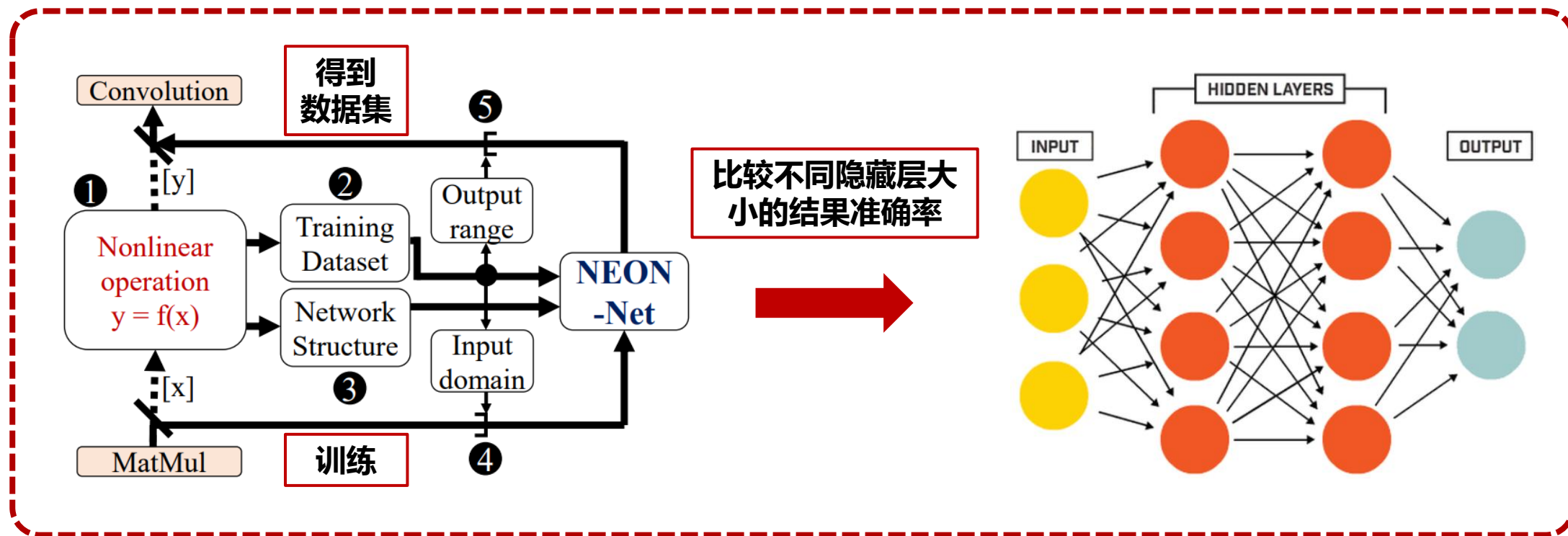
- ❑ 数据传输到片外完成
- ❑ **优点：** 直接利用通用CPU
- ❑ **缺点：** **数据移动**过多影响能耗和延时



算子级加速：非线性算子加速

• 非线性函数转换：神经网络实现

- 利用存算一体等对神经网络加速比较高的实现方式
- 通过泛函数逼近定理精确地模拟非线性函数



算子级加速：非线性算子加速

• 非线性函数转换：数学表达式近似

GELU

- ❑ 误差函数近似
- ❑ 根据输入范围近似

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2) dt$$

$$= \operatorname{sgn}(x)[a(\operatorname{clip}(|x|, \max = -b) + b)^2 + 1]$$

LayerNorm

- ❑ 迭代实现开根号效果
- ❑ 重复 $x_{i+1} = \left\lfloor \left(x_i + \frac{n}{x_i}\right) / 2 \right\rfloor$

Softmax

- ❑ 和最大值进行归一化

$$\begin{aligned} \operatorname{Softmax}(x) &= \frac{\exp(x_i)}{\sum_{j=1}^k \exp(x_j)} = \frac{\exp(x_i - x_{\max})}{\sum_{j=1}^k \exp(x_j - x_{\max})} \\ &= \exp[x_i - x_{\max} - \ln(\sum_{j=1}^k \exp(x_j - x_{\max}))], \end{aligned}$$

缩小输入范围

- ❑ 指数函数的多项式近似

$$\exp(x) = 0.3585(x + 1.353)^2 + 0.344$$

避免除法器

- ❑ 指数函数/对数函数拆分2的基数

$$e^y = 2^{u+v} = 2^u \times 2^v = \begin{cases} 2^v \ll u & u > 0 \\ 2^v \gg u & u \leq 0 \end{cases}, u = \lfloor y \times \log_2 e \rfloor, v = y \times \log_2 e - u$$

小数运算+移位

$$\ln F = \ln 2 \times \log_2 F = \ln 2(w + \log_2 k) = \ln 2(k - 1 + w), F = 2^w + k$$

目录

- 研究背景
- 模型级加速
 - 模型压缩
 - 稀疏性处理
- 模块级加速
 - 注意力模块加速
 - 前馈神经网络加速
- 算子级加速
 - 线性算子加速
 - 非线性算子加速
- 总结及展望

总结及展望



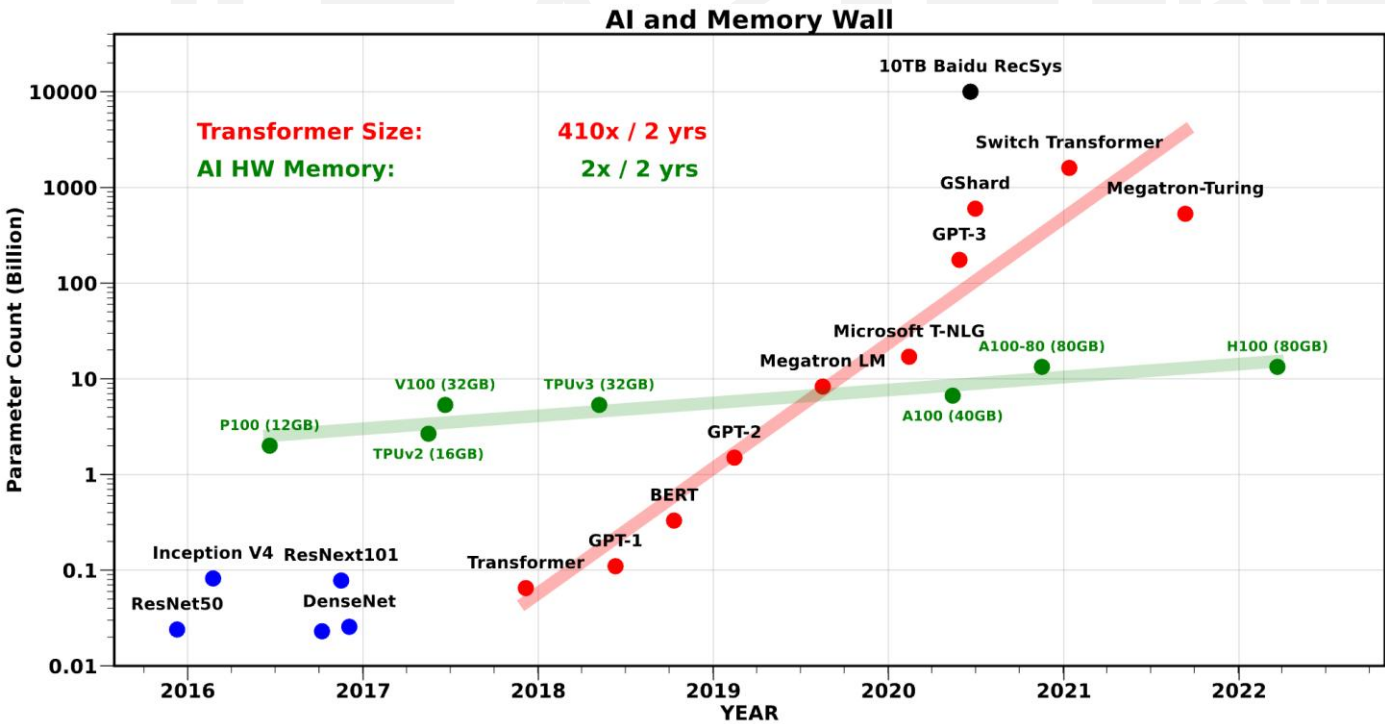
目录

CONTENTS



- 01. 典型AI芯片架构**
- 02. AI芯片软硬件协同设计**
- 03. AI大模型芯片设计**
- 04. 存算一体AI芯片架构**

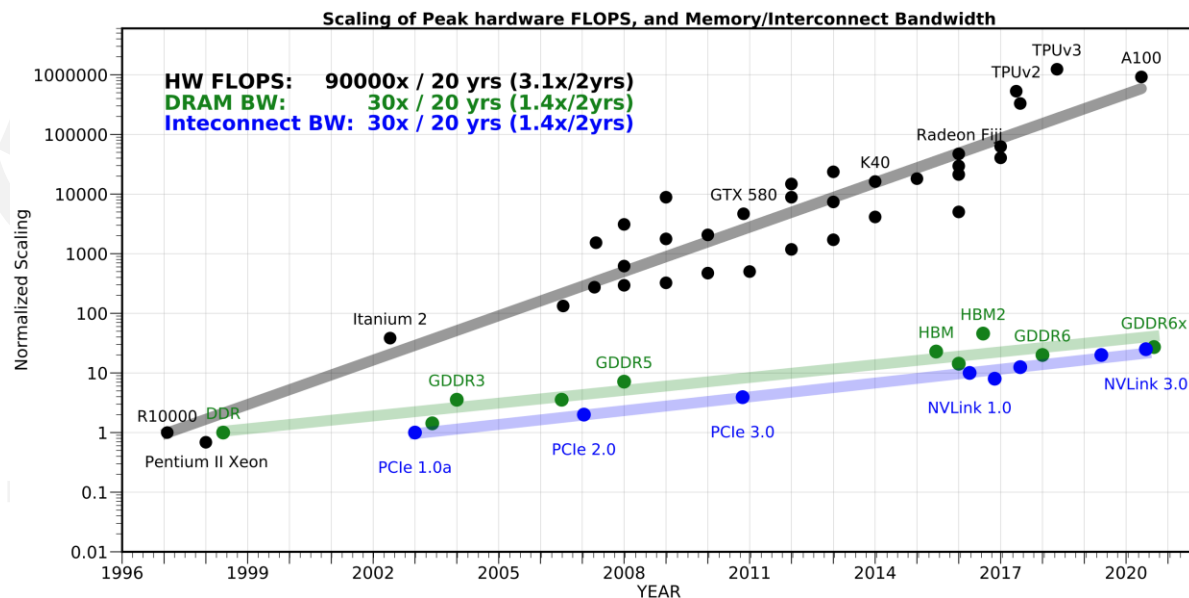
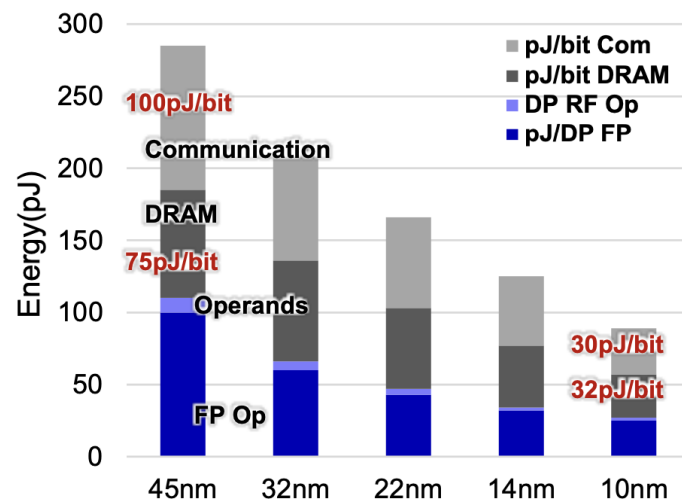
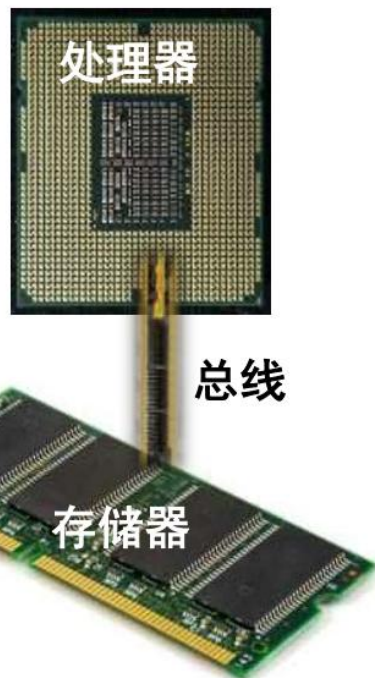
- 随着人工智能的快速发展，神经网络模型**规模指数级增长**，对人工智能处理器的计算、存储和传输带宽需求也显著提升



时间	模型	模型参数量 (GB)	模型算力需求 (TFLOPs-day)
2012	AlexNet	10^{-2}	10^{-3}
2015	ResNets	10^{-1}	10^{-1}
2017	Transformer	1	10^1
2020	GPT-3	10^2	10^6
2023	ChatGPT	10^3	10^7

https://github.com/amirgholami/ai_and_memory_wall

- 然而，现有的冯诺依曼架构采用存储和计算分离的架构，面临“存储墙”和“功耗墙”瓶颈
 - 内存带宽/互连带宽增长与算力增长速度不一致 ($30\times$ vs $90,000\times$)，特别是对于先进节点
- 如何克服层“存储墙”瓶颈，进一步提升AI处理器算力和能效成为重要问题



https://github.com/amirgholami/ai_and_memory_wall

• 回顾：什么样的算子更容易受到“存储墙”影响？

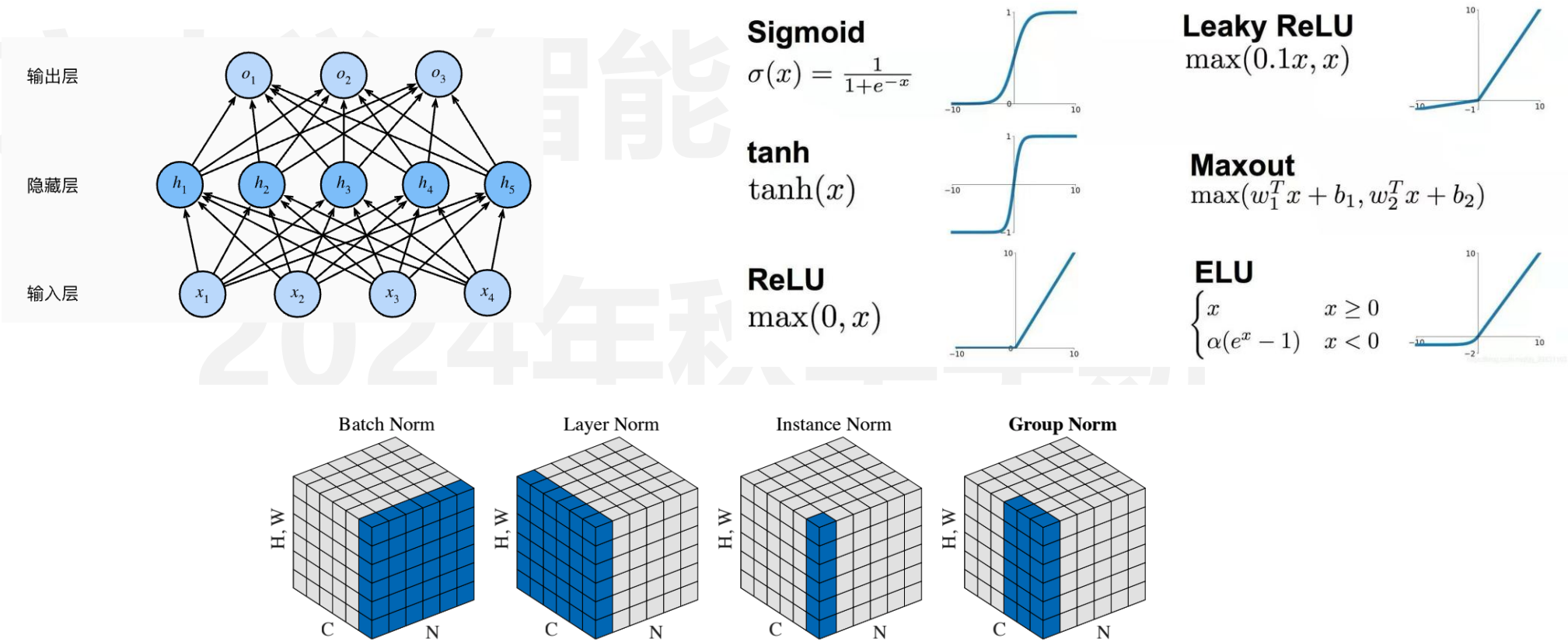
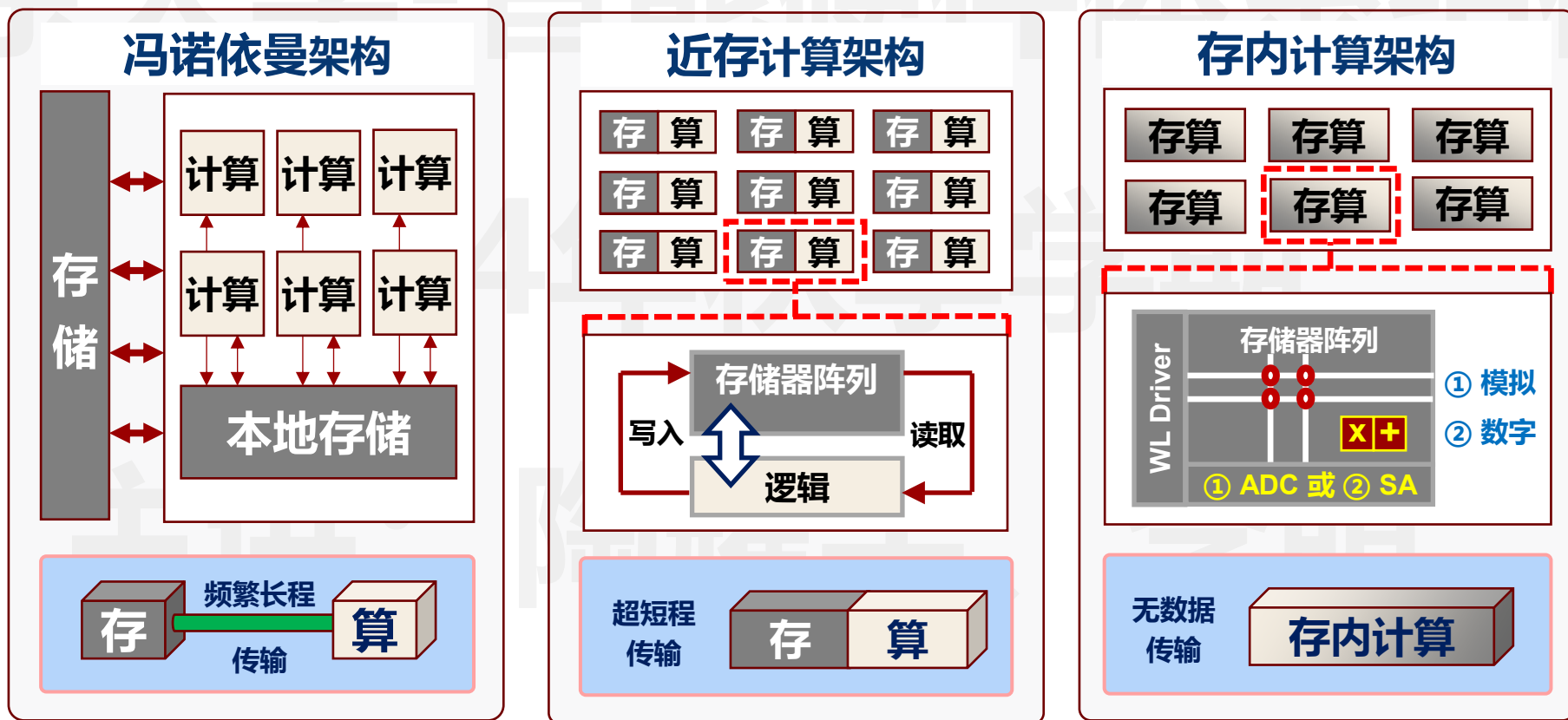


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

存算一体新架构

- 针对于访存瓶颈的算子，**存算一体技术**成为打破算力瓶颈的重要途径
- **近存计算** (NDP/PNM)：存储阵列一般无需改动，只提供数据存储功能，计算模块安放在阵列附近
- **存内计算** (PIM)：存储器件可以参与计算操作，这通常意味着存储阵列需要改动来支持计算



- 思考：“存储墙”主要在哪里？
- 从学术研究（1969-2016）到商业化探索（2016至今），DRAM近存计算得益于工艺能力的进步和AI应用的驱动

• Kautz, “Cellular Logic-in-Memory Arrays”, IEEE TC 1969

IEEE TRANSACTIONS ON COMPUTERS, VOL. C-18, NO. 8, AUGUST 1969

Cellular Logic-in-Memory Arrays

WILLIAM H. KAUTZ, MEMBER, IEEE

Abstract—As a direct consequence of large-scale integration, many advantages in the design, fabrication, testing, and use of digital circuitry can be achieved if the circuitry can be arranged in a two-dimensional, recursive, or cellular, array of identical elementary networks, or cells. When a small amount of storage is included in each cell, the same array may be regarded either as a logically enhanced memory array, or as a logic array whose elementary gates and connections can be “programmed” to realize a desired logical behavior.

In this paper the specific engineering features of such cellular logic-in-memory (CLIM) arrays are discussed, and one such special-purpose array, a cellular sorting array, is described in detail to illustrate how these features may be achieved in a particular design. It is shown how the cellular sorting array can be employed as a single-address, multistage memory that keeps in order all words stored within it. It can also be used as a content-addressed memory, a pushdown memory, a buffer memory, and (with a lower logical efficiency) a programmable array for the realization of arbitrary switching functions. A second version of a sorting array, operating on a different sorting principle, is also described.

Index Terms—Cellular logic, large-scale integration, logic arrays, logic in memory, push-down memory, sorting, switching functions.

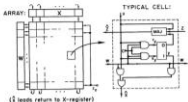


Fig. 1. Cellular sorting array I.

Startup plans to embed processors in DRAM

October 13, 2016 // By Peter Clarke

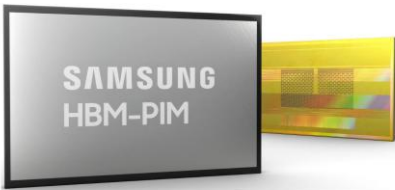
Email print Share in Share reddit

UPMEM PIM-DIMM

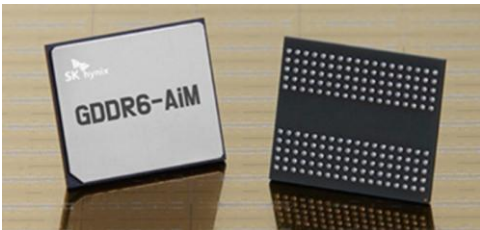


DDR4 2400 R-DIMM module

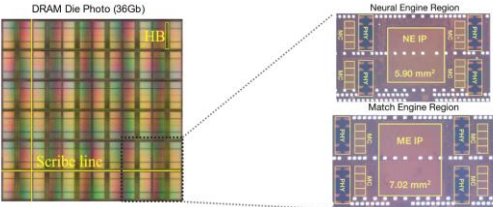
三星PIM-HBM, AxDIMM



海力士GDDR6-PIM



阿里 HB-PNM



1969

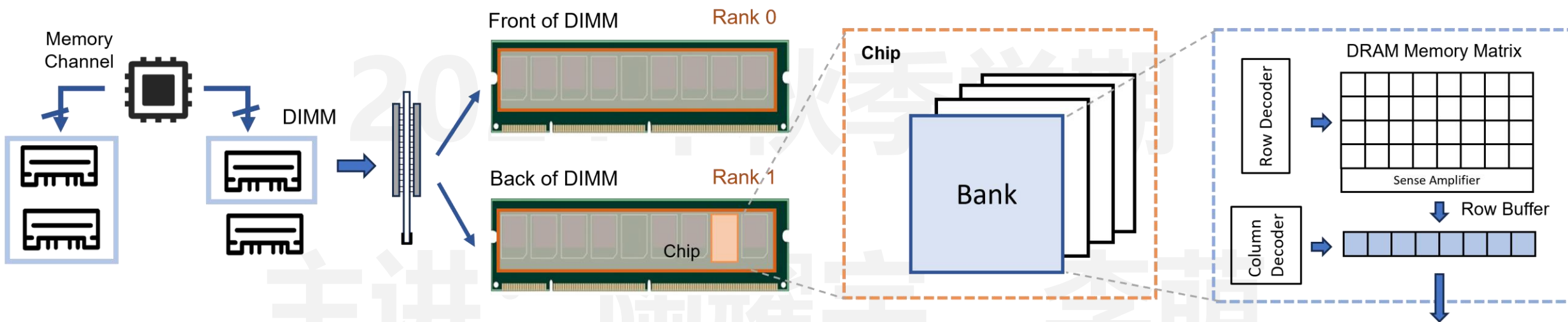
2016

2019

2021

2022

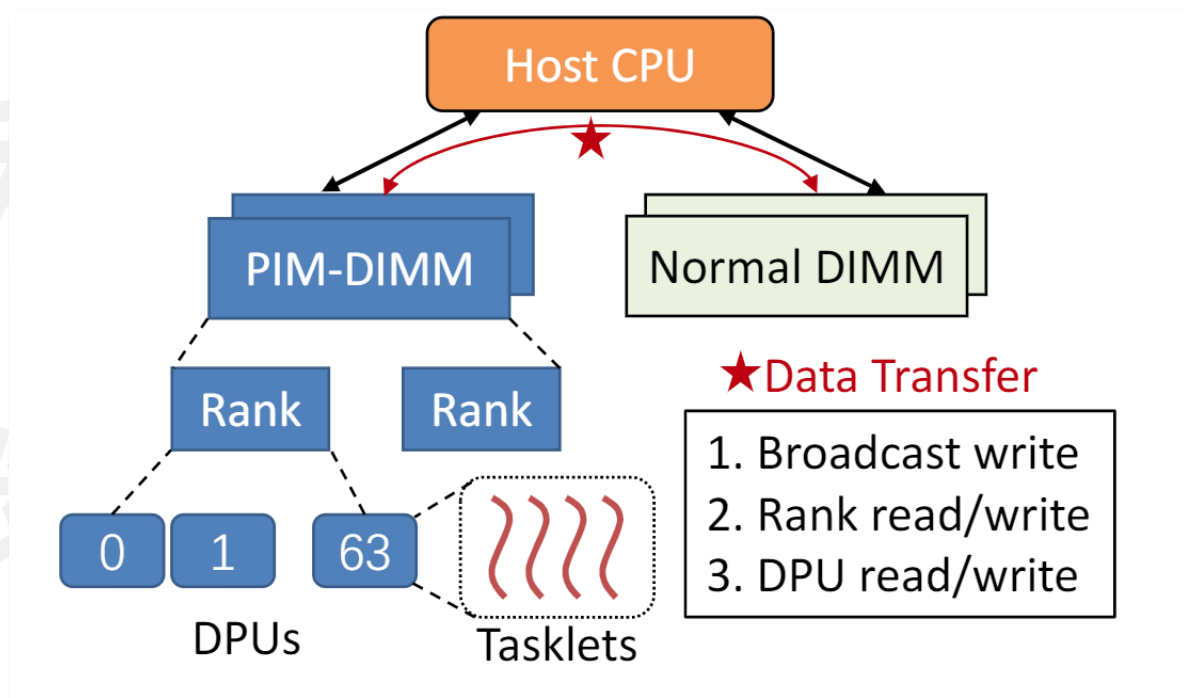
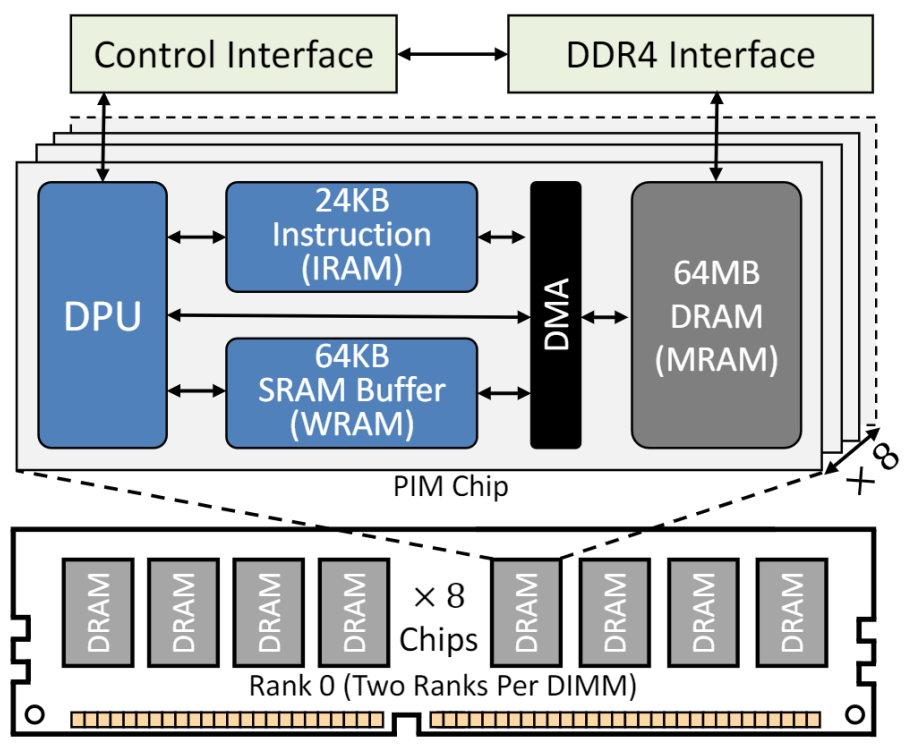
- DRAM的内存架构和组织形式
 - DRAM channel, DIMM, rank, chip, bank, column/row decoder
- 内部带宽 vs 外部带宽



Reference: Prof. Onur Mutlu's computer architecture course

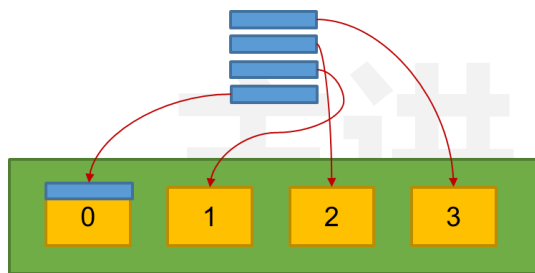
• UPMEM PIM-DIMM架构

- 单条8GB, 128个计算核心 (DPU), 64KB便签存储器
- 内部带宽约为**100 GB/s** (DDR4带宽 < 20 GB/s)

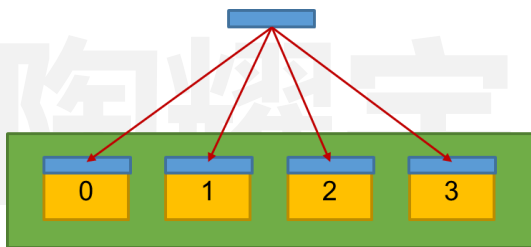


- UPMEM PIM-DIMM系统集成

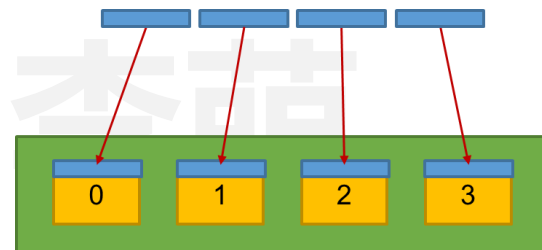
- 为PIM-DIMM分配了独立的物理地址空间
- 数据传入包括serial、parallel和广播模式
- 资源分配/调度:
 - Tasklet (线程): 一个DPU 可以运行24个线程, 有各自的stack
 - DPU: 64MB Main RAM, DPU之间无法直接进行数据传递
 - Rank: 64个DPU, 可以进行并行CPU-DPU数据传输



CPU-DPU serial



CPU-DPU parallel



CPU-DPU broadcast

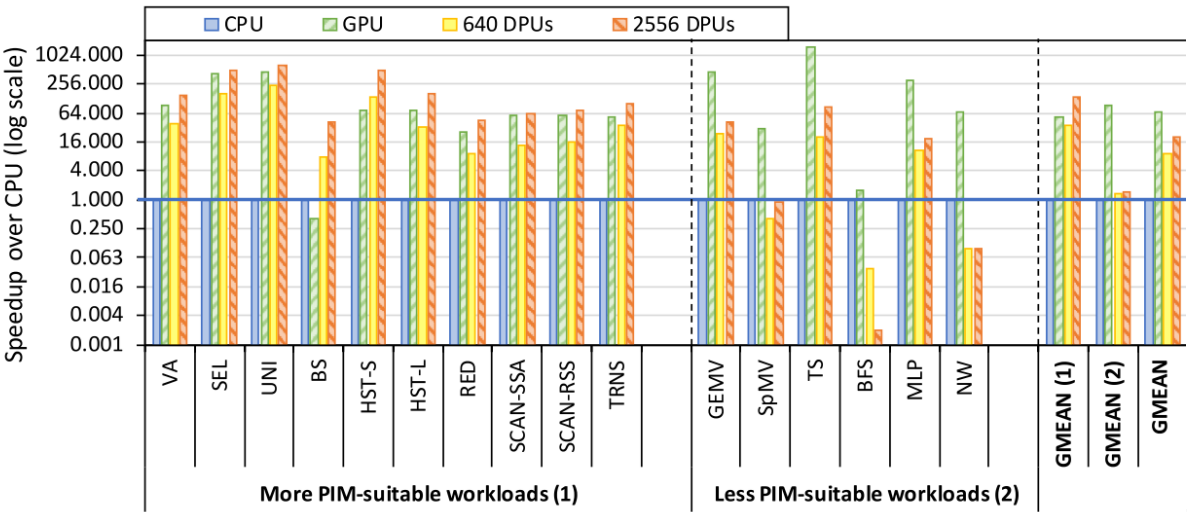
性能测试

Table 4. Evaluated CPU, GPU, and UPMEM-based PIM Systems.

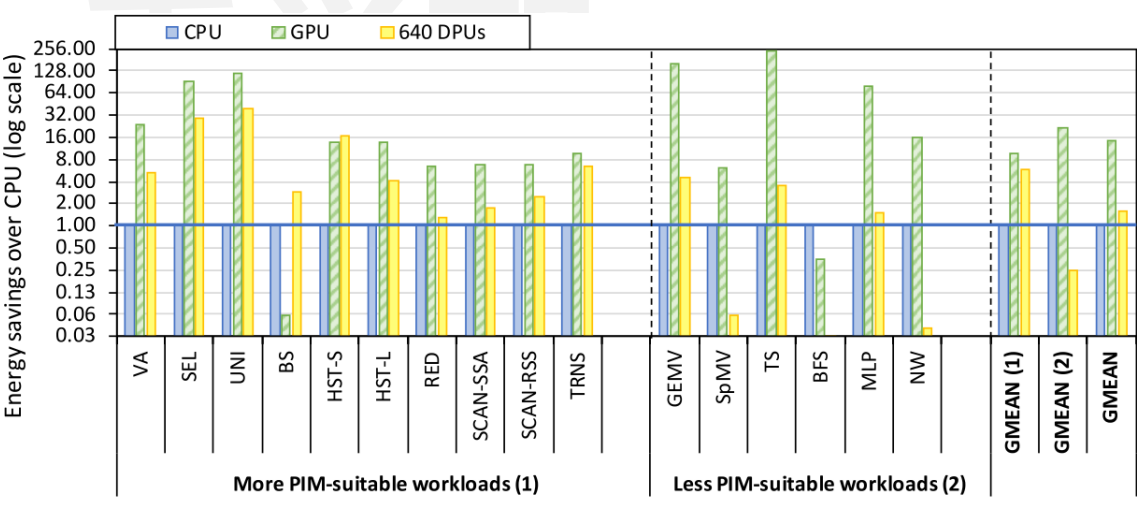
System	Process Node	Processor Cores			Memory		TDP
		Total Cores	Frequency	Peak Performance	Capacity	Total Bandwidth	
Intel Xeon E3-1225 v6 CPU [106]	14 nm	4 (8 threads)	3.3 GHz	26.4 GFLOPS*	32 GB	37.5 GB/s	73 W
NVIDIA Titan V GPU [192]	14 nm	80 (5,120 SIMD lanes)	1.2 GHz	12,288.0 GFLOPS	12 GB	652.8 GB/s	250 W
2,556-DPU PIM System	2x nm	2,556 ⁹	350 MHz	894.6 GOPS	159.75 GB	1.7 TB/s	383 W [†]
640-DPU PIM System	2x nm	640	267 MHz	170.9 GOPS	40 GB	333.75 GB/s	96 W [†]

*Estimated GFLOPS = 3.3 GHz × 4 cores × 2 instructions per cycle.

[†]Estimated TDP = $\frac{\text{Total DPUs}}{\text{DPUs/chip}} \times 1.2 \text{ W/chip}$ [52].



性能对比



功耗对比

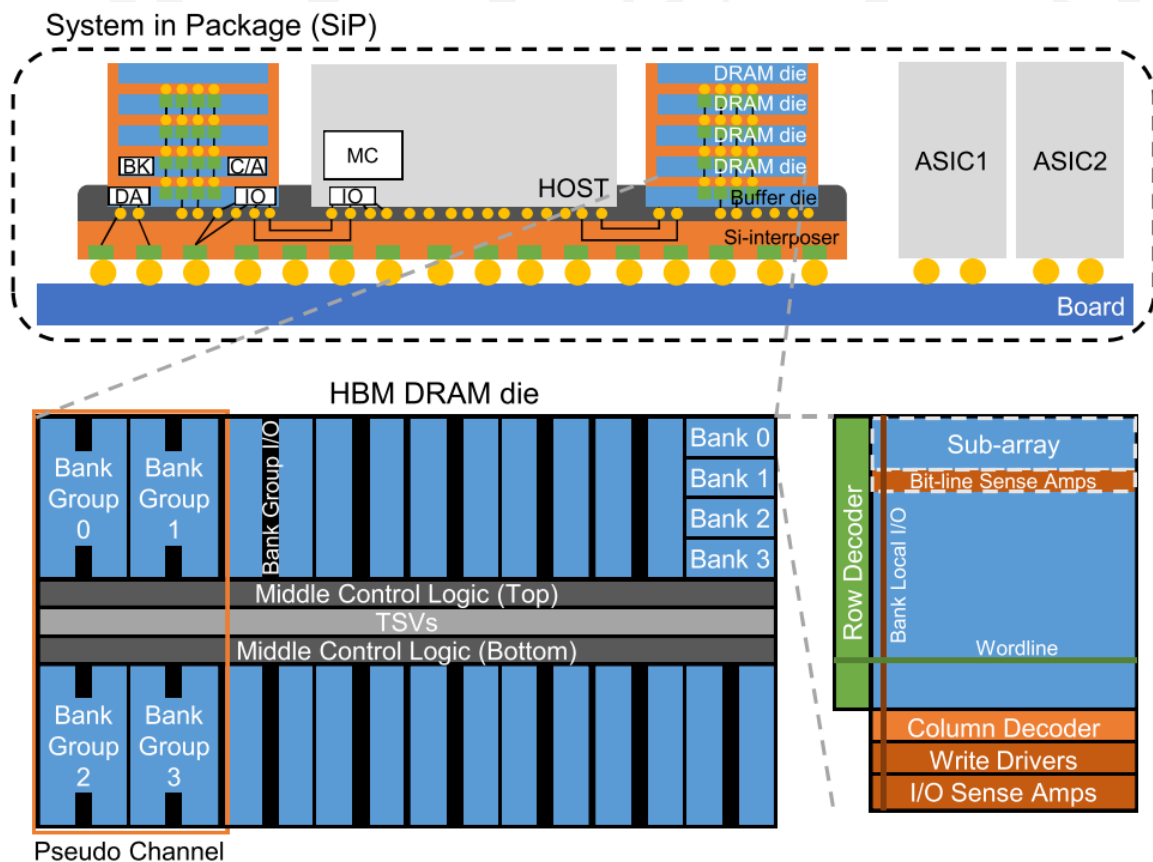
- 优势

- Scalable memory bandwidth, 相对于HBM等容量更大 (同代对比)
- 通用编程模型, 且兼容目前的server系统

- 缺点

- 无法当作普通main-memory 使用: CPU 和Memory 都有内存控制器, 协调困难, 存在 **Memory interleaving**、Cache coherence 问题和系统支持问题 (虚拟内存等)
- 数据传输性能是瓶颈, 难以处理需要频繁同步的应用
- 应用切分困难: 总共包含 2048个DPU, 64MB local DRAM/DPU, DPU之间通信代价高
- 无SIMD / FP 单元, 浮点算力太低, 在深度学习应用场景受限

- **HBM**的带宽对于访存密集型深度学习算子仍然存在不足

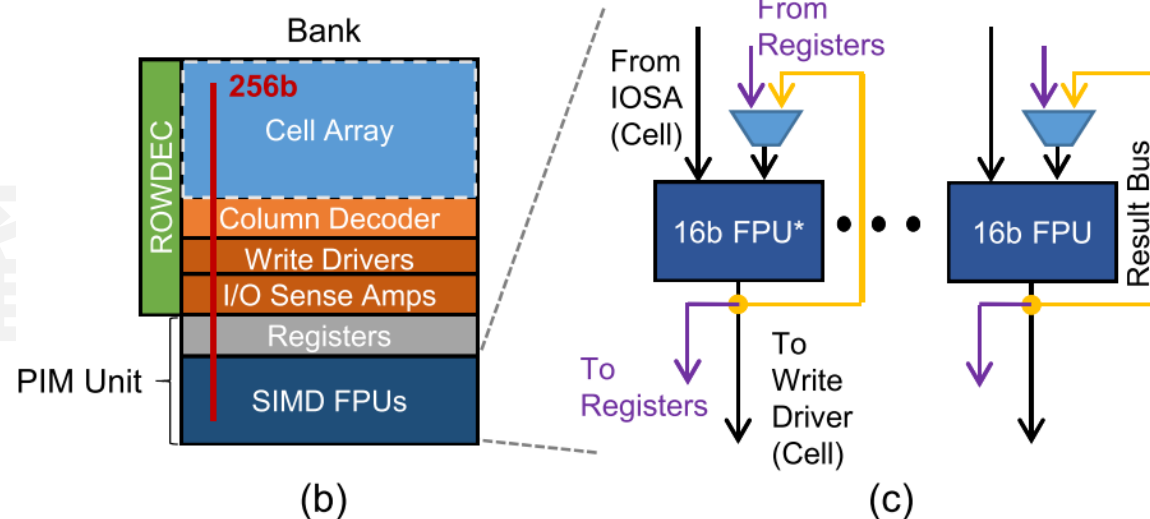
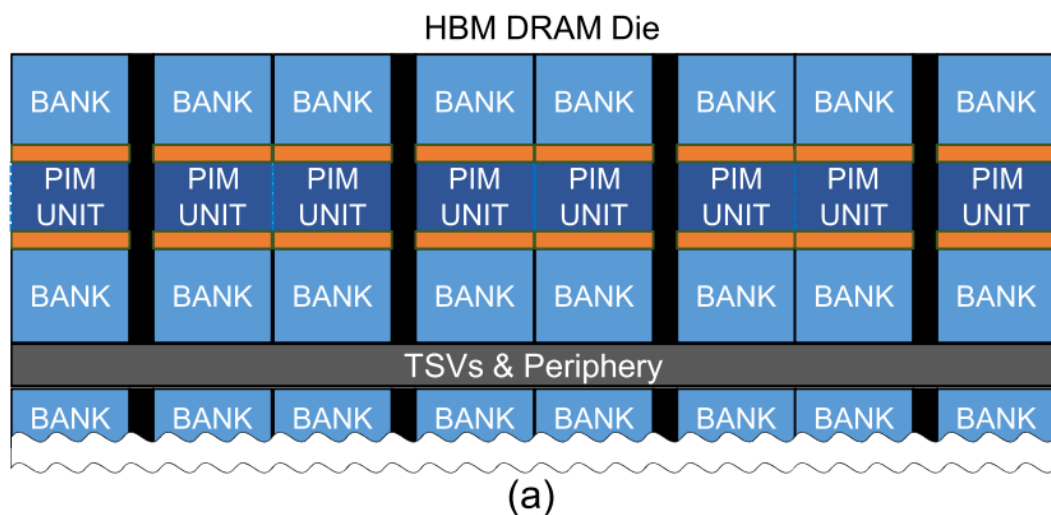


HBM2 架构:

- 4 pseudo channels (pCH) per Die
- 4 bank group per pCH
- 4 banks per bankgroup
- 256-bit data block over 4 64-bit bursts over one pCH
- 256GB/s (1GHz IO)

Samsung's PIM-HBM

- **设计目标**：支持PIM和普通HBM模式，维持原本的DRAM bank和阵列设计
- 在Bank的I/O boundary处设置PIM单元
 - **Bank row数量减半**，两个bank共享PIM单元，每个PIM单元内包含16 x 16bit的SIMD单元
- PIM单元由标准的DRAM column commands (RD、WD) 控制
 - 和UPMEM相似，指令提前load进memory，用RD指令操作数地址，一个channel所有bank可以并行读出4096个bit，RD指令同时激活PIM单元的计算



- PIM-HBM 三种操作模式

- Single Bank Mode

- 标准DRAM 模式

- 每次访问一个channel中的一个Bank

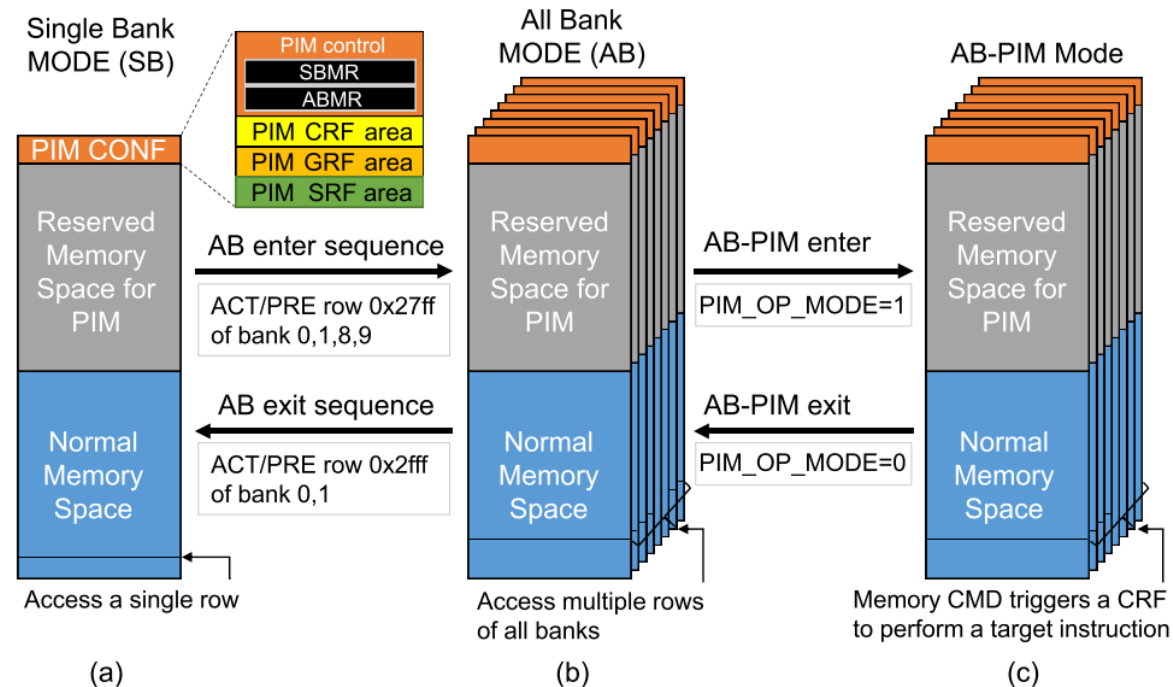
- All Bank Mode

- 一个DRAM 指令可以同时控制多个Bank

- 8x higher on-chip bandwidth

- All Bank PIM Mode

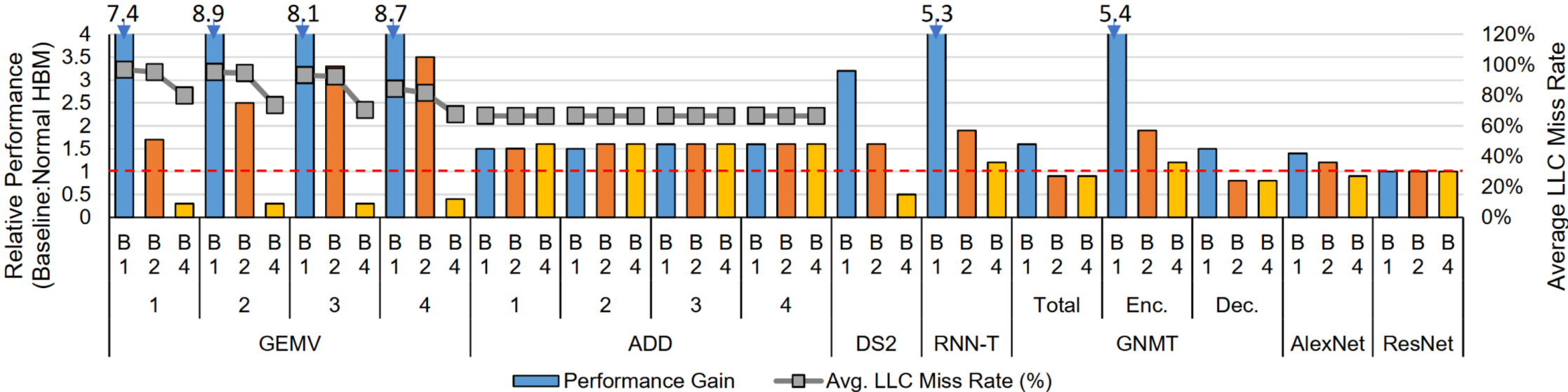
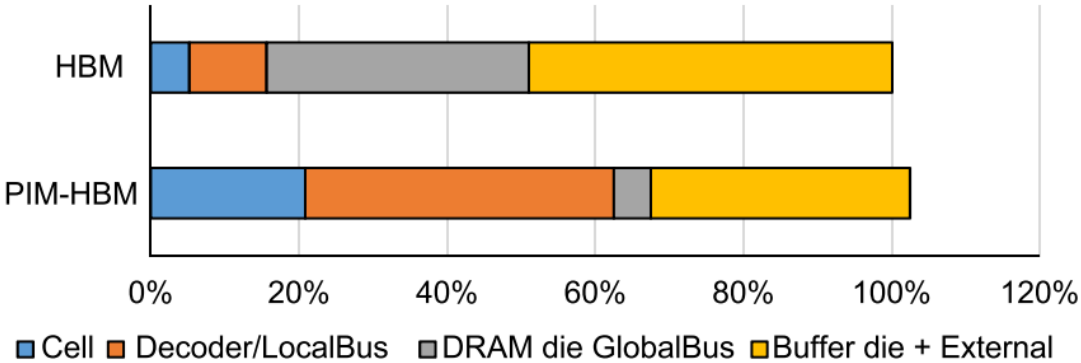
- 通过写high row地址以及PIM mode 寄存器来切换mode



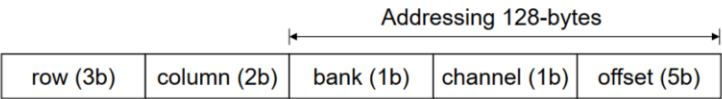
访存密集型应用上加速比明显

TABLE VI: Microbenchmark.

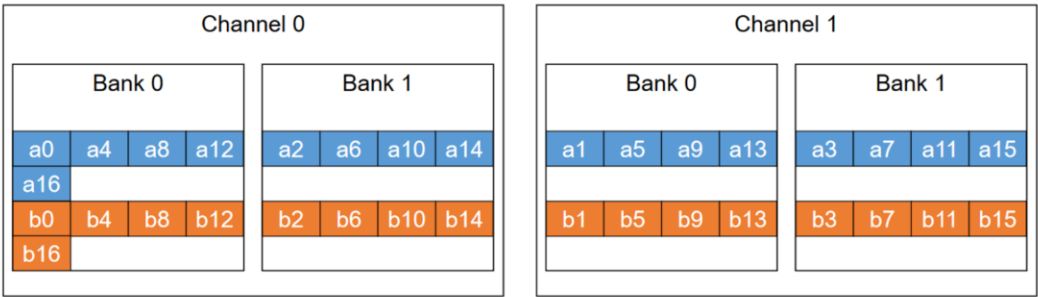
Name	GEMV Dim.	Name	ADD Dim.
GEMV1	1k × 4k	ADD1	2M
GEMV2	2k × 4k	ADD2	4M
GEMV3	4k × 8k	ADD3	8M
GEMV4	8k × 8k	ADD4	16M



- **Data Interleaving 问题:**
 - 对于elementwise 应用, interleaving 不影响计算结果
 - 对于GEMV类应用, 权重矩阵需要软件层进行重排
- 控制粒度问题:
 - PIM计算和Host计算无法细粒度pipeline
 - 多cube后bank数太多, 小规模算子利用率低
- 数据Replica 问题
 - 以GEMV 为例, Host 需要向每个bank 写入输入
- PIM间没有直接的数据通路, 需要Host进行转发

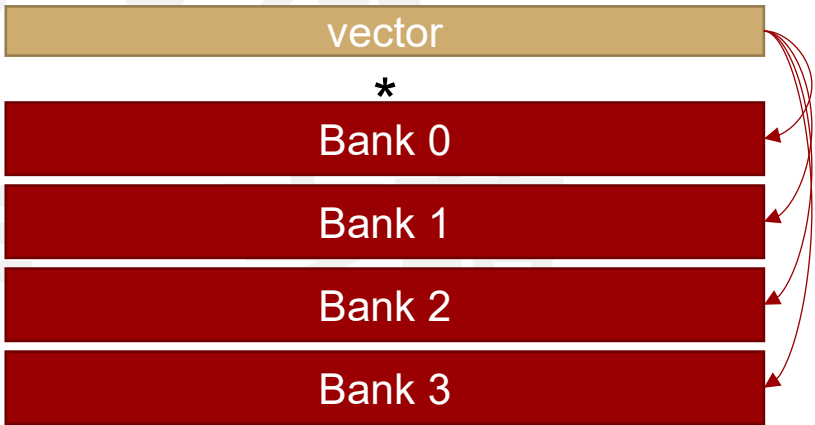


(a) Sample address map



(b) Data placement of vector a and b (allocated to 128-bytes aligned address)

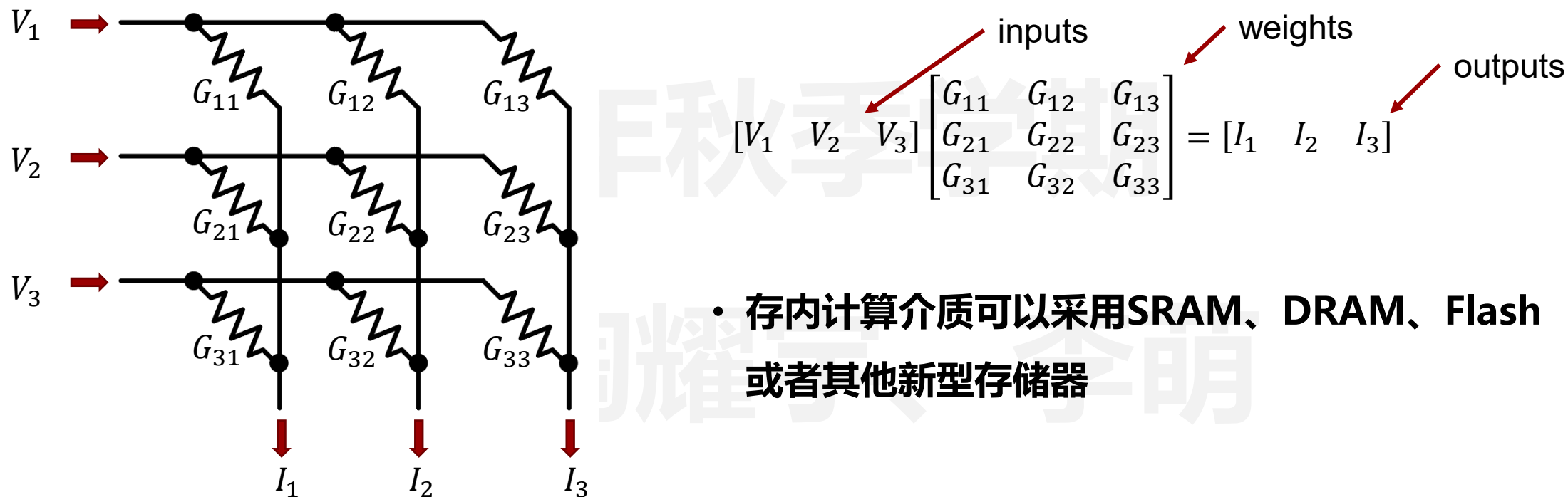
数据interleaving



4x 重复

数据 duplicate

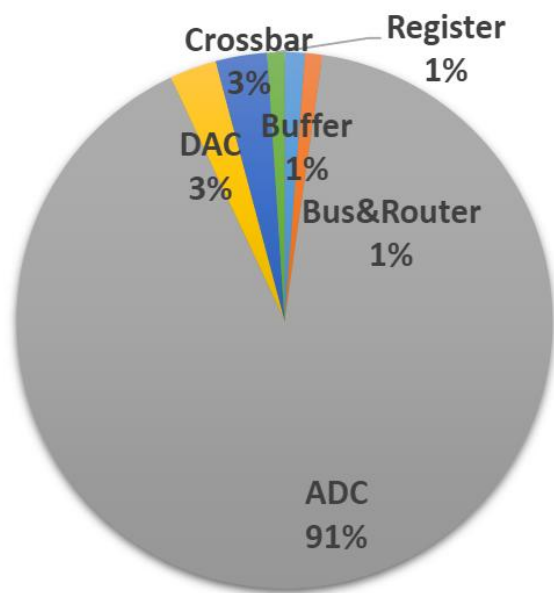
- 存内计算可以分为**模拟**存内计算和**数字**存内计算
- 模拟存内计算利用基尔霍夫电流定律，可以高效实现矩阵向量乘法



- 存内计算介质可以采用SRAM、DRAM、Flash或者其他新型存储器

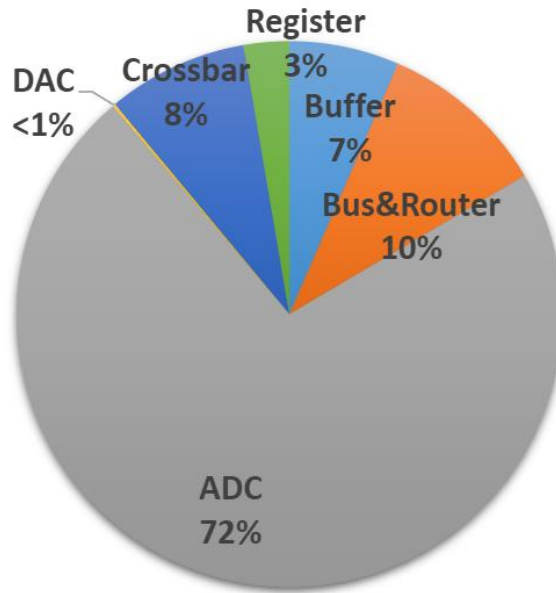
- 模拟存内计算主要面临以下挑战:

- ADC/DAC的开销很大
- 众多非理想效应, 例如IR drop、stuck-at faults、高阻态和低阻态等

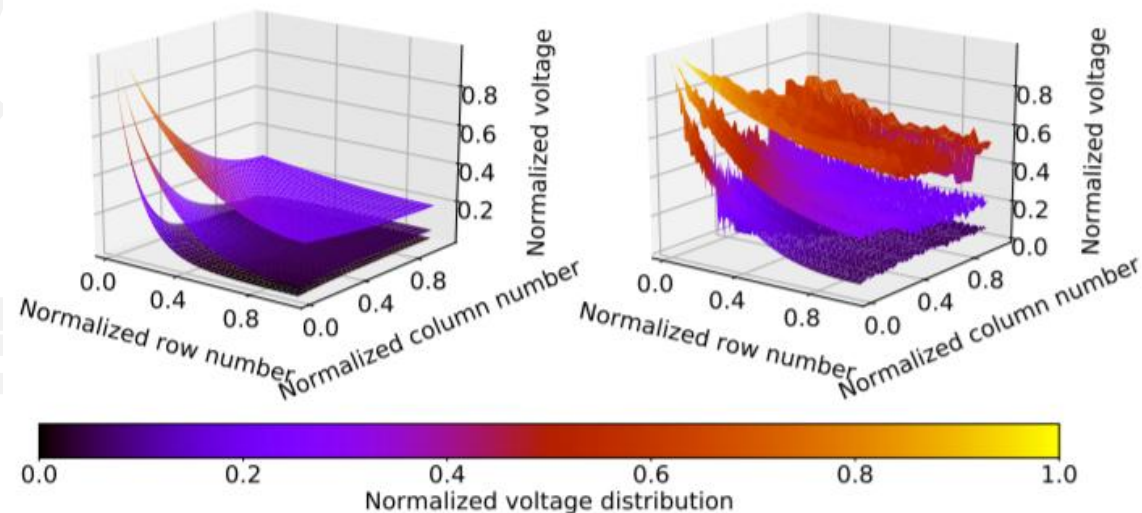


(a) Energy breakdown.

芯芯白田 禾甘力巴



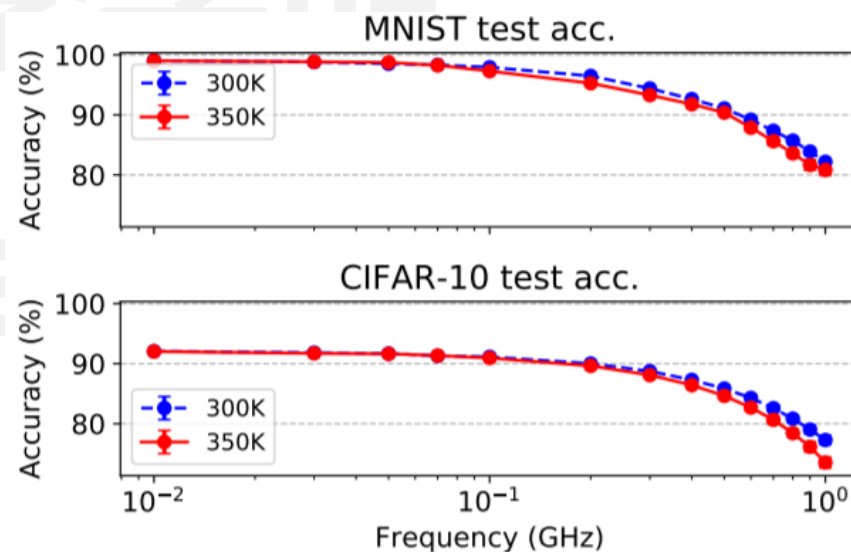
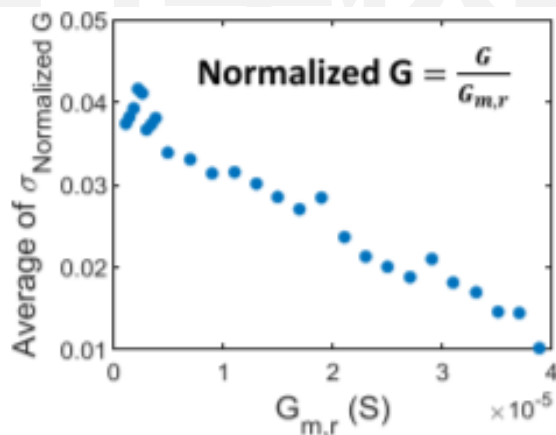
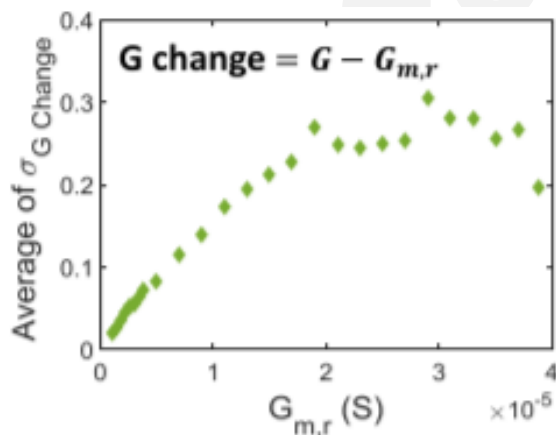
(b) Area breakdown.



IR Drop

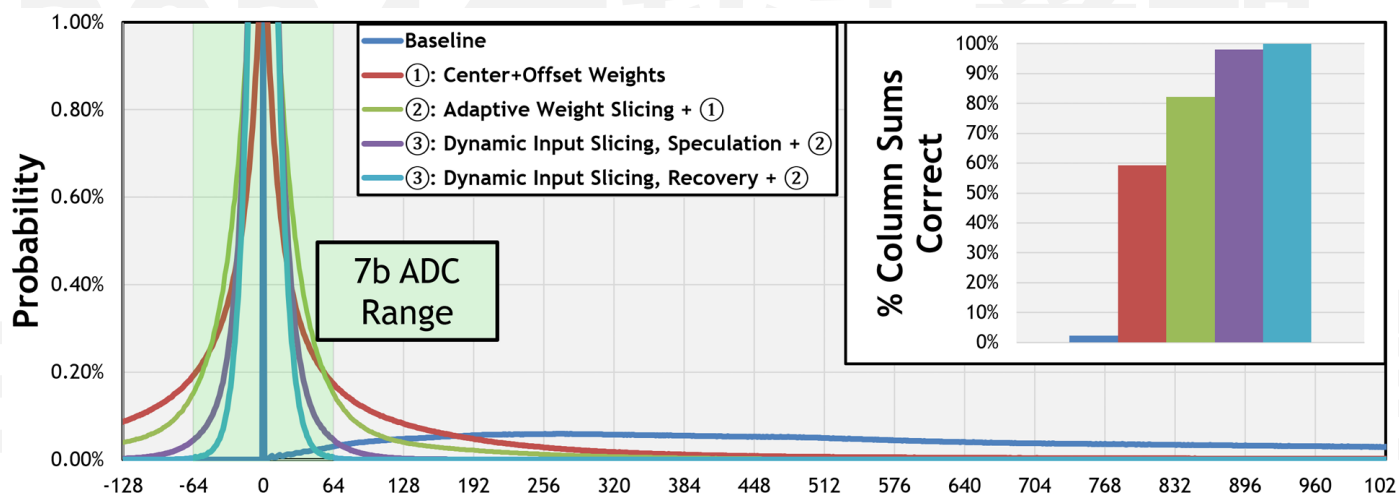
- 模拟存内计算主要面临以下挑战：

- ADC/DAC的开销很大
- 众多非理想效应，例如IR drop、stuck-at faults等
- 数据保持错误、编程错误等
- 本征噪声，例如热噪声、随机电报噪声等 → 如何避免噪声影响成为了重要的研究方向



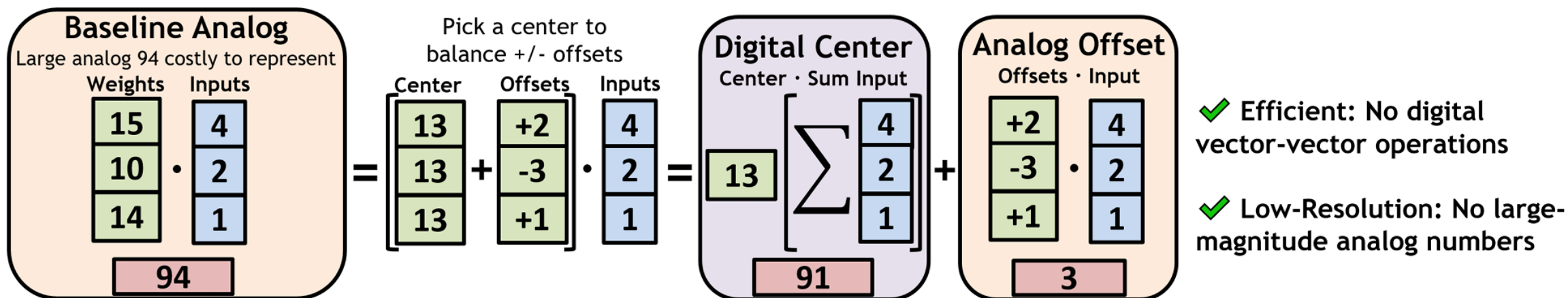
代表性架构RAELLA

- MIT Vivienne Sze组发表于**ISCA 2023**
- 基于RRAM的存内计算架构，文章核心在于**不进行重训练**的前提下，**降低ADC精度**
 - 原有方法需要调整模型参数或者直接使用低精度ADC，往往需要重训练或造成计算误差
- 提出一系列方法，**降低输出结果的范围**，包括 (1)center + offset weights, (2) adaptive weight slicing, (3) dynamic input slicing

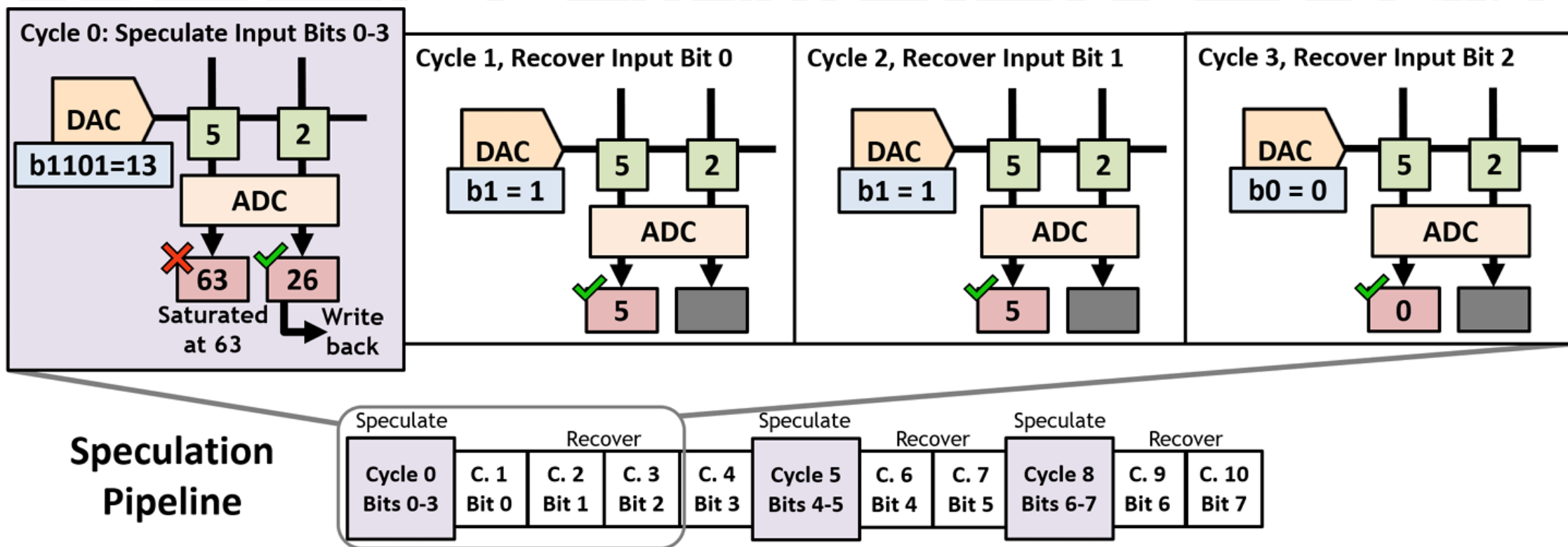


Tanner Andrusis et al., *RAELLA: Reforming the Arithmetic for Efficient, Low-Resolution, and Low-Loss Analog PIM: No Retraining Required!*, ISCA 2023

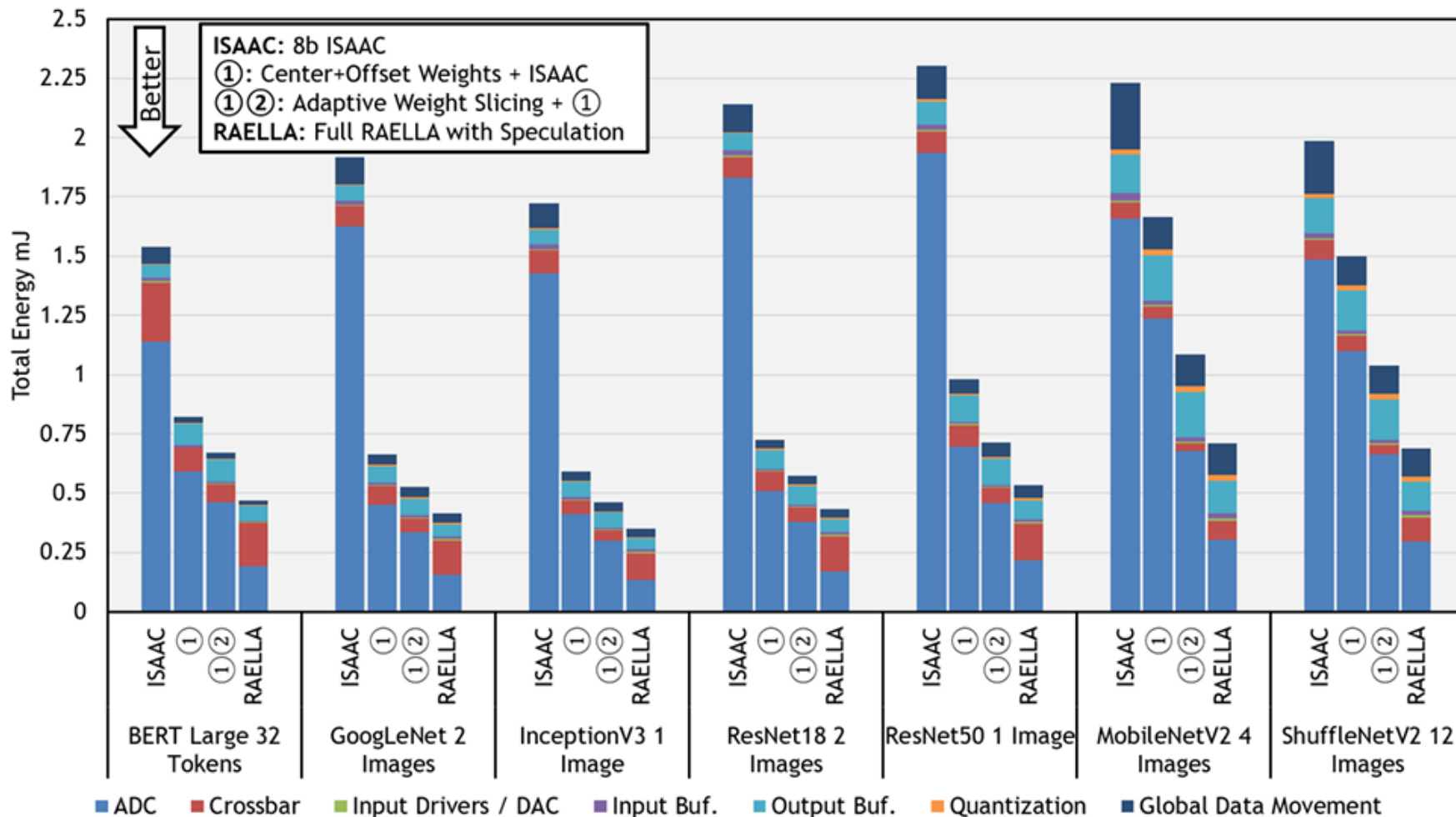
- Center + offset weights
- 通过移动0点，均衡模型权重中的正负值，并通过支持有符号数计算，降低累加结果范围



- Adaptive weight slicing + dynamic input slicing
- 通过控制权重和输入的slicing比特位数，平衡计算次数与单次计算的累加结果范围

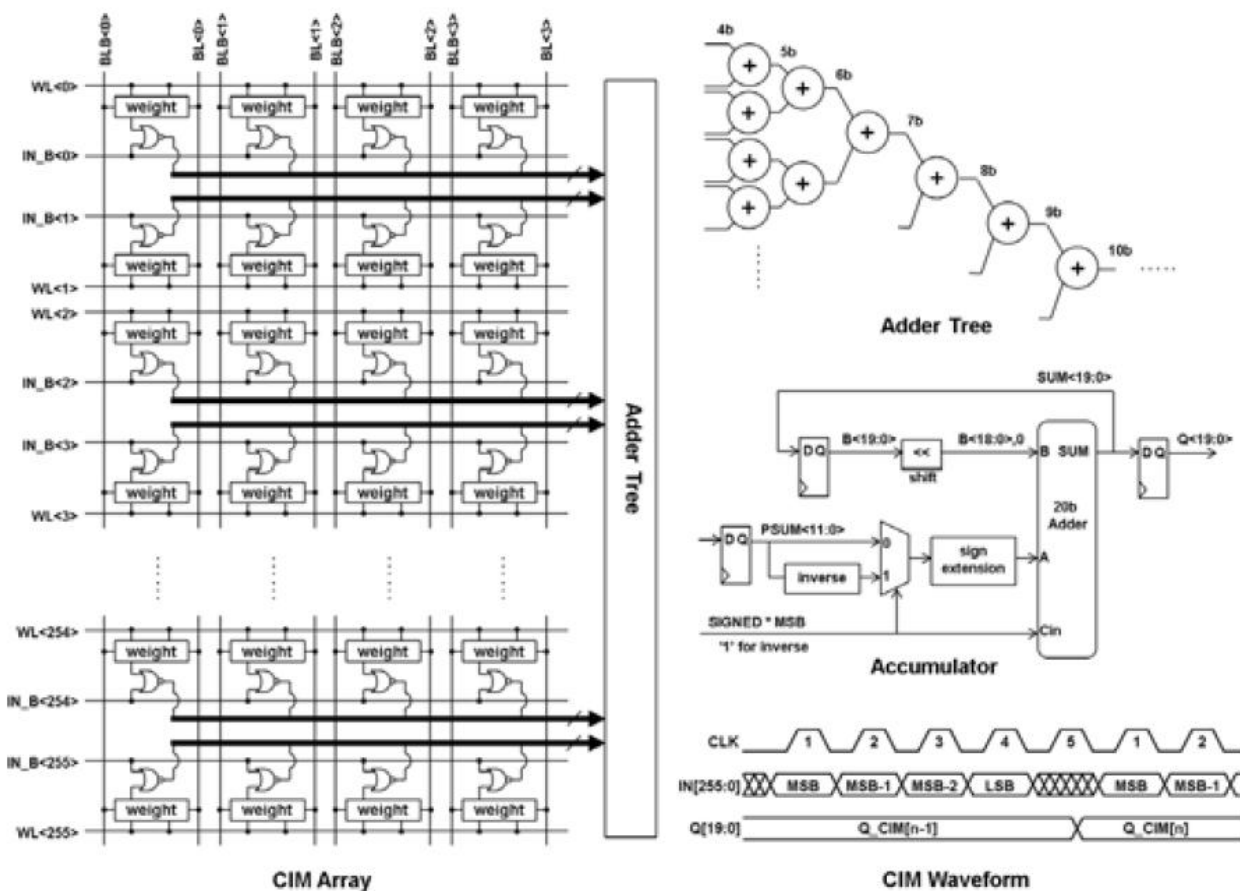


代表性架构RAELLA



Tanner Andrusis et al., *RAELLA: Reforming the Arithmetic for Efficient, Low-Resolution, and Low-Loss Analog PIM: No Retraining Required!*, ISCA 2023

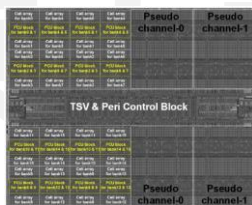
- 为了克服模拟存内计算的精度等挑战，**数字存内计算**被提出，代表性架构包括SRAM数字存算



- 显著降低噪声影响，准确率更高
- 但是计算并行度较低，累加树开销较大

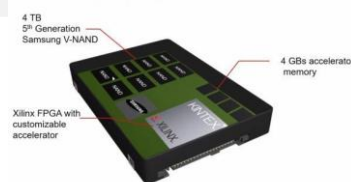
存算一体分类

NDP+DRAM:
数字计算、通用
利用bank级别并行
神经网络、图计算



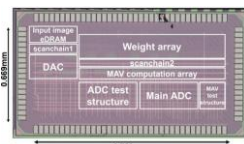
Samsung
ISSCC 2021

NDP+NAND Flash:
数字计算、通用
SSD+计算单元
数据搜索、分析等
有商业化产品



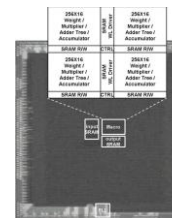
三星 SmartSSD CSD

CIM+DRAM
低比特 (1bit/cell)
模拟计算
面向神经网络



UT Austin, Intel
ISSCC 2021

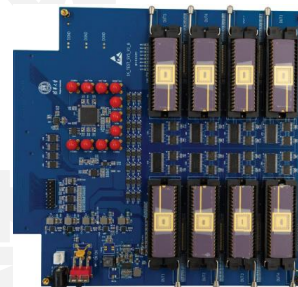
CIM+SRAM
中-高比特
数字、模拟计算
工艺成熟、频率高
各种规模神经网络



TSMC
ISSCC 2021



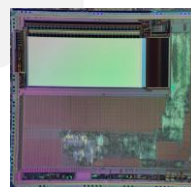
CIM+新型NVM
低、中比特 (1~4bit/cell)
数字、模拟计算
工艺成熟度问题
神经网络、通用逻辑



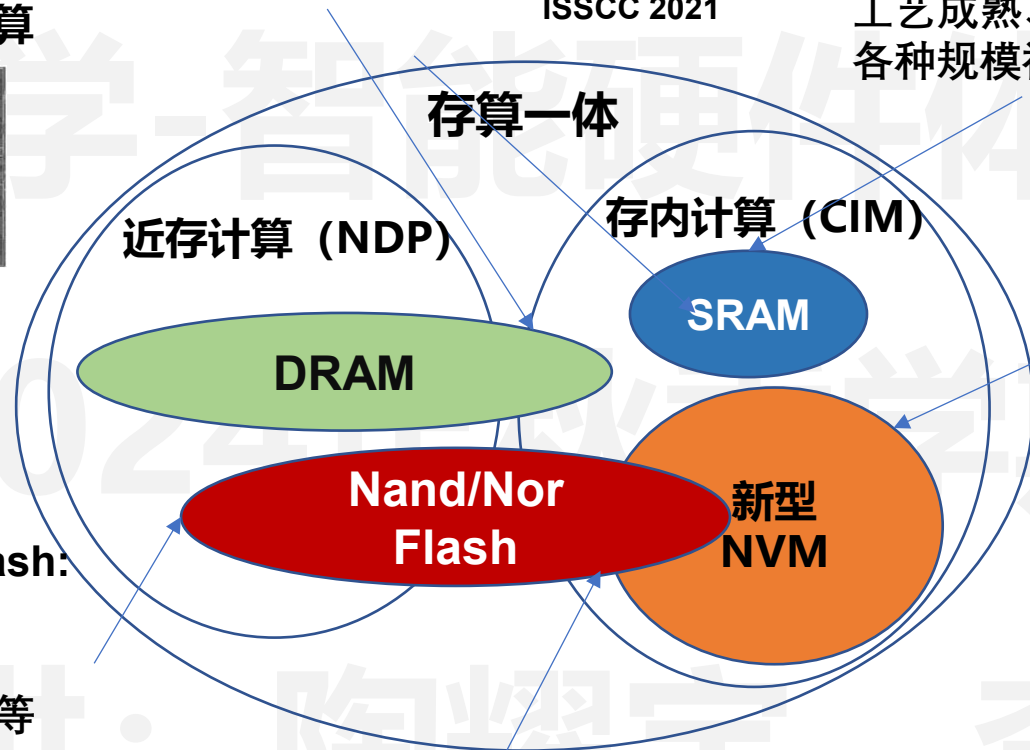
清华大学
Nature 2020

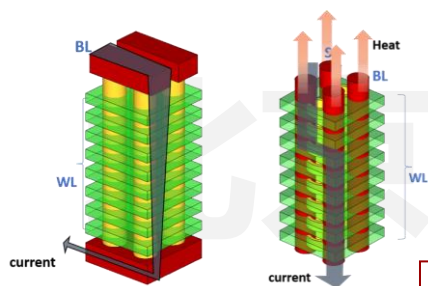
国立清华
TSMC
ISSCC 2021

CIM+Nor Flash
高比特 (>7bit/cell)
模拟计算、功耗低
工艺成熟、节点受限
低功耗AIoT场景



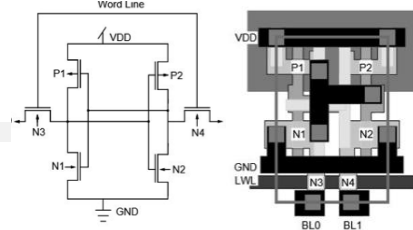
知存科技





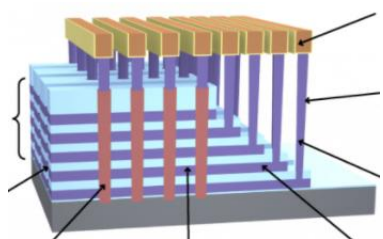
DRAM

优点: 工艺成熟、
密度高
缺点: 速度低、
刷新、只近存
非易失性: 否
适合场景: 冯氏
架构过渡



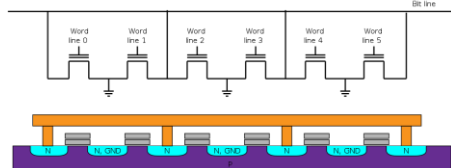
SRAM

优点: 工艺成熟、
IP化应用
缺点: 能效低、
密度低
非易失性: 否
适合场景: 端侧、
边缘中小算力



SSD/Nand
Flash

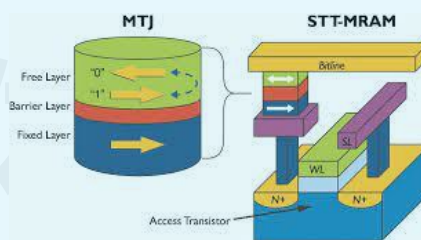
优点: 工艺成熟、
容量大、成本低
缺点: 速度低、
只能近存
非易失性: 是
适合场景: 云端
大容量



Nor Flash

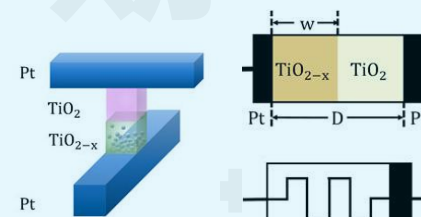
优点: 工艺成熟、
密度高、成本低
缺点: 对PVT变
化敏感、能效低
非易失性: 是
适合场景: 端侧、
边缘低成本

新兴器件



磁器件
(MRAM)

优点: 能效、速
度、密度高
缺点: 与CMOS
大规模集成难
非易失性: 是
适合场景: 端侧、
边缘中小算力



忆阻器
(RRAM/PCM)

优点: 算力、能
效、密度高
缺点: 工艺爬坡
成熟中
非易失性: 是
适合场景: 云边
端大算力

实现存算一体的多种技术路径

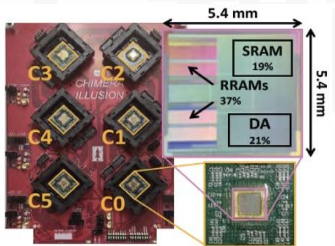
- 存算一体技术受到国内外产业界和学术界的高度重视，已有诸多产业化布局

器件

基于嵌入式存储的近存计算

存内没有计算

多种嵌入式存储器
(DRAM、SRAM
等)

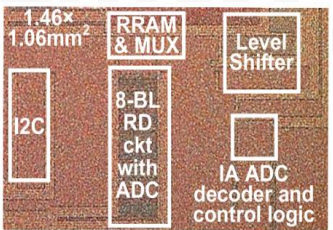


M. Giordano et al., VLSI 2021

基于新兴非易失存储器的存内计算

存内含计算

需新兴器件与电路
架构的协同创新

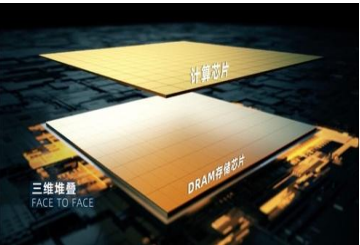


J.H. Yoon et al., JSSC 2022

基于先进封装的近存计算

存内没有计算

多种先进封装
(Chiplet、三维
封装等)

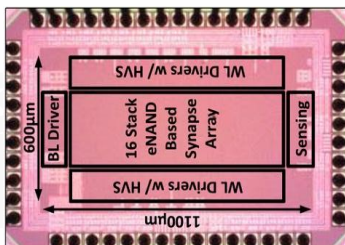


D. Niu et al., ISSCC 2022

基于存储级存储器的存内计算

存内含计算

将存内计算应用于
成熟的存储级存储
器 (如Flash)



M. Kim et al., JSSCC 2022

计算架构

路径一：先进封装实现近存计算

- 将存储器芯片与逻辑处理芯片通过多种先进封装方式拉近距离

存储器芯片

↑↑↑↑

处理芯片

先进封装

优势

与现有系统兼容性好
设计难度低

典型案例

AMD Zen CNM, SK-Hynix
HBM-PIM, Alibaba CNM

性能	先进封装 近存计算
算力	高
能效	低
设计难度	低
可重构性	高
成本	低

12Hi Section View

Core die

SK-海力士
HBM-PIM

DRAM Die Photo (8G8)

Neural Engine Region

Match Engine Region

Alibaba 达摩院3D
封装近存计算芯片

Core 0

Core 1

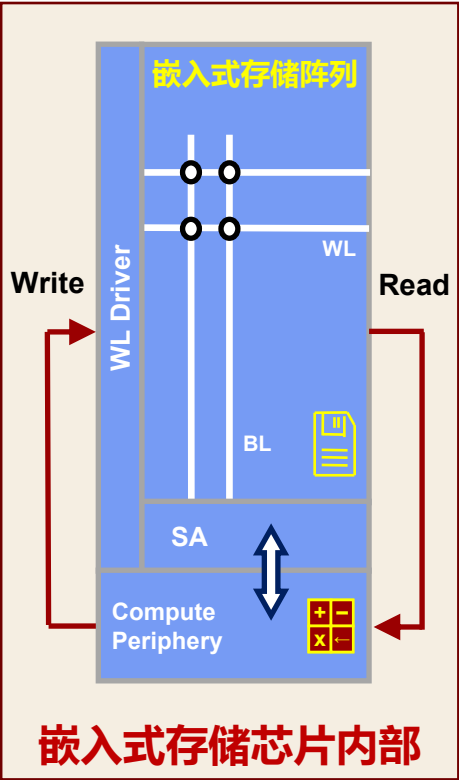
Core 2

Core 3

AMD Zen近存
计算样片

路径二：嵌入式存储实现近存计算

- 在嵌入式存储器芯片（DRAM/SRAM等）内部集成近存逻辑电路



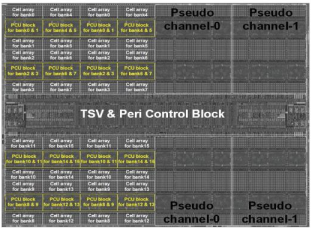
优势

数据搬运距离较先进封装
更加缩短

典型案例

三星 HBM CNM,
GraphCore SRAM CNM,
Stanford RRAM CNM

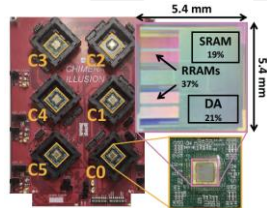
性能	嵌入式存储近存
算力	中等
能效	中等
设计难度	中等
可重构性	中等
成本	中等



三星HBM
CNM



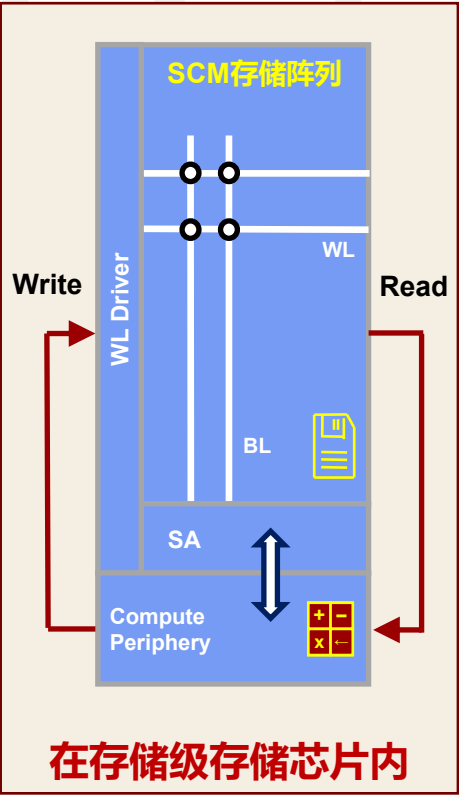
GraphCore SRAM
CNM



Stanford RRAM
CNM

路径二：存储级存储实现存内计算

- 利用成熟的存储级存储器（Flash等）进行存内计算



优势

工艺成熟、成本低

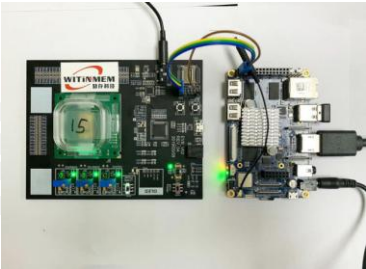
典型案例

Mythic Flash CNM,
WitinMem Nor Flash CIM

性能	存储级存储器 存内计算
算力	中等
能效	低
设计难度	中等
可重构性	中等
成本	低



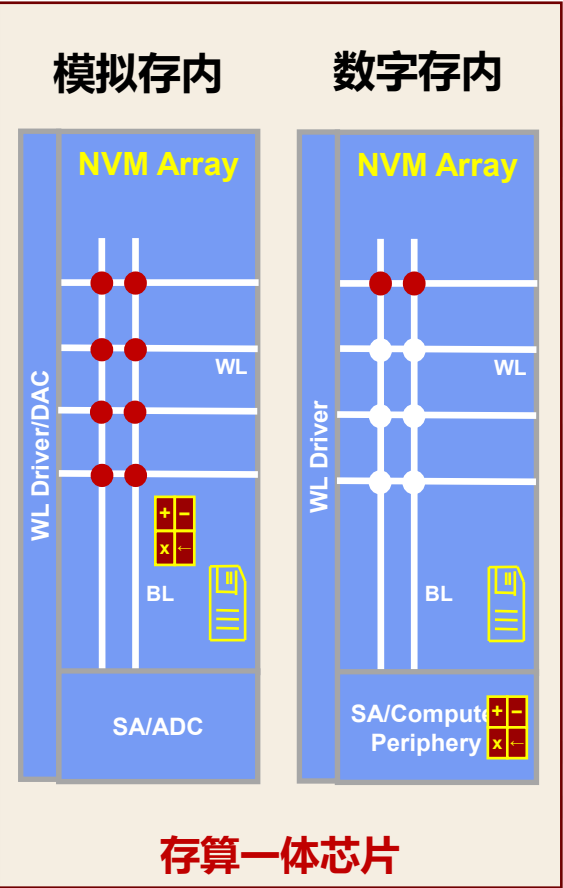
Mythic Flash CIM Chip



知存科技 Flash CIM

路径二：新兴非易失性器件实现存内计算

- 使用新兴的非易失性存储器（ReRAM等）进行存内计算



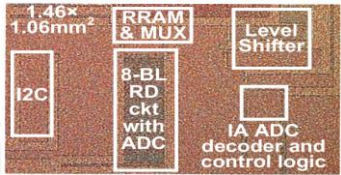
优势

大算力、高能效、高密度

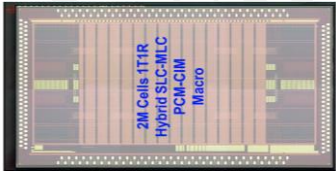
典型案例

台积电ReRAM芯片、台积电PCM芯片、三星MRAM芯片

性能	新兴非易失性存储器存内计算
算力	高
能效	高
设计难度	高
可重构性	中等
成本	高



TSMC RRAM CIM



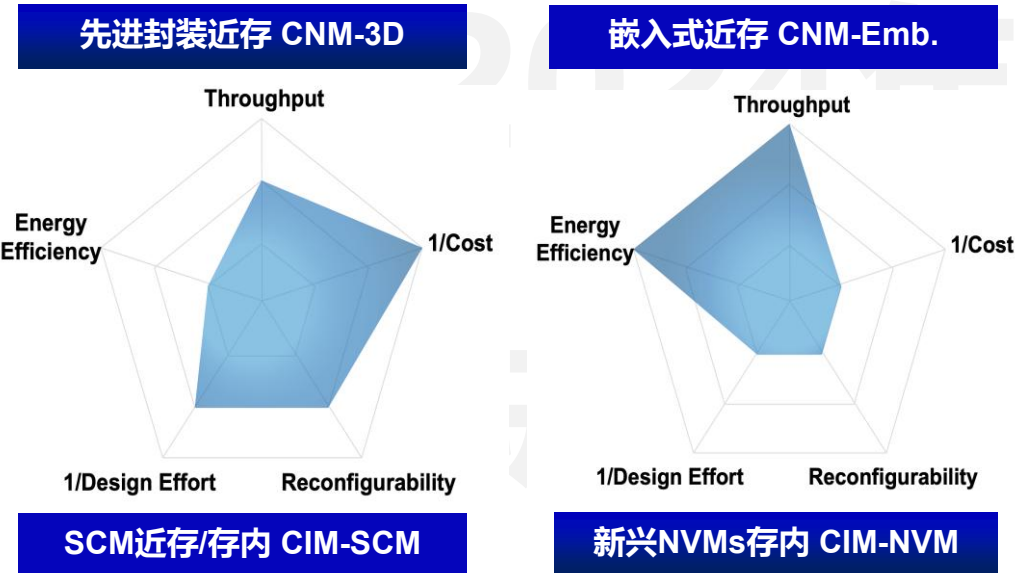
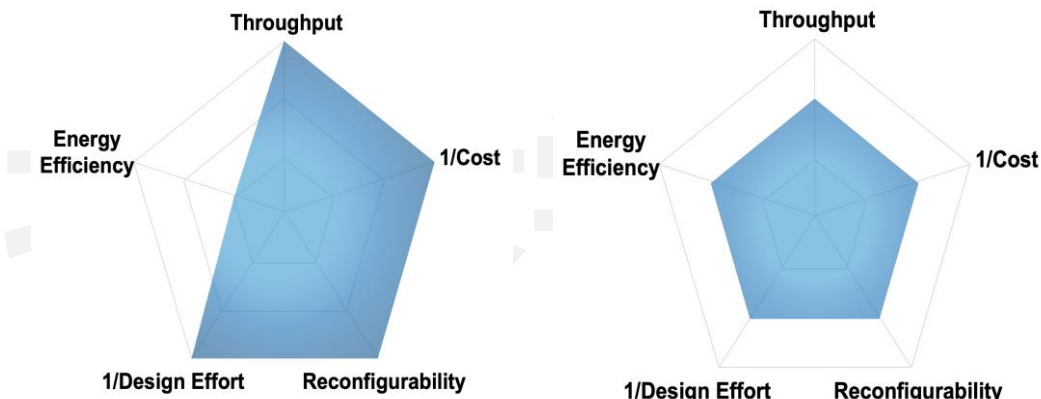
TSMC PCM CIM



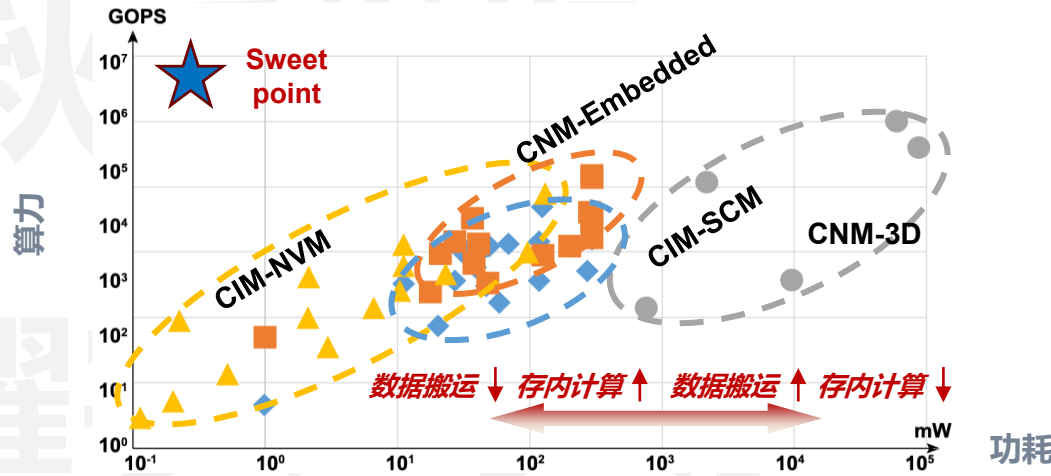
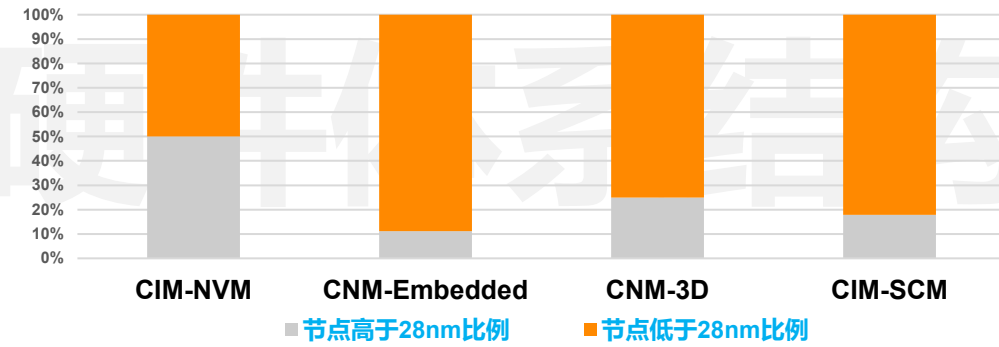
三星 MRAM CIM

存算一体多种技术路径的比较

- 存算一体技术针对不同应用场景可以采用不同的技术路径



新兴NVMs存内计算 (CIM-NVM) 向先进节点演进中

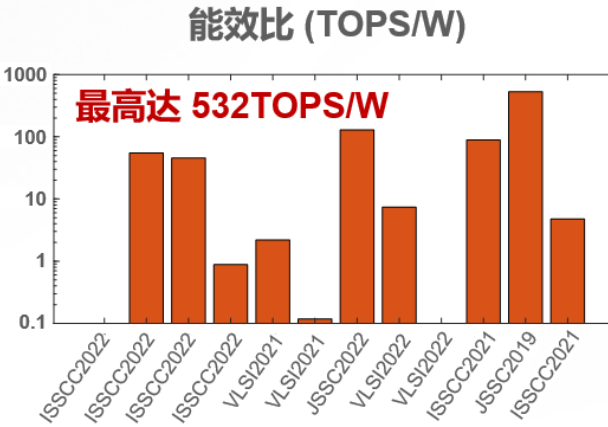
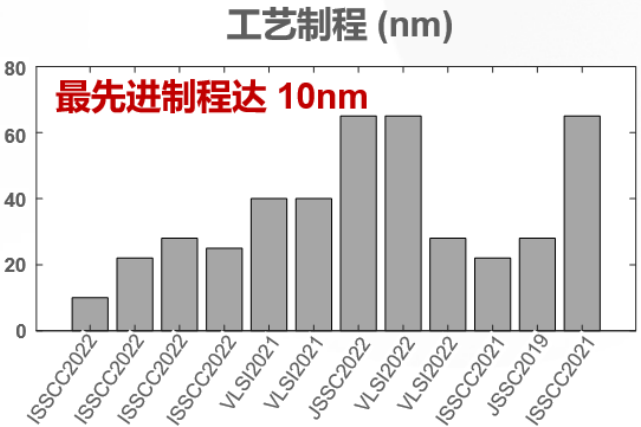
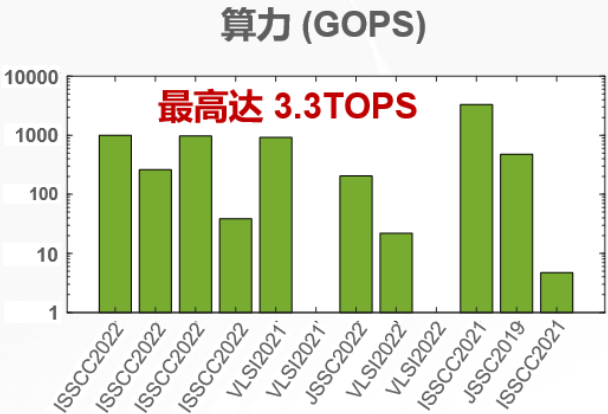
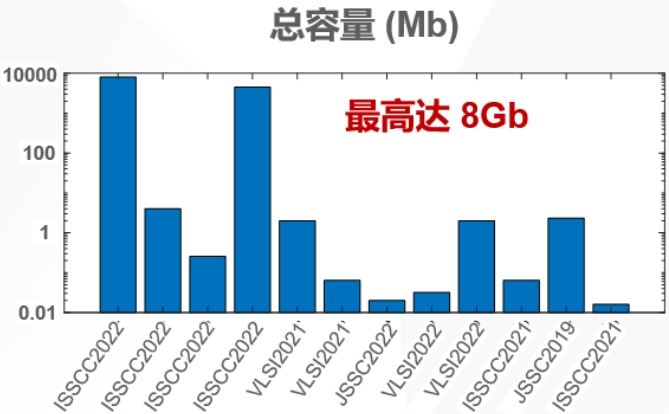


当前存算一体芯片算力、功耗图谱

存算一体芯片发展现状

- 近存计算芯片

Org	Pub	Device
海力士	ISSCC2022	DRAM
台积电	ISSCC2022	MRAM
阿里巴巴	ISSCC2022	DRAM
斯坦福	VLSI2021	SRAM
斯坦福	VLSI2021	RRAM
海力士	JSSC2022	NAND FLASH
UT Austin	VLSI2022	DRAM
Micro Chip	VLSI2022	FLASH
台积电	ISSCC2021	SRAM
斯坦福	JSSC2019	SRAM
UT Austin	ISSCC2021	DRAM

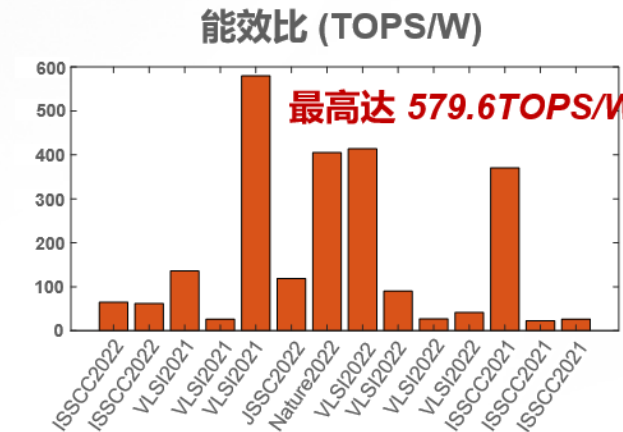
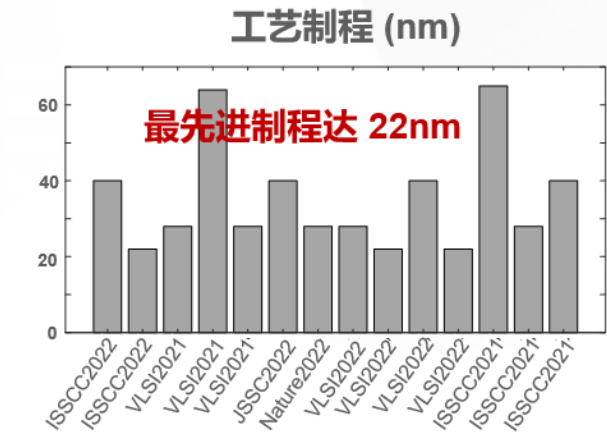
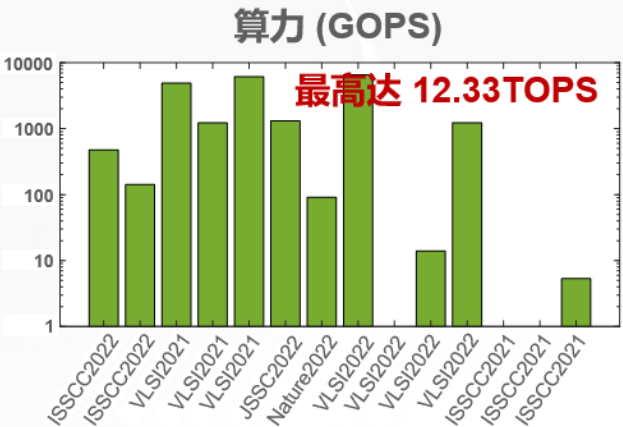
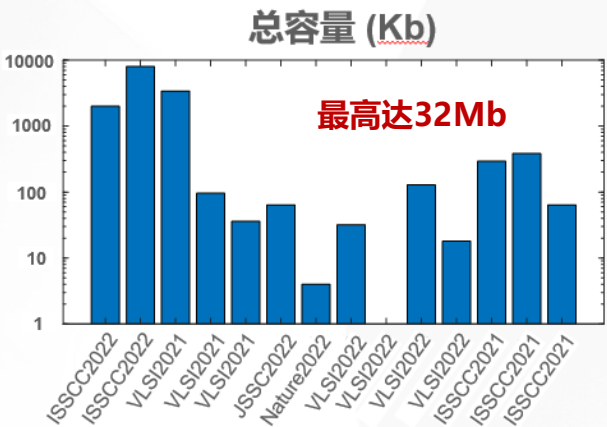


适合针对现有冯氏架构的存算一体改造

存算一体芯片发展现状

- 模拟存内计算芯片

Org	Pub	Device
台积电	ISSCC2022	PCM
台积电	ISSCC2022	RRAM
三星	VLSI2021	SRAM
北大	ISSCC2021	SRAM
普林斯顿	VLSI2021	SRAM
佐治亚理工	JSSC2022 /ISSCC2021	RRAM
三星	Nature2022	MRAM
KAIST	VLSI2022	SRAM
圣母大学	VLSI2022	FeFET
佐治亚理工	VLSI2022	RRAM
普林斯顿	VLSI2022	MRAM
清华	VLSI2021	SRAM
中国台湾大学	ISSCC2021	SRAM
佐治亚理工	ISSCC2022	RRAM

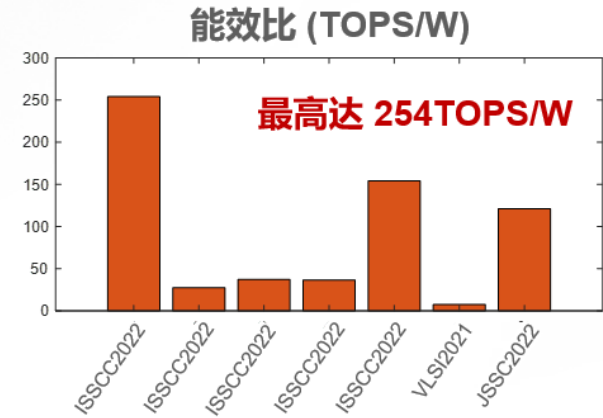
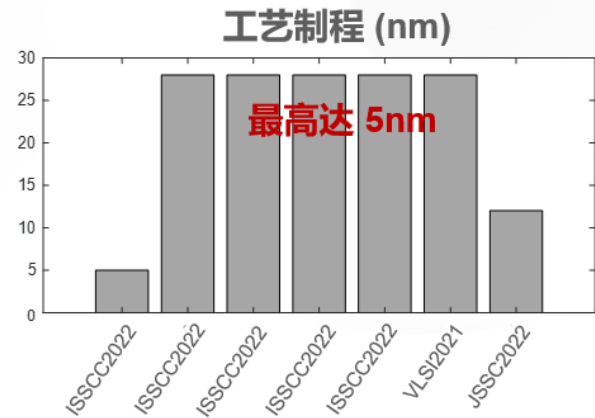
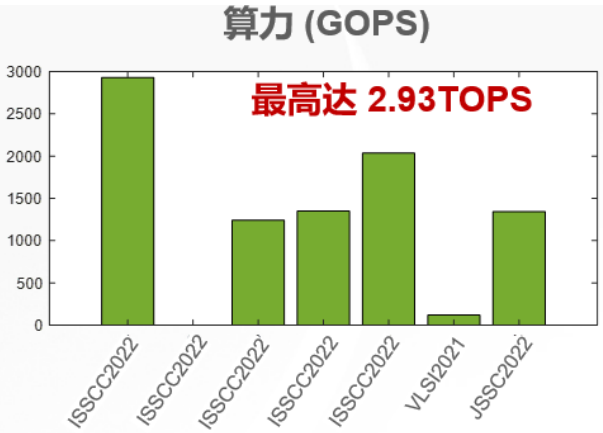
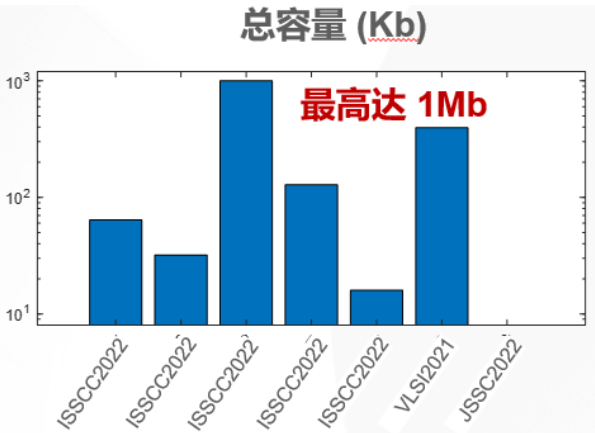


适合对计算精度不敏感的场景

存算一体芯片发展现状

- 数字存内计算芯片

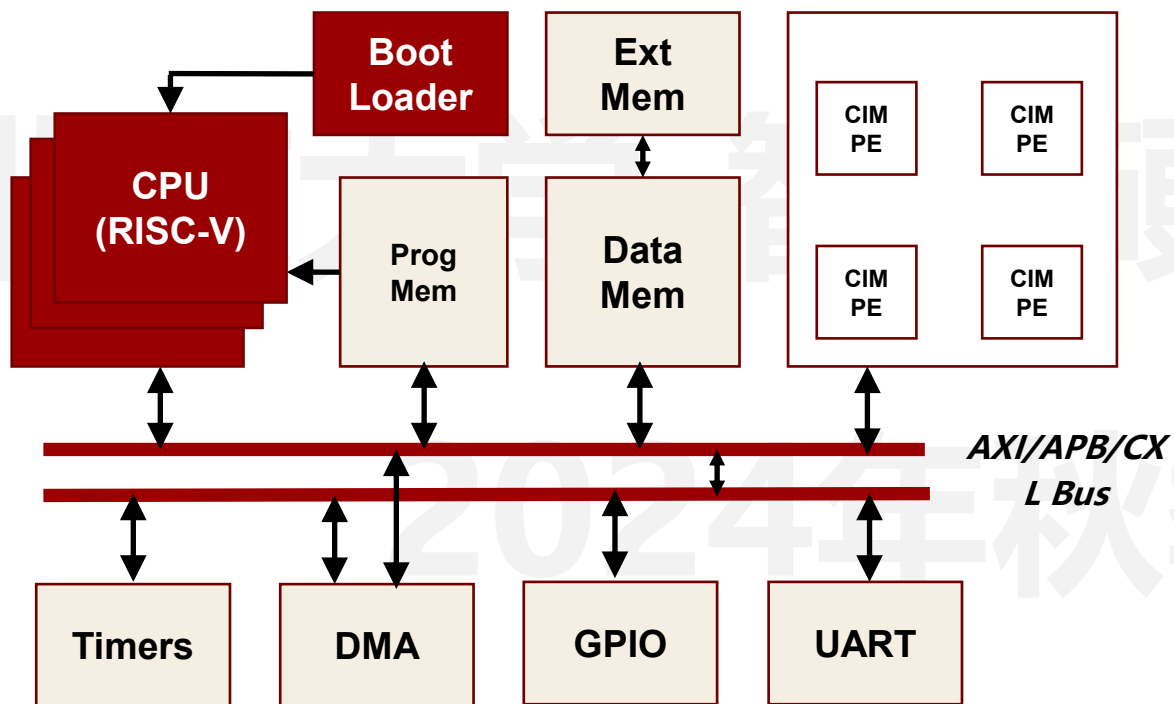
Org	Pub	Device
台积电	ISSCC2022	SRAM
北大	ISSCC2022	SRAM
中国台湾大学	ISSCC2022	SRAM
清华	ISSCC2022	SRAM
哥大	ISSCC2022	SRAM
KAIST	VLSI2021	SRAM
台积电	VLSI2022	SRAM



适合需要高计算精度的场景

存算一体的异构集成

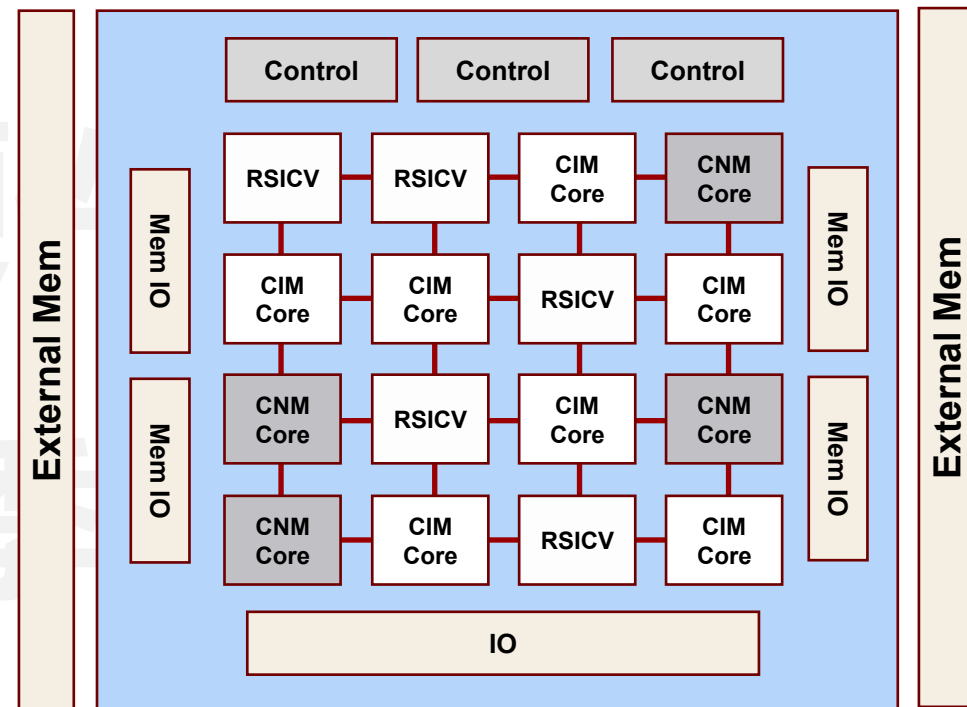
- 融合存算一体技术的多阵列异构集成芯片



集中式异构集成架构

设计难度相对较低

但基于集中式总线的架构难以最大限度发挥存算一体的优势



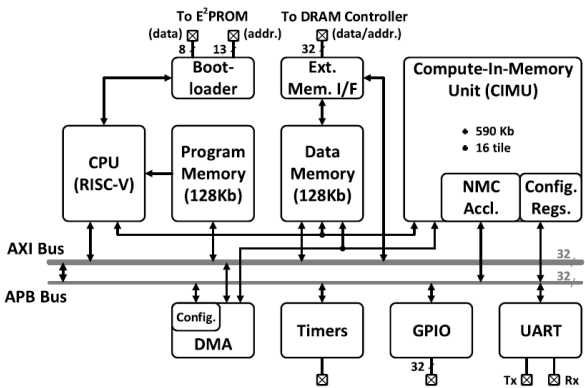
分布式异构集成架构

可扩展性、可重构性好

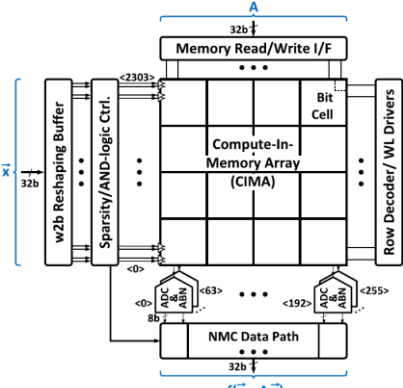
但片上网络设计及通信难度高

典型存算一体异构集成芯片 – 总线式集成芯片

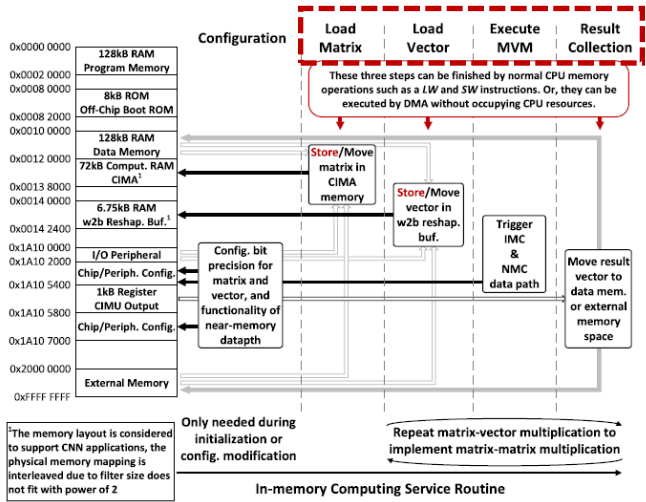
融合存算一体技术的多阵列异构集成芯片（普林斯顿大学）



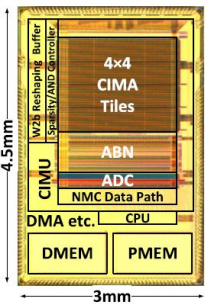
顶层架构



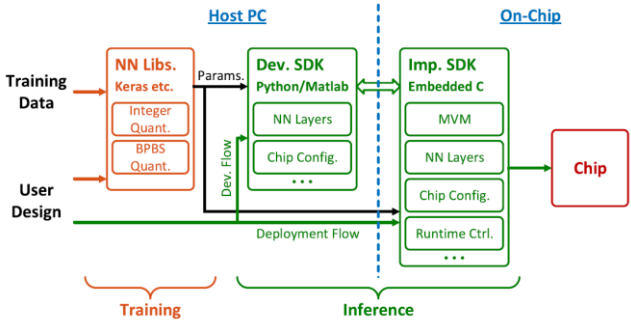
存内计算阵列架构



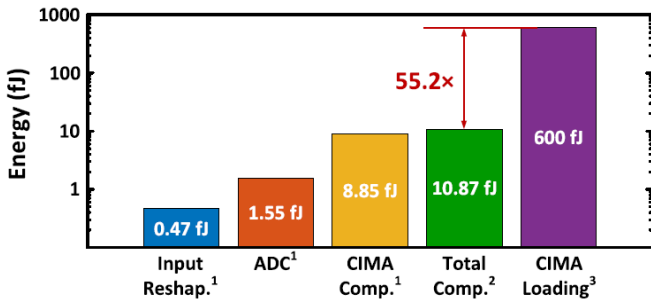
存内计算实现矩阵乘法时序图



芯片版图



存内和非存内计算的映射方式

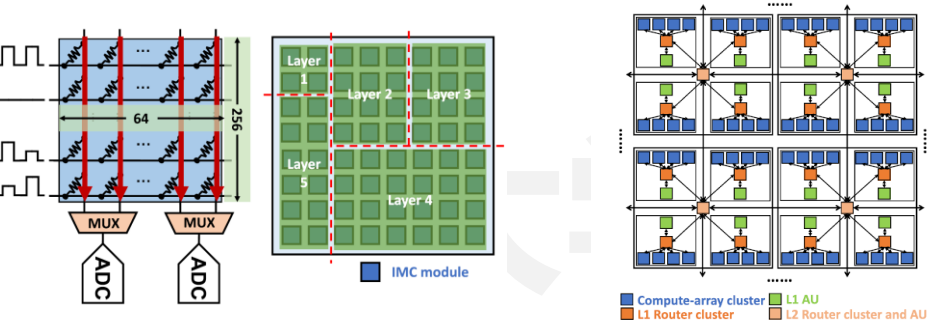


能耗分布分析

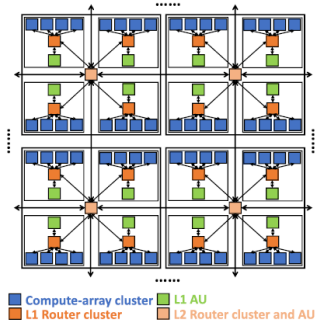
- 4x4 8T SRAM 存内计
- 阵列单阵列576x64，共 590Kb
- 集成RISCV CPU
- 128Kb ProgMem
- 128Kb DataMem
- 模拟存内 8b ADC 精度

典型存算一体异构集成芯片 – 分布式集成芯片

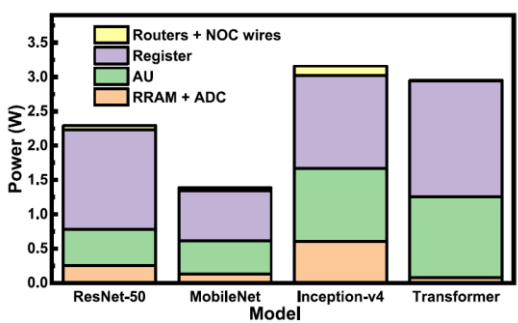
融合存算一体技术的多阵列异构集成芯片（密歇根大学）



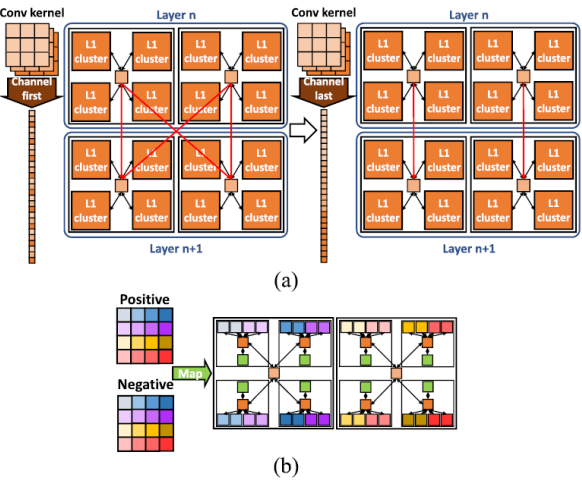
ReRAM模拟存内计算阵列



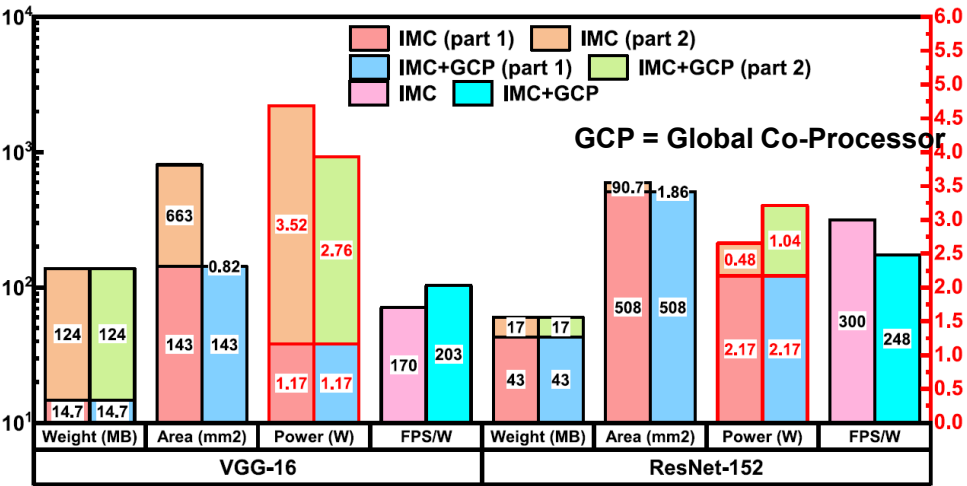
层次式片上网络



能耗分布分析



4x4个cluster、每个cluster
包含4x4个存内计算阵列

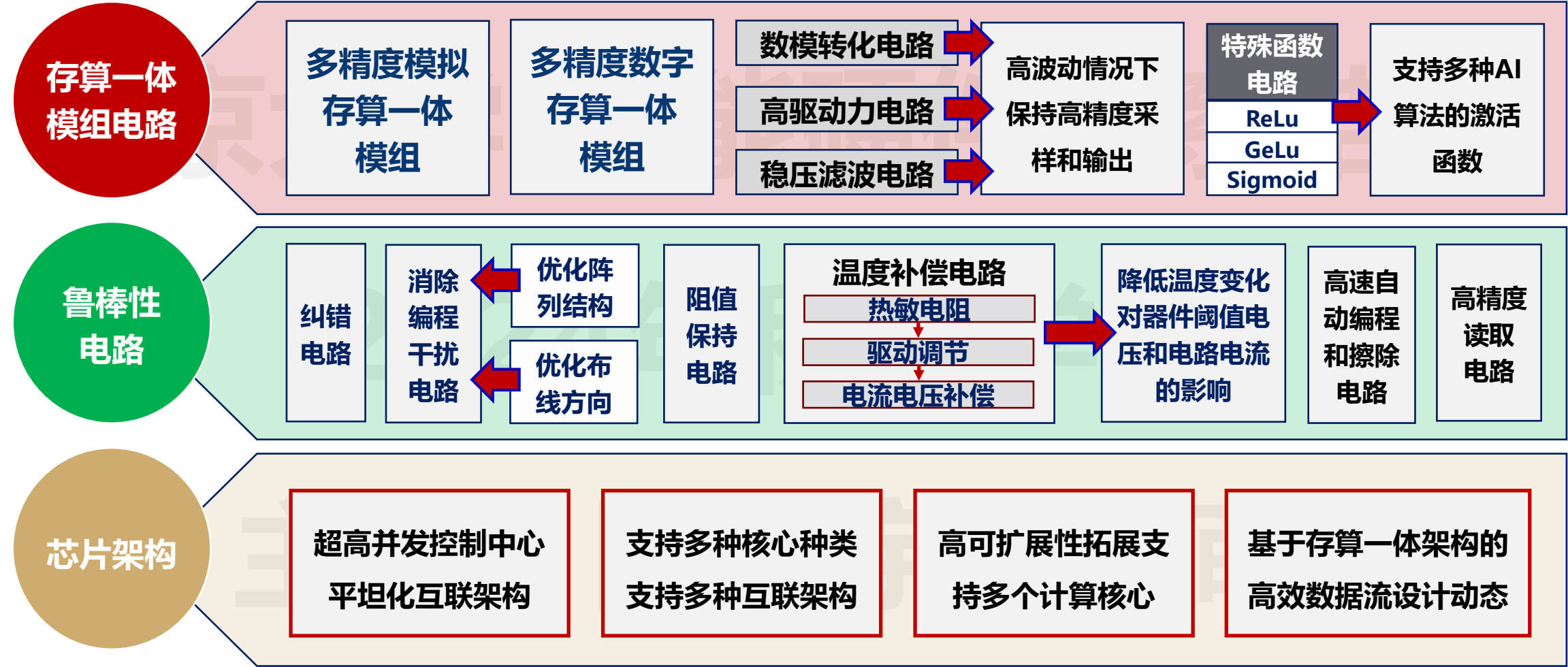


分布式异构架构运行VGG-16和ResNet-152的性能分析

- 16x16层级式网格片上网络
- 集成4种不同尺寸的ReRAM
- 模拟存内计算阵列
- 模拟存内 8b ADC 精度

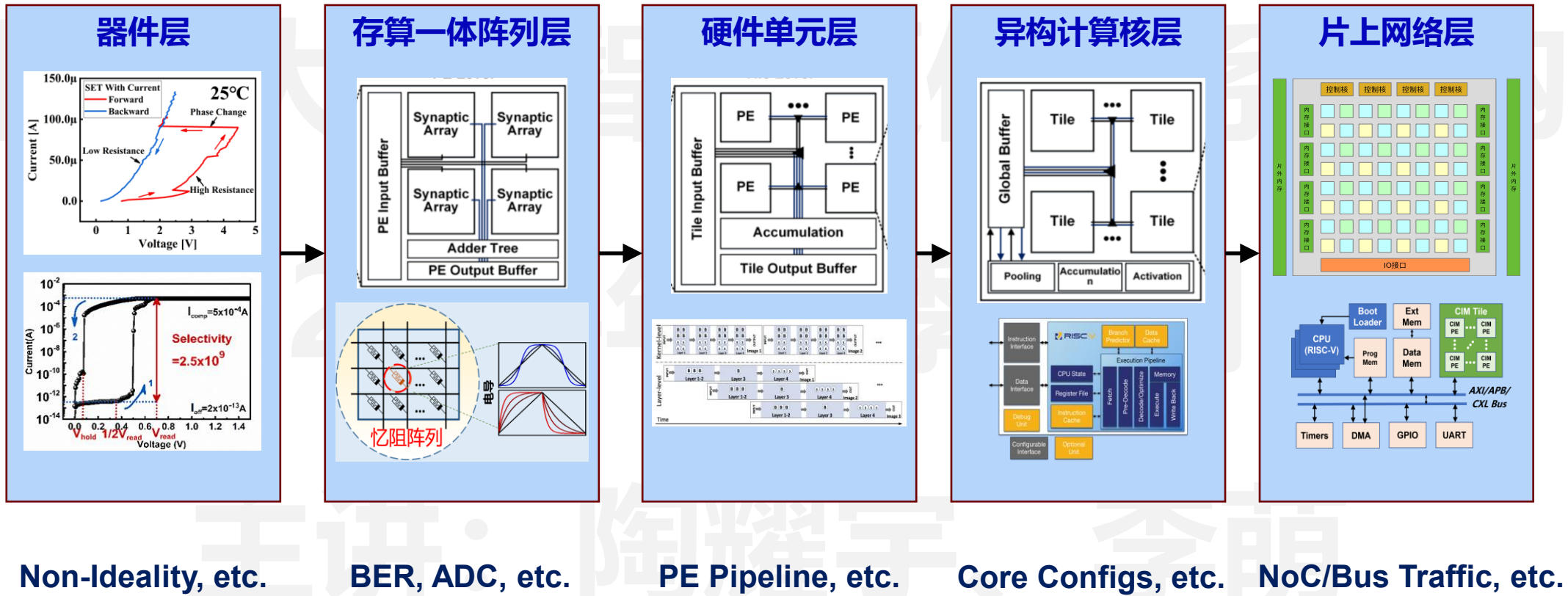
存算一体异构集成的电路与架构挑战

- 优化电路与芯片架构，保障能效优势和迭代演进能力



存算一体异构集成的系统级多层次挑战

- 需要考虑多层次的协同设计，设计空间非常大

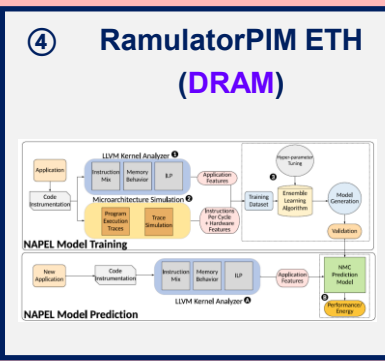
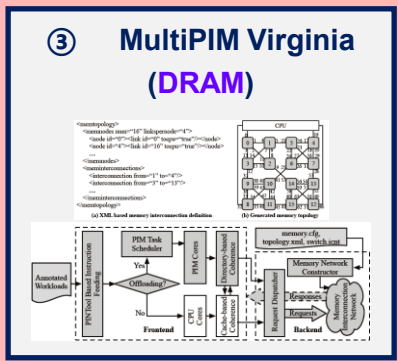
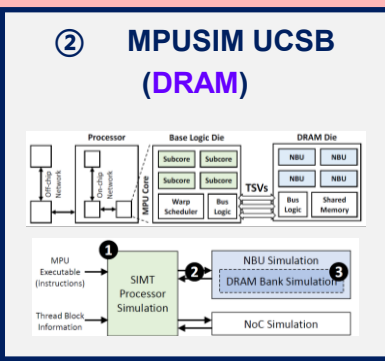
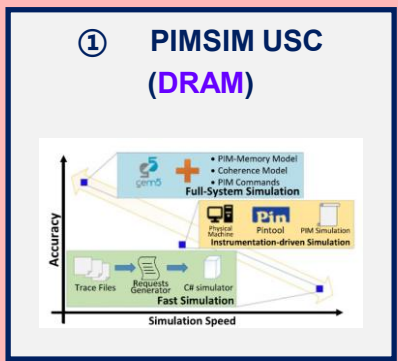


存算一体异构集成的架构仿真

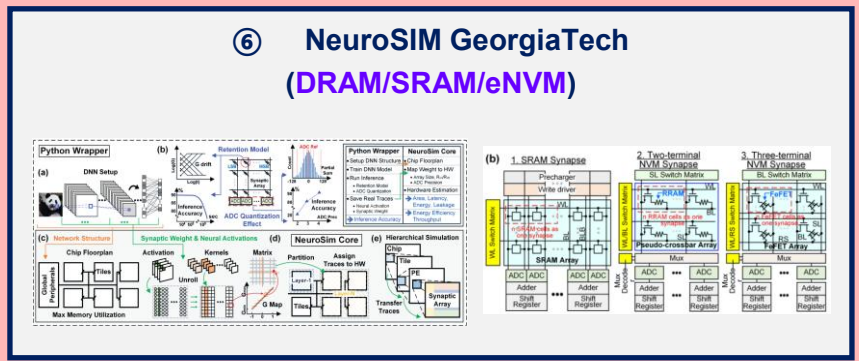
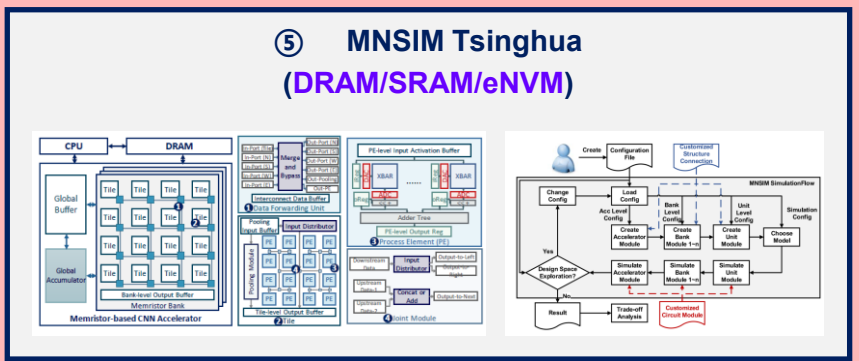
- 架构仿真为设计空间非常大的存算一体异构集成提供支撑

目标应用

通用计算



AI计算 (MAC/Conv)



目录

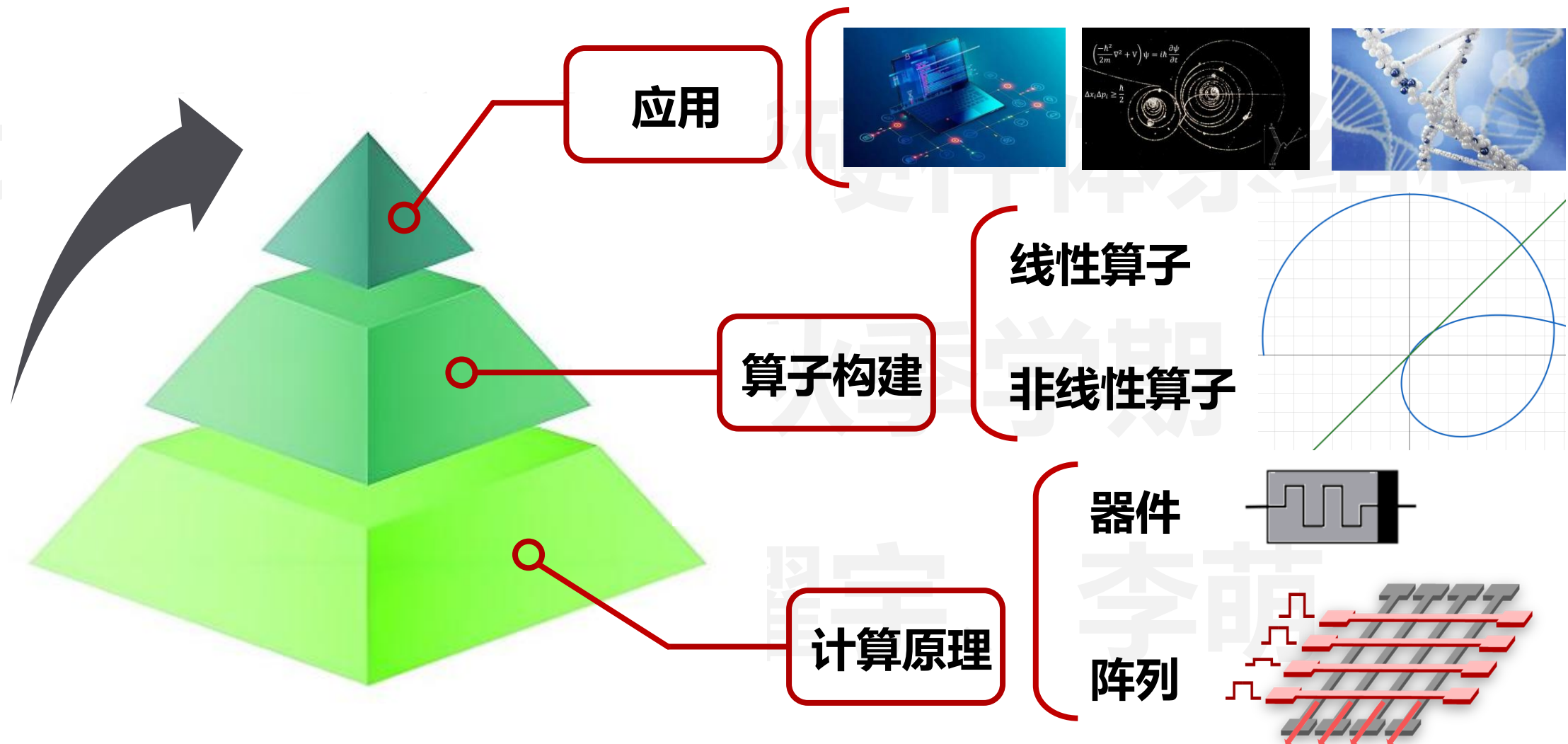
CONTENTS



- 01. 存算一体芯片发展概述
- 02. 存算一体芯片算子拓展
- 03. AI大模型存算一体加速
- 04. 存算一体任务编译部署

存算一体化算子拓展

- 从当前矩阵乘法、卷积等少量线性计算迈向多种非线性算子



- 基于阵列的线性算子实现方法

□ 线性算子

1. 矩阵向量乘法
2. 矩阵矩阵乘法
3. 矩阵矩阵向量乘法
4. 傅里叶变换

.....

□ 非线性算子

1. 汉明距离 (查找比对)
2. 欧氏距离
3. 排序
4. 正态分布生成
5. 非线性初等函数

.....

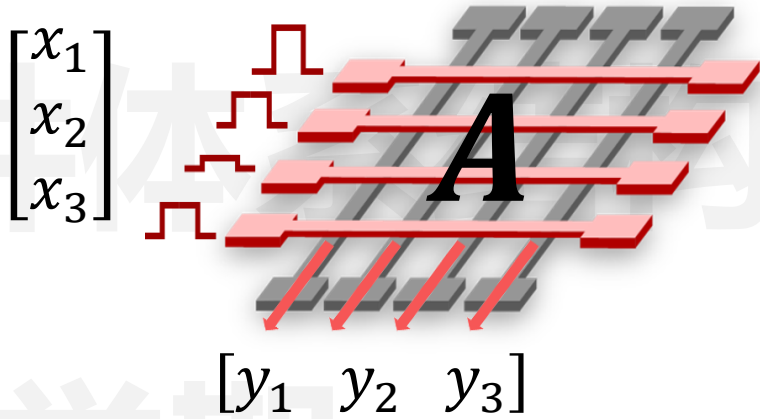
叠加原理: $A(x + y) = A(x) + A(y)$

齐次原理: $A(\alpha x) = \alpha A(x)$

- 矩阵向量乘法与卷积

□ 矩阵向量乘法

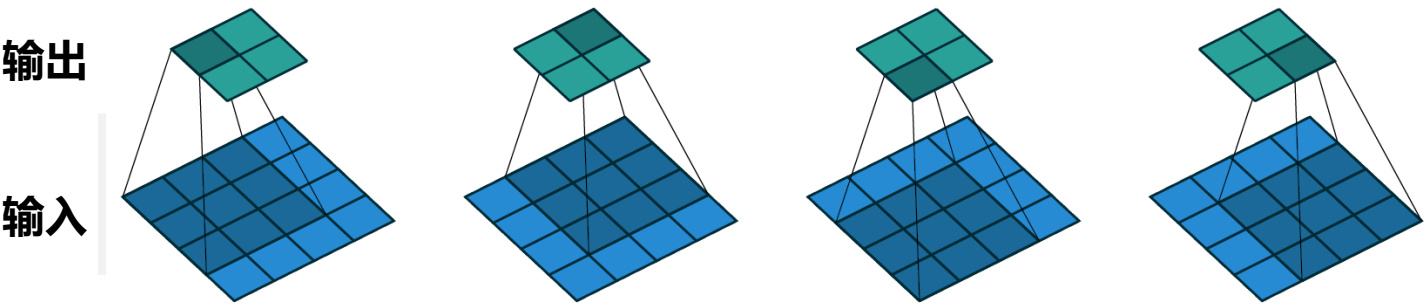
$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}^T \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} A_{11}x_1 + A_{21}x_2 + A_{31}x_3 \\ A_{12}x_1 + A_{22}x_2 + A_{32}x_3 \\ A_{13}x_1 + A_{23}x_2 + A_{33}x_3 \end{bmatrix}^T$$



$$y_j = \sum_i A_{i,j} * x_i$$

□ 卷积：神经网络基本算子

$$y_{i,j} = \sum_{u,v} w_{u,v} * x_{i+u,j+v}$$



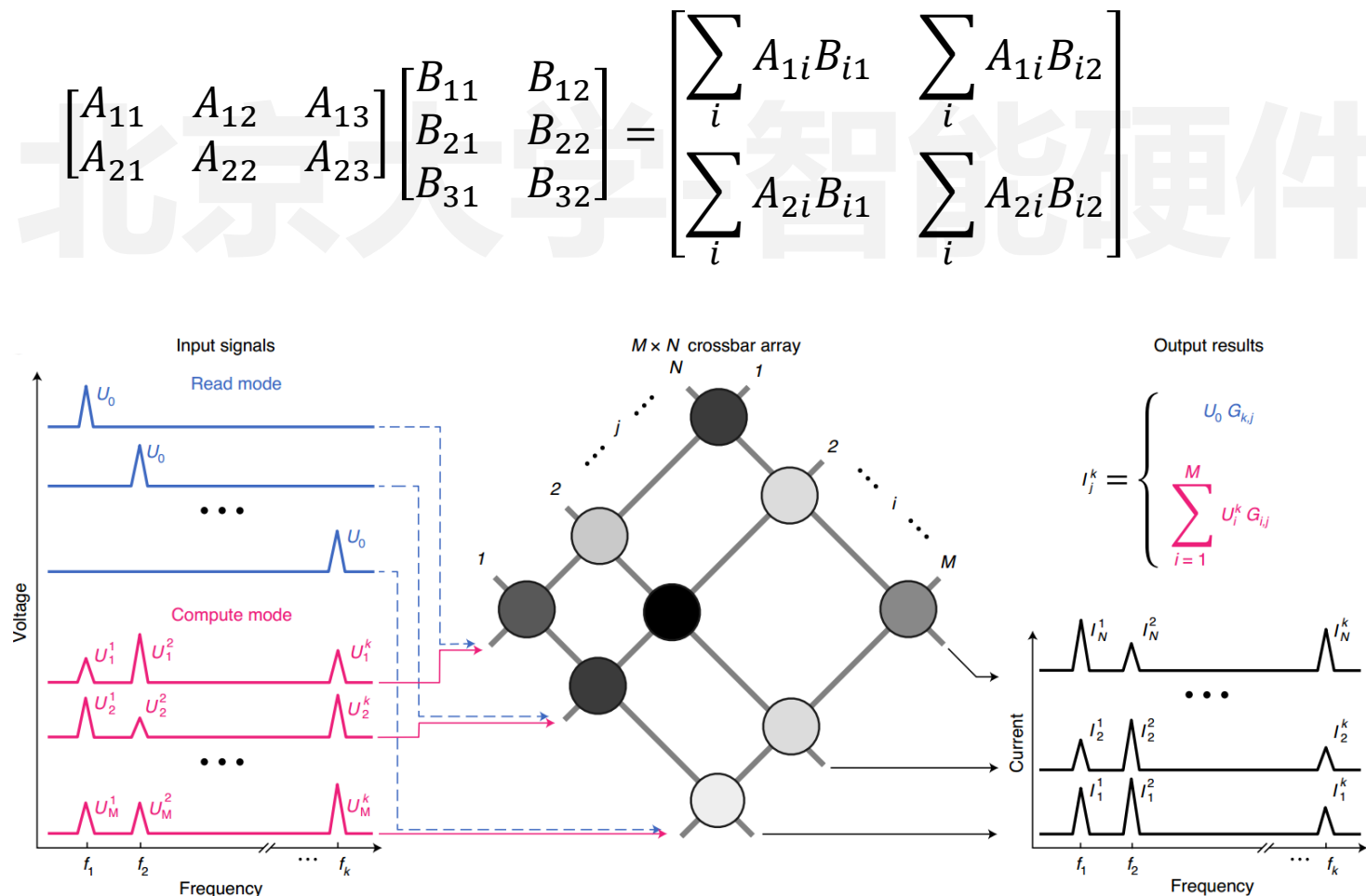
Dumoulin, Vincent, and Francesco Visin. arXiv 2016.

存算一体化典型算子 – 矩阵矩阵乘法

• 矩阵矩阵乘法

□ 构建方法:

- 时域 → 频域
- 在频域进行计算



Wang, Cong, et al. Nature Nanotechnology 2021.

- 模拟权重
- 速度快, 精度低
- 额外开销

□ 傅里叶变换

$$F(\omega) = \int_{-\infty}^{+\infty} f(t)e^{-i\omega t} dt$$

□ 离散傅里叶变换 $\sim O(N^2)$

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-i\frac{2\pi}{N}kn} \quad 0 \leq k < N$$



□ 忆阻器存算一体实现 $\sim O(N)$

$$\begin{bmatrix} X_0 \\ X_1 \\ \dots \\ X_{N-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \omega & \dots & \omega^{N-1} \\ \dots & \dots & \dots & \dots \\ 1 & \omega^{N-1} & \dots & \omega^{(N-1)^2} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_{N-1} \end{bmatrix}$$

欧拉公式 $\omega = e^{-i\frac{2\pi}{N}} = \cos \frac{2\pi}{N} - i \sin \frac{2\pi}{N}$

□ 快速傅里叶变换 $\sim O(N \log N)$

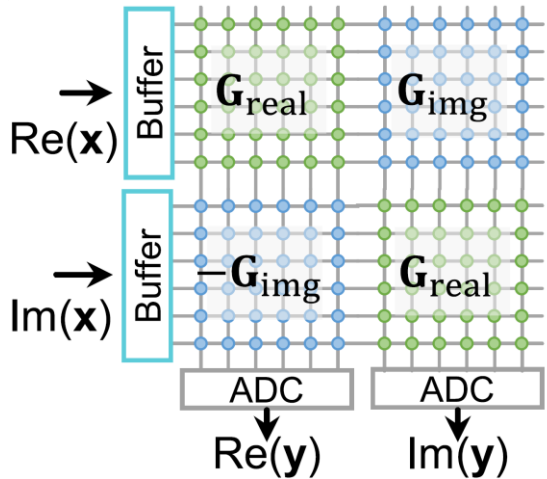
$$\begin{cases} X[k] = F_{\text{even}}(k) + e^{-i\frac{2\pi}{N}k} F_{\text{odd}}(k) & 0 \leq k < \frac{N}{2} \\ X\left[k + \frac{N}{2}\right] = F_{\text{even}}(k) - e^{-i\frac{2\pi}{N}k} F_{\text{odd}}(k) & 0 \leq k < \frac{N}{2} \end{cases}$$

构建方法

$$\begin{bmatrix} X_0 \\ X_1 \\ \dots \\ X_{N-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & \omega & \dots & \omega^{N-1} \\ \dots & \dots & \dots & \dots \\ 1 & \omega^{N-1} & \dots & \omega^{(N-1)^2} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_{N-1} \end{bmatrix}$$

$$\omega = e^{-i\frac{2\pi}{N}} = \cos \frac{2\pi}{N} - i \sin \frac{2\pi}{N}$$

映射方案1

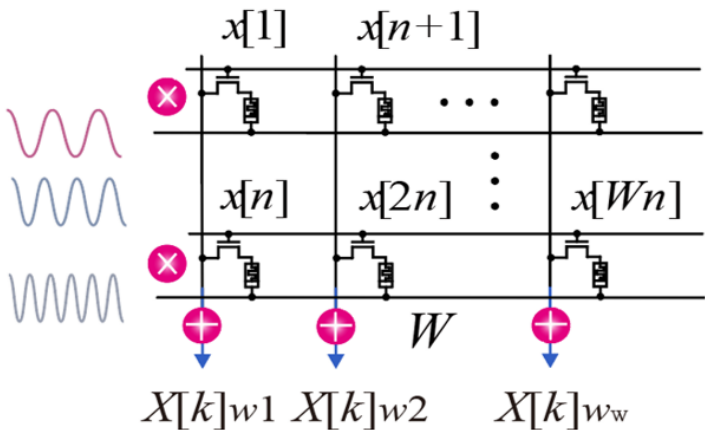


- 模拟计算
- 长度限制
- 阵列冗余

Zhao, Han, et al. Nature Communications 2023.

$$\begin{bmatrix} X_{real} \\ X_{img} \end{bmatrix} = \begin{bmatrix} Re(W) & -Im(W) \\ Im(W) & Re(W) \end{bmatrix} \begin{bmatrix} x_{real} \\ x_{img} \end{bmatrix}$$

映射方案2



- 模拟计算
- 频繁写入
- 多周期输入

$$X[k] = \sum_{n=0}^{N-1} x[n] \left(\cos \frac{2\pi}{N} kn - i \sin \frac{2\pi}{N} kn \right)$$

- 基于阵列的非线性算子实现方法

□ 线性算子

1. 矩阵向量乘法
2. 矩阵矩阵乘法
3. 矩阵矩阵向量乘法
4. 傅里叶变换

.....

□ 非线性算子

1. 汉明距离 (查找比对)
2. 欧氏距离
3. 排序
4. 正态分布生成
5. 非线性初等函数

.....

叠加原理: $A(x + y) = A(x) + A(y)$

齐次原理: $A(\alpha x) = \alpha A(x)$

□ 闵可夫斯基距离 (L-P范数)

$$L_p = \|x - w\|_p = \left(\sum_i |x_i - w_i|^p \right)^{\frac{1}{p}}$$

难点：指数的计算

难点1：异或的计算

难点2：求和的计算

□ 曼哈顿距离 (p=1)

$$L_1 = \|x - w\|_1 = \sum_i |x_i - w_i|$$

□ 汉明距离

■ 01串: $L_d = L_1 = \sum_i |x_i - w_i|$

■ 其他情况: $L_d = \sum_i \text{xor}(x_i, w_i)$

□ 欧氏距离 (p=2)

$$L_2 = \|x - w\|_2 = \sqrt{\sum_i (x_i - w_i)^2} = \sqrt{\sum_i (x_i^2 - 2x_iw_i + w_i^2)}$$

难点1：根号的计算

难点2：向量平方项的计算

存算一体化典型算子 – 汉明距离

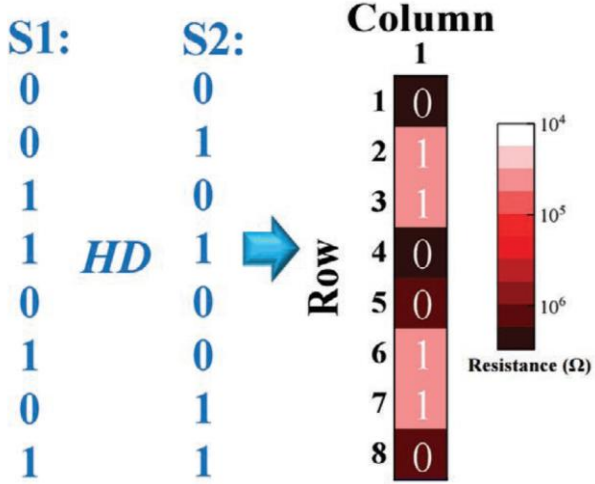
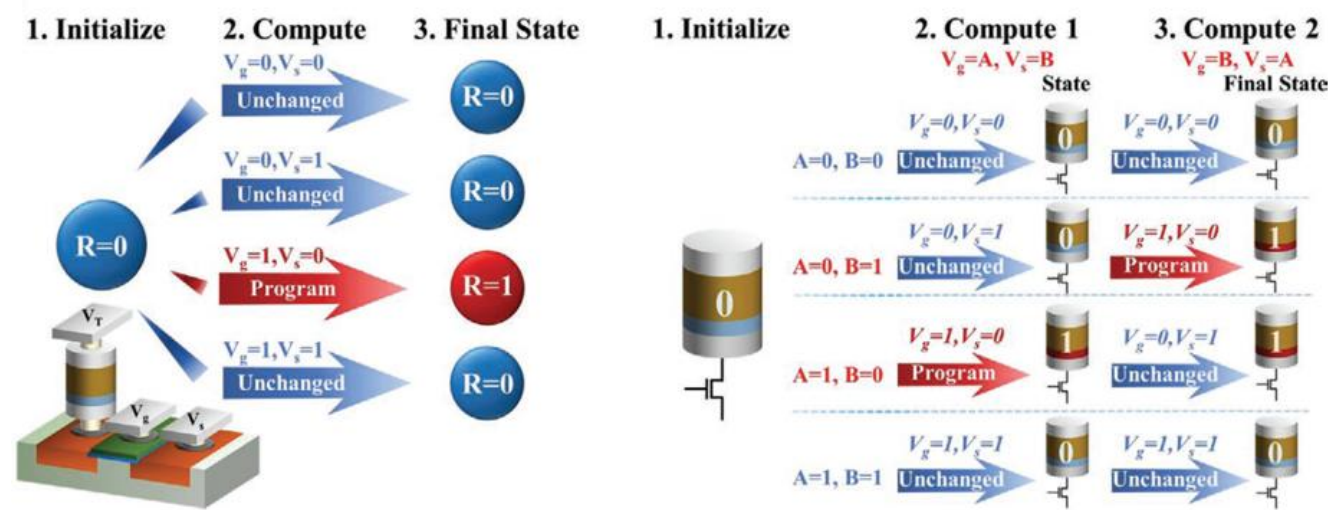
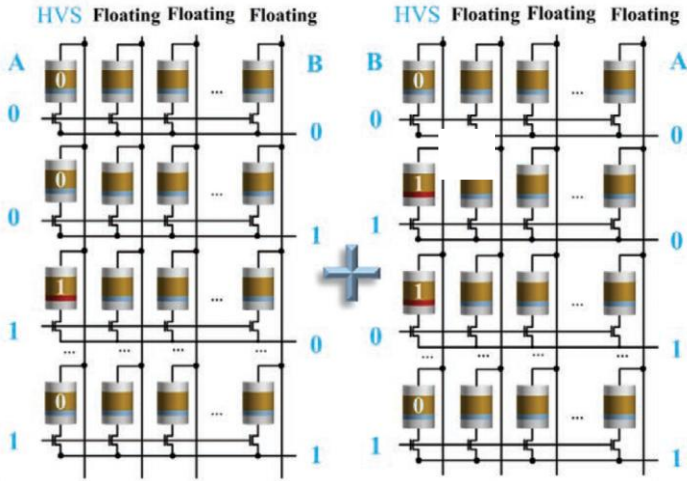
汉明距离实现方案1

- 可以处理任意字符串
- 频繁写入 —— 耐久性
- 多步操作 —— 时间

$$L_d = \sum_i \text{xor}(x_i, w_i)$$

忆阻器电导 BL输入 WL输入

基尔霍夫电流输出



Lin, Huai, et al. Advanced Materials Technologies 2021.

汉明距离实现方案2

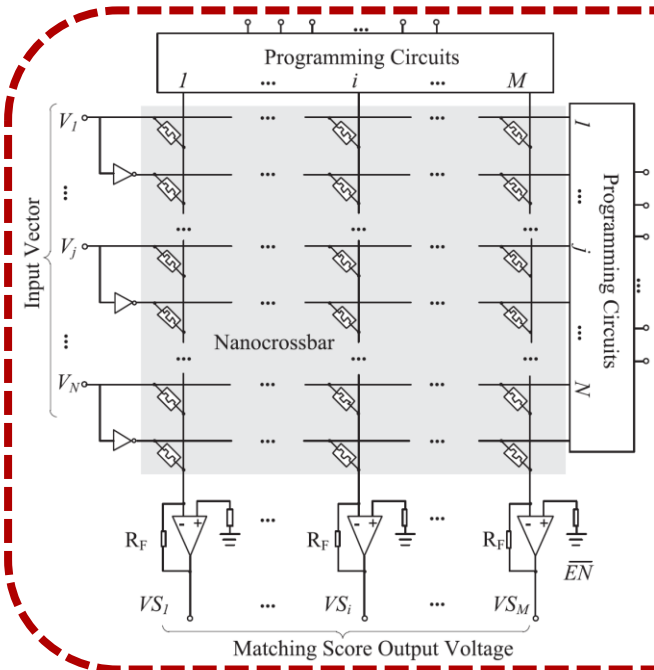
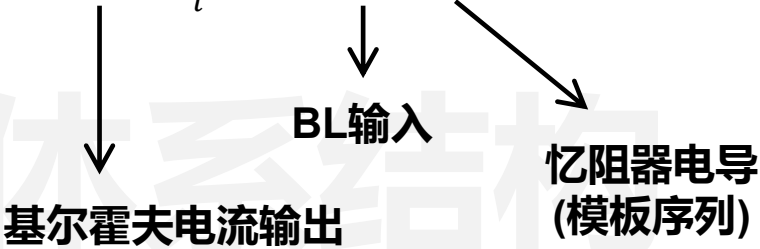
- 某一序列固定
- 翻倍的阵列和输入
- 支持多bit

序列编码方式

XOR

XNOR

$$L_d = \sum_i xor(x_i, w_i)$$



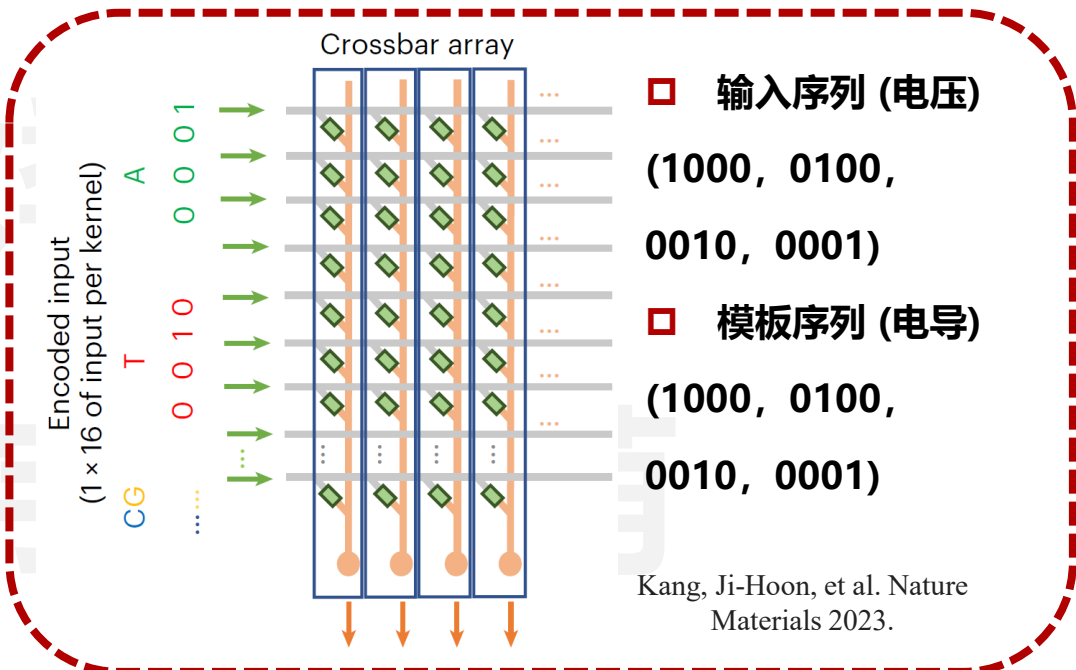
输入序列 (电压)

- 0 → (0, Vr)
- 1 → (Vr, 0)

模板序列 (电导)

- 0 → (Gh, Gl)
- 1 → (Gl, Gh)

Zhu, Xuan, et al. IEICE Electronics Express 2013.



输入序列 (电压)

- (1000, 0100, 0010, 0001)

模板序列 (电导)

- (1000, 0100, 0010, 0001)

Kang, Ji-Hoon, et al. Nature Materials 2023.

存算一体化典型算子 – 汉明距离 (查找比对)

查找比对

- 并行性
- 输出简化

$$\sum_i xor(x_i, w_i) = 0 ?$$

查找比对

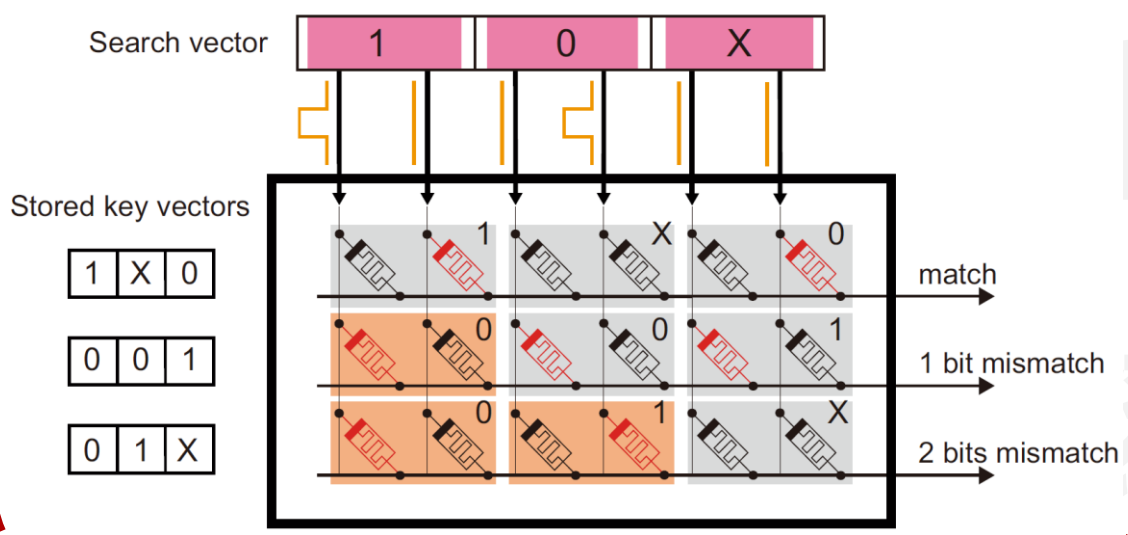
- 多条序列
- 判断匹配

汉明距离

- 两条序列
- 求和

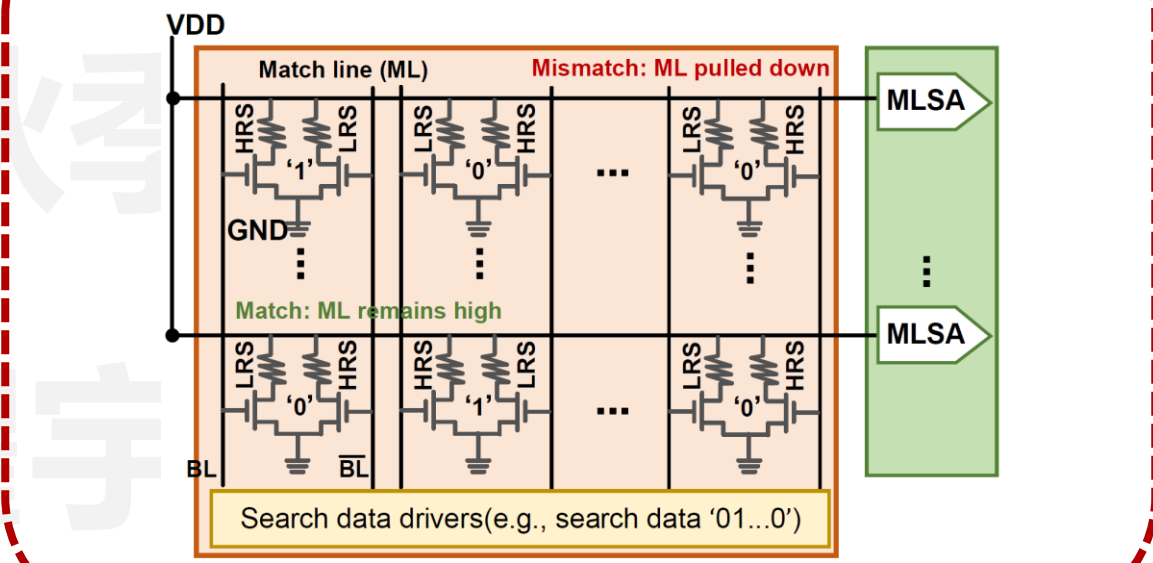
$$xor(x_i, w_i)$$

引入不关心位 X → (0, 0) 或 (GI, GI)



Mao, Ruibin, et al. Nature communications 2022.

引入不关心位 X + 输出简化



Liu, Fangxin, et al. DAC 2022.

□ 欧氏距离实现

- 写操作开销过大 → 忽略**输入序列**平方项的计算
- 引入平方计算行 → 进行**模板序列**平方项的计算
- 针对只需比较的特殊情况 → 可忽略**根号**的计算

$$L_2 = \sqrt{\sum_i (x_i - w_i)^2} = \sqrt{\sum_i (x_i^2 - 2x_iw_i + w_i^2)}$$

□ 根据基尔霍夫定律

$$\begin{aligned} I_j &= \sum_i x_i w_{ij} - \frac{M}{2} * \sum_i \frac{(w_{ij})^2}{M} \\ &= \sum_i (x_{ij} w_{ij} - \frac{(w_{ij})^2}{2}) \end{aligned}$$

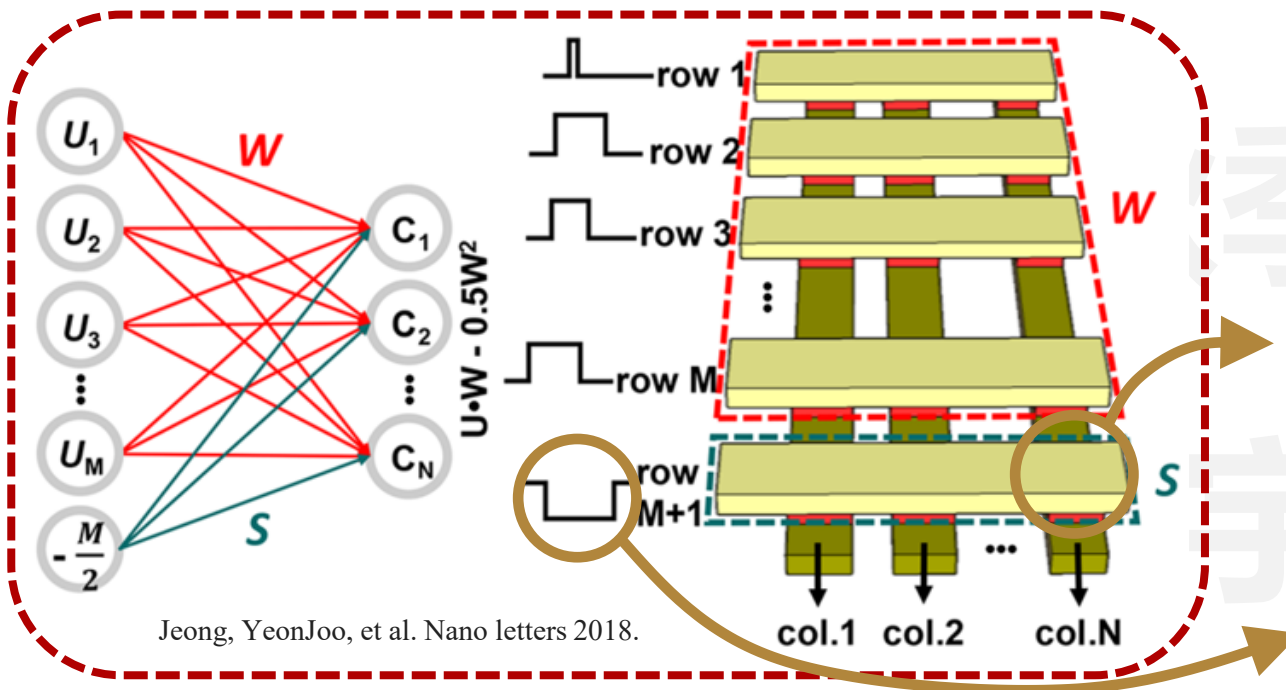
$$\sum_i \frac{(w_{ij})^2}{M}$$

$$-\frac{M}{2}$$

□ 器件电导被M约束



□ 输入也会受M影响



Jeong, YeonJoo, et al. Nano letters 2018.

□ 欧氏距离优化

- 扩展平方计算行
- 输入更改 + 权重更改

$$L_2 = \sqrt{\sum_i (x_i - w_i)^2} = \sqrt{\sum_i (x_i^2 - 2x_iw_i + w_i^2)}$$

□ 根据基尔霍夫定律

$$\begin{aligned} I_j &= \sum_i x_i w_{ij} - \frac{1}{2} * L * \sum_i \frac{(w_{ij})^2}{L} \\ &= \sum_i (x_{ij} w_{ij} - \frac{(w_{ij})^2}{2}) \end{aligned}$$

$$\sum_i \frac{(w_{ij})^2}{L}$$

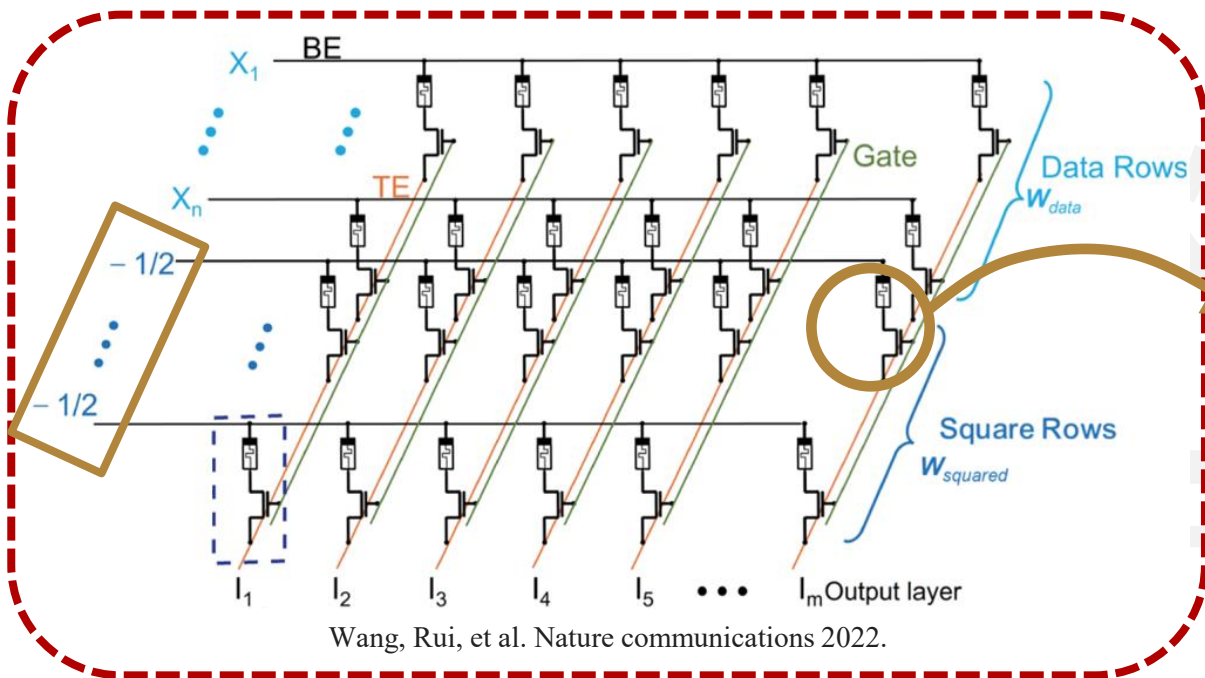
□ 器件电导被L约束



□ 输入被约束在正常范围



□ 额外的器件面积



Wang, Rui, et al. Nature communications 2022.

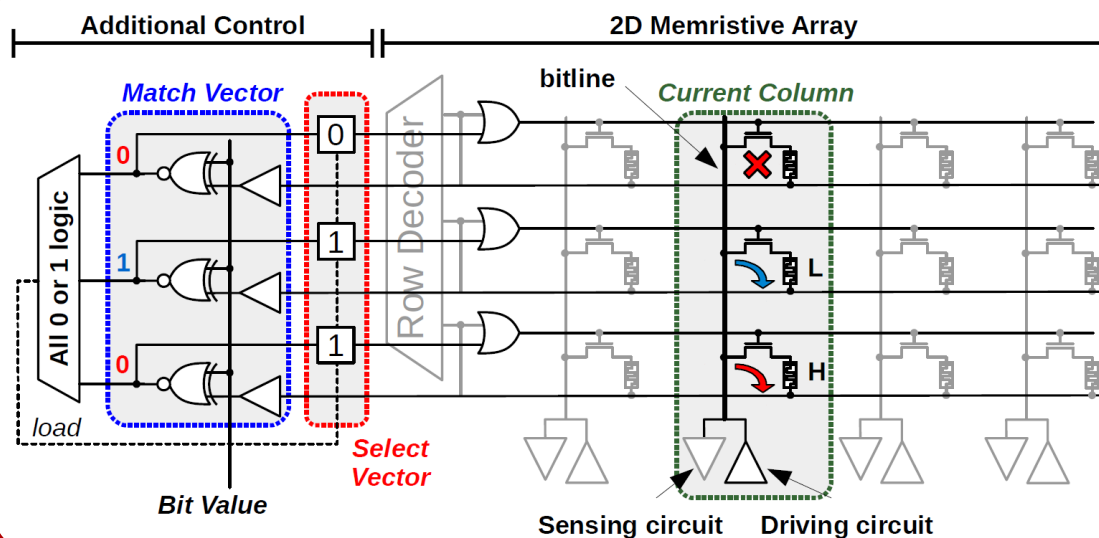
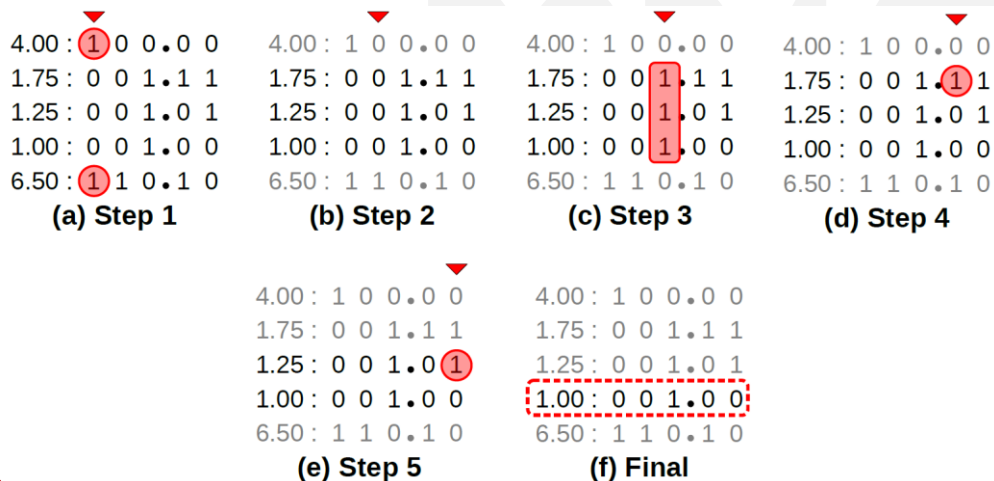
□ 排序：将一组杂乱无章的数据元素，通过一定方法按关键字顺序排列

- 待排序数据存储为器件电导
- WL上的电压映射为数据状态

算法限制速度 $O(NW)$

电路与阵列限制可拓展性

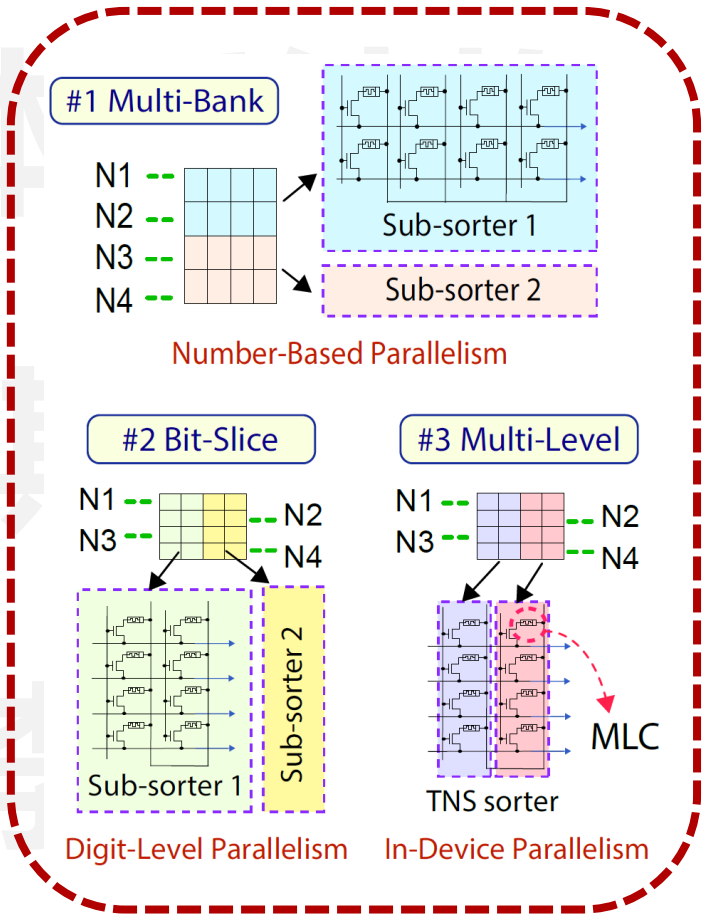
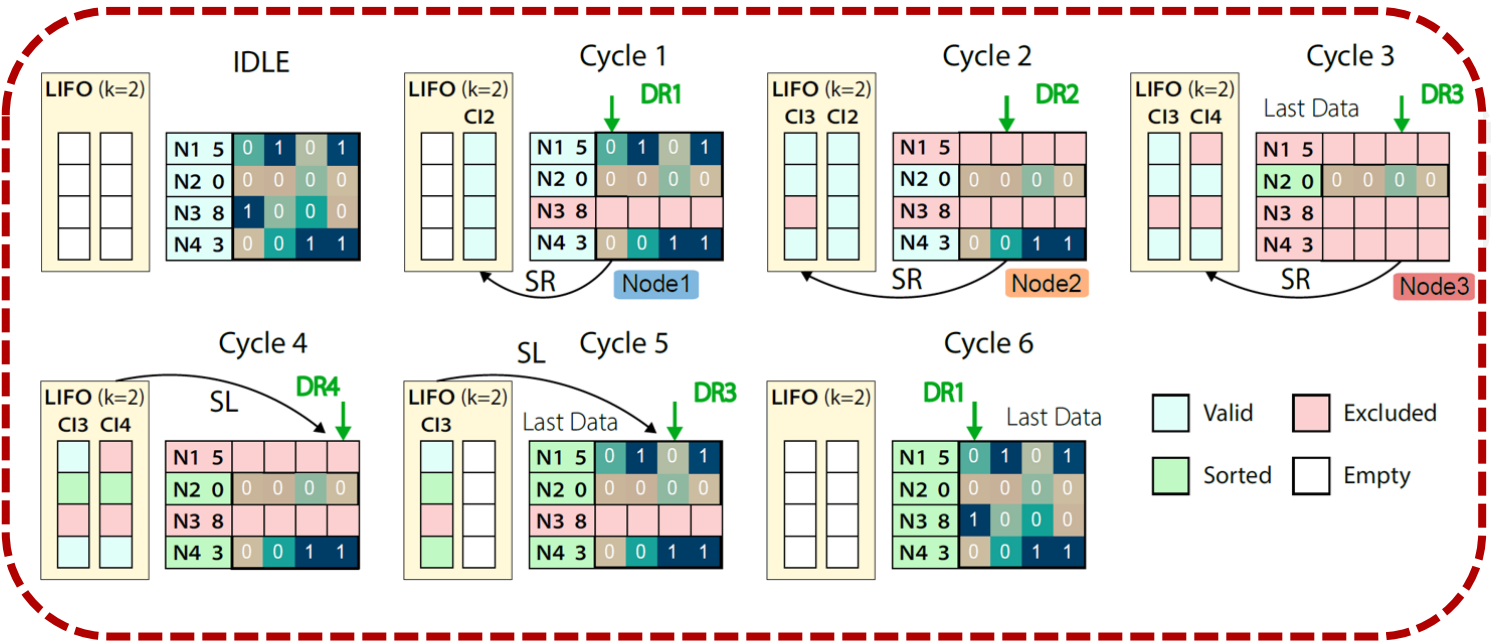
逐位并行读操作实现最小/大值搜索



Prasad, Ananth Krishna, et al. HPCA 2021.

□ 排序优化

- 速度优化 → 更高效的排序算法 $O(N + W)$
- 可拓展性优化 → 三种配置 (阵列规模, 并行性, 多值器件)
- 排序数据固定



存算一体化典型算子 – 生成统计分布

□ 正态分布实现方案1

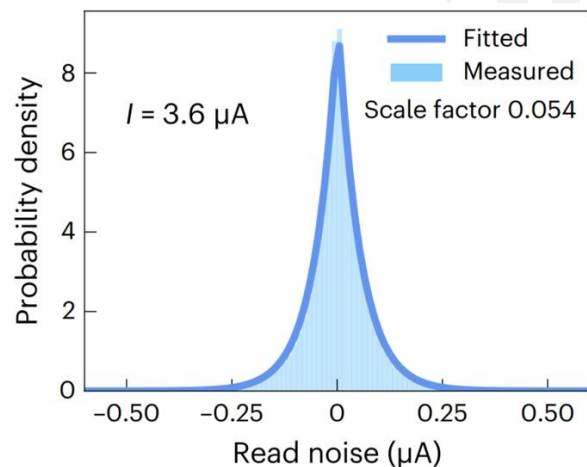
对于独立并同样分布 (IID) 的随机变量，即使原始变量本身不是正态分布，标准化样本均值的抽样分布也趋向于标准正态分布

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad X \sim N(\mu, \sigma^2)$$

难点1：随机数生成

难点2：支持可控参数

□ 忆阻器读噪声 (C2C随机性)

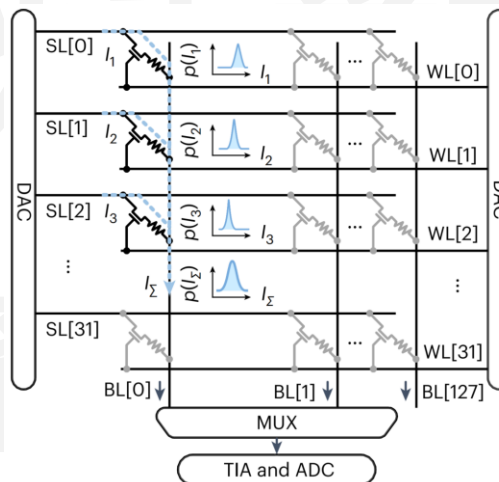


□ 多器件产生高斯分布 (中心极限定理)

$$f(x) = \frac{1}{2b} \exp\left(-\frac{|x|}{b}\right)$$

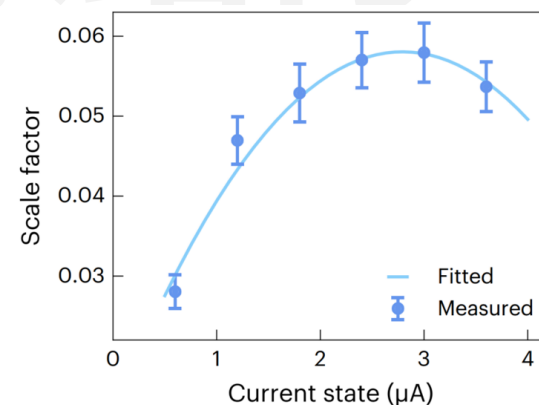
$\{X_n\}$ i.i.d

$$\sum_i X_i \sim N(n\mu, n\sigma^2)$$



□ 多值忆阻器

□ 任意分布的中心极限定律



$$\frac{\sum_i (X_i - \mu_i)}{\sqrt{\sum_i \sigma_i^2}} \sim N(0, 1)$$

Lin, Yudeng, et al. Nature Machine Intelligence 2023.

□ 正态分布实现方案2

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$X \sim N(\mu, \sigma^2)$$

难点1: 随机数生成

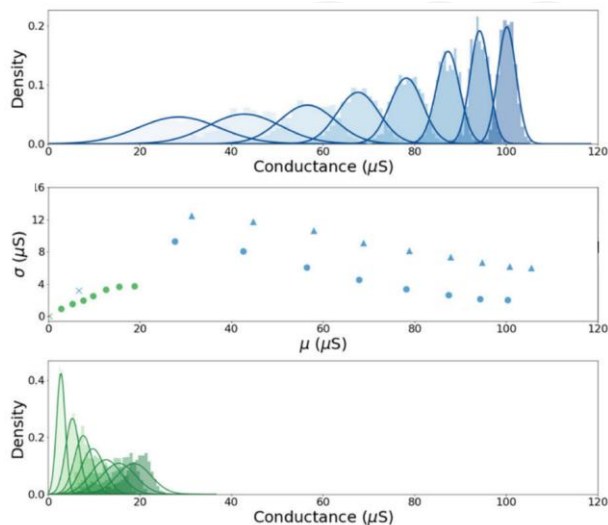
难点2: 支持可控参数

方案1

方案2

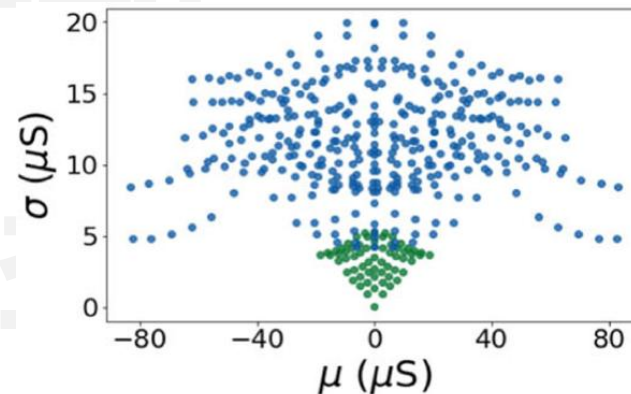
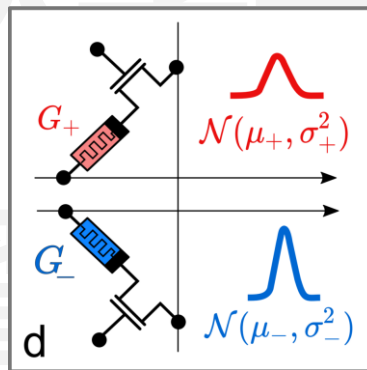
- | | | |
|---------|------|-------|
| □ 单随机数: | 多器件 | 1-2器件 |
| □ 分布: | 一组器件 | 多组器件 |
| □ 采样: | 时间 | 空间 |

□ 忆阻器电导分布 (D2D随机性)



- 多值忆阻器 + 不同忆阻器
- 作差 (正态分布的性质)

$$(G_+ - G_-) \sim N(\mu_+ - \mu_-, \sigma_+^2 + \sigma_-^2)$$



Bonnet, Djohan, et al. Nature communications 2023.

存算一体化典型算子 – 非线性初等函数

□ 非线性初等函数包括:

■ 幂函数

$$y = x^a$$

■ 指数函数

$$y = a^x$$

■ 对数函数

$$y = \log x$$

■ 三角函数

$$y = \sin x$$

■ 反三角函数

$$y = \arcsin x$$

■ 双曲函数

$$y = \sinh x$$

传统实现方案

□ 数字域

□ $+-\times\div$

□ 泰勒展开

$$y = e^x = 1 + x + \cdots + \frac{x^n}{n!}$$

计算开销很大

□ 秦九韶算法

$$\begin{aligned} y &= a_n x^n + a_{n-1} x^{n-1} + \cdots + a_0 \\ &= ((a_n x + a_{n-1})x + \cdots)x + a_0 \end{aligned}$$

计算开销大

□ 查找表 + 插值

预先计算好, 需要从内存中提取
其他值采取对最接近值插值的方式

存储开销大

□ 其他算法: CORDIC, BKM

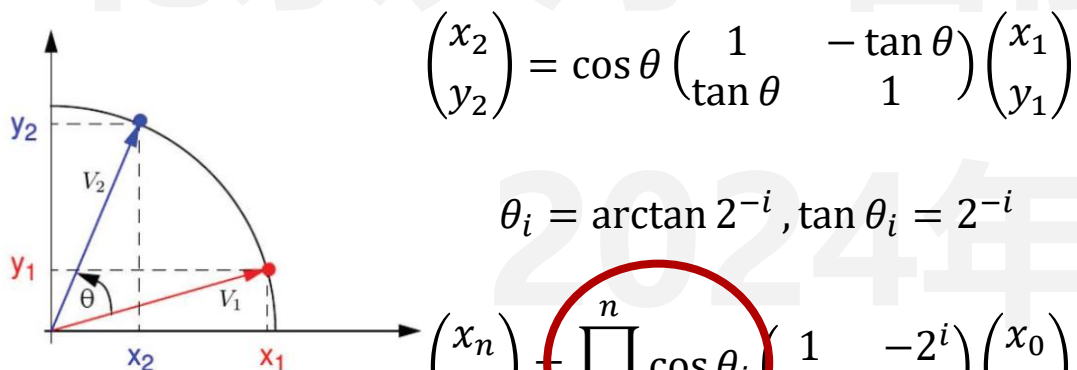
硬件友好

常用

□ CORDIC算法

■ 基本原理：伪旋转

■ 基本思想：进行很多次伪旋转直到目标角度，每次伪旋转的角度选择方便计算的值得



Zhang, Zihan, et al.
GLSVLSI. 2020.

将 $\tan$ 计算转化
成移位计算

$\tan \theta_i$	θ_i
2^{-0}	45°
2^{-1}	26.565°
2^{-2}	14.036°
2^{-3}	7.1250°
2^{-4}	3.5763°
2^{-5}	1.7899°
2^{-6}	0.8951°

$$\begin{cases} x_{i+1} = x_i - \mu d_i (2^{-i} y_i) \\ y_{i+1} = y_i + d_i (2^{-i} x_i) \\ z_{i+1} = z_i - d_i \theta_i \end{cases}$$

- z_i 实现角度累加
- d_i 控制旋转方向
- μ 选择坐标系

$$K = \prod_{i=0}^n \cos \theta_i = 0.607252935 \quad (n \rightarrow +\infty)$$

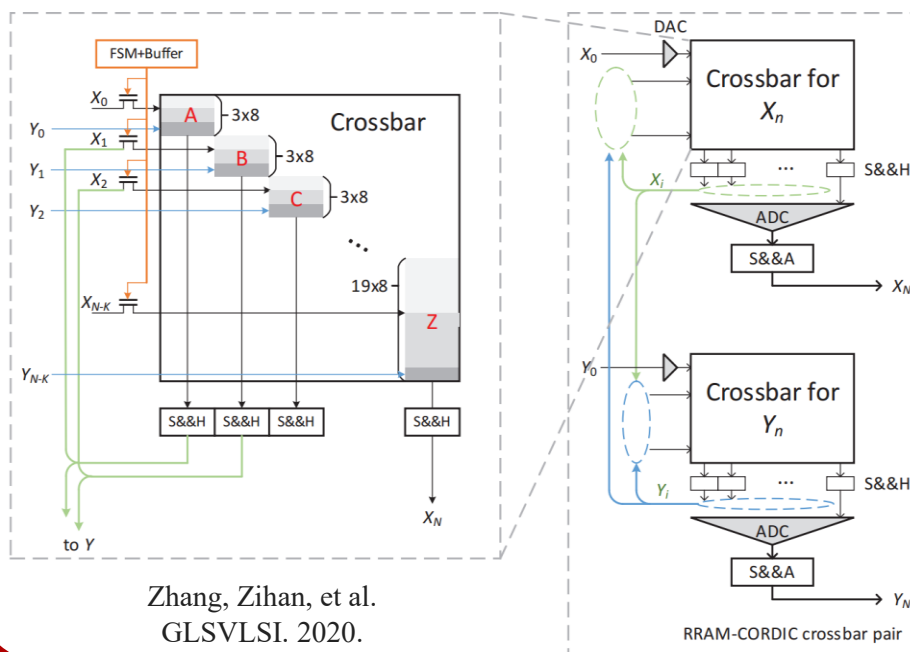
CORDIC算法的忆阻器阵列实现

- 稀疏矩阵
- 迭代次数固定
- 计算粒度广，时间长

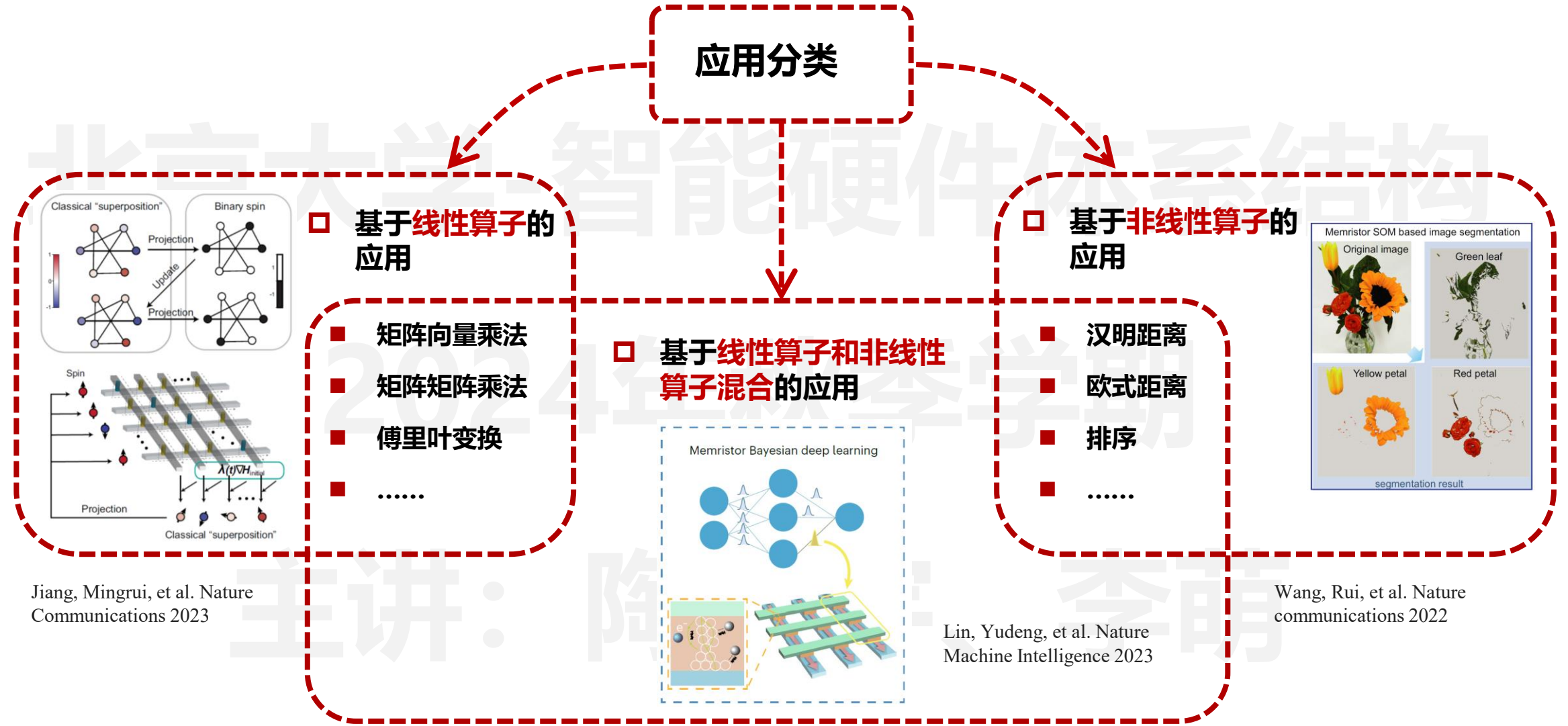
移位操作适合CMOS电路实现，但是并不适合忆阻器阵列

$$\begin{cases} x_{i+1} = x_i - \mu d_i (2^{-i} y_i) \\ y_{i+1} = y_i + d_i (2^{-i} x_i) \\ z_{i+1} = z_i - d_i \theta_i \end{cases}$$

- z_i 实现角度累加
- d_i 控制旋转方向
- μ 选择坐标系



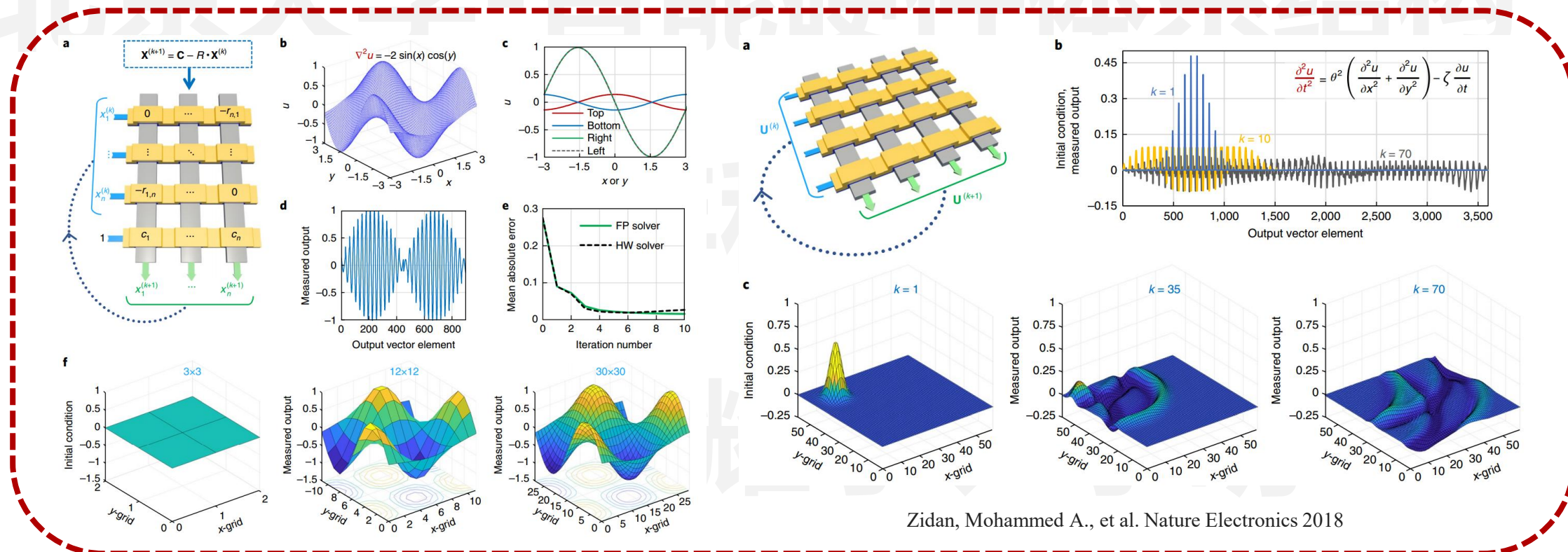
- 三行分别实现：
 $1, \pm 2^{-i}$
- z_i 的计算在外部进行
- x_i 和 y_i 进行反复迭代
- 不重要的位数一次算完



线性算子：偏微分方程求解

□ 偏微分方程求解（矩阵向量乘法）

- 在数学、物理及工程技术中应用非常广泛，习惯上把这些方程称为数学物理方程
- 使用忆阻器阵列实现矩阵向量乘法，用于雅可比迭代法求解偏微分方程组

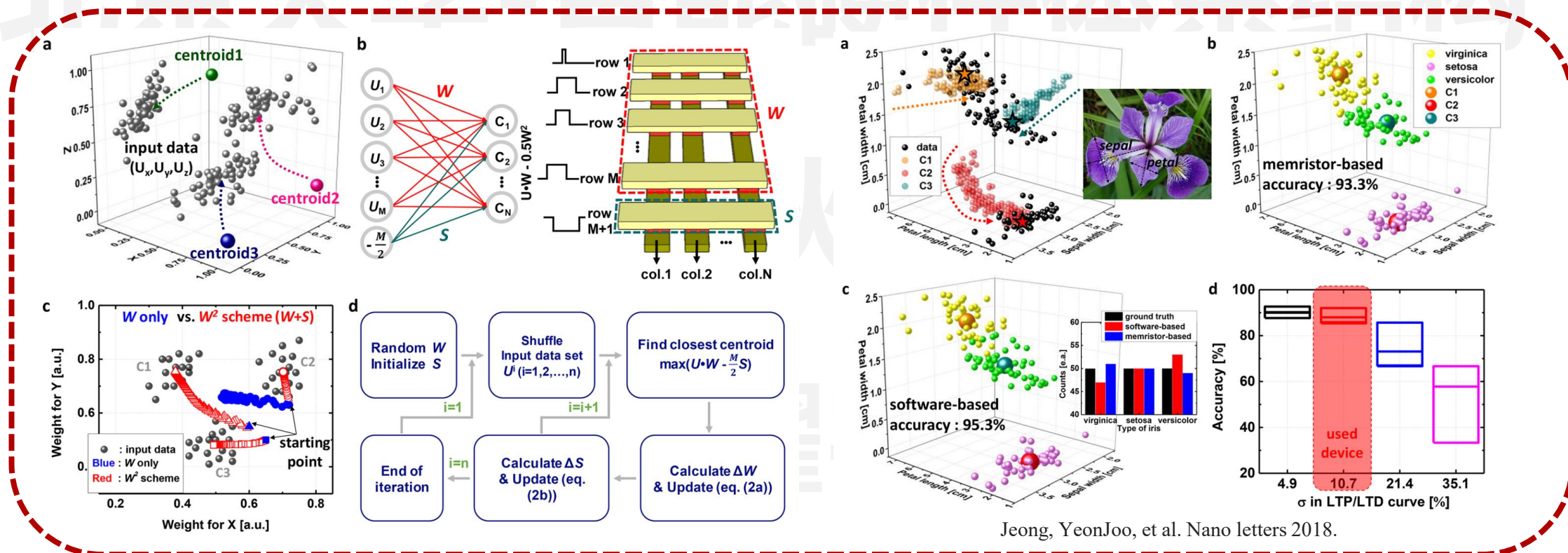


Zidan, Mohammed A., et al. Nature Electronics 2018

非线性算子：聚类算法

□ K 均值算法 (欧氏距离)

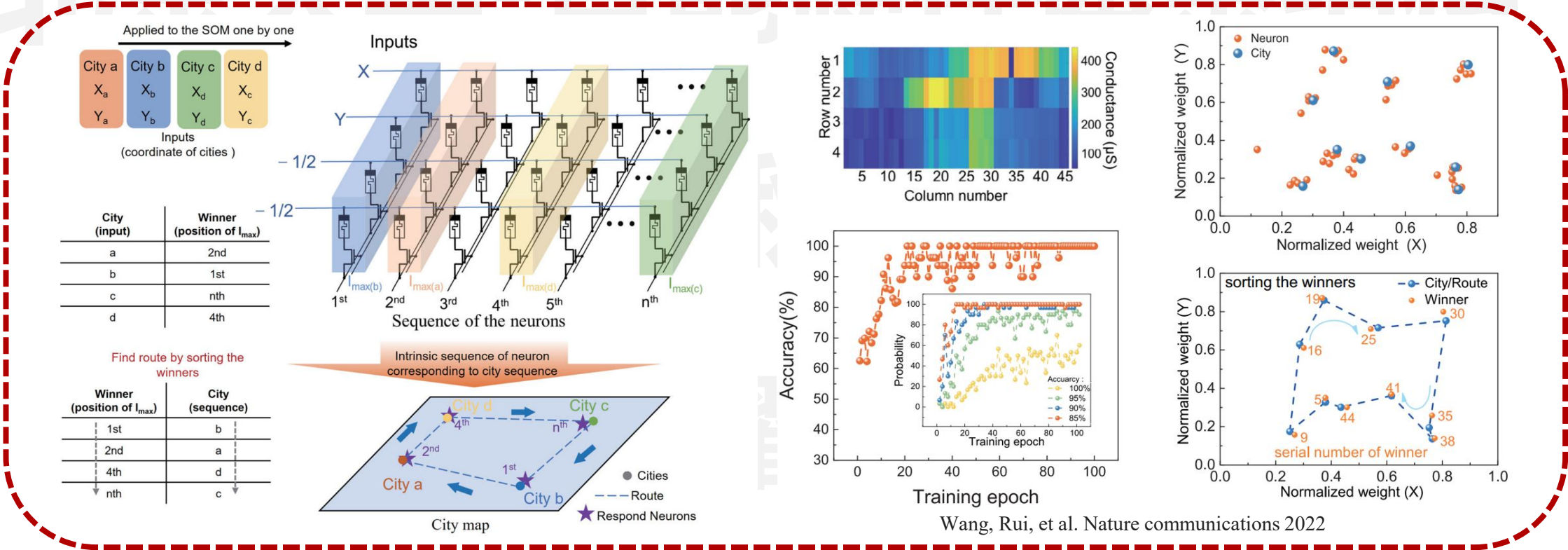
- 一种无监督算法，旨在将一组向量输入划分为 K 个簇，即对未标记数据集进行预聚类
- 使用忆阻器阵列实现欧氏距离的比较，而无需计算精确值



Jeong, YeonJoo, et al. Nano letters 2018.

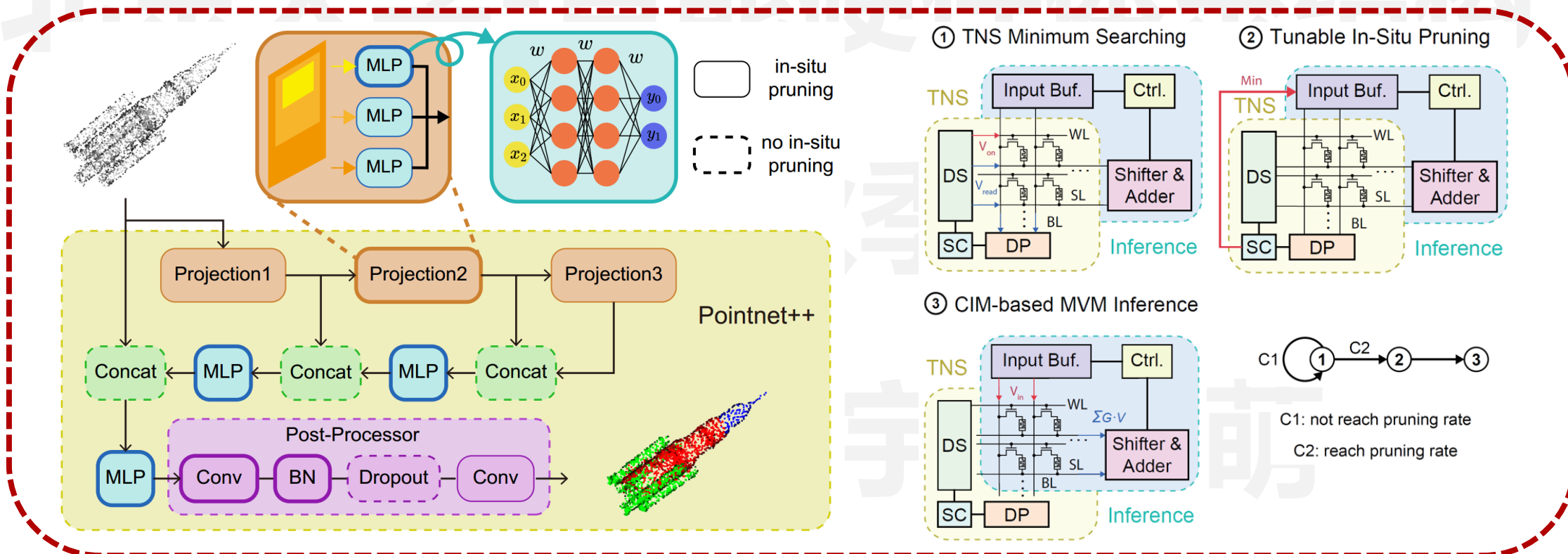
自组织映射算法（欧氏距离）

- 一种无监督算法，旨在通过学习输入空间中的数据，生成一个低维、离散的映射
- 使用忆阻器阵列实现欧氏距离的比较，用于寻找最佳匹配单元（BMU）



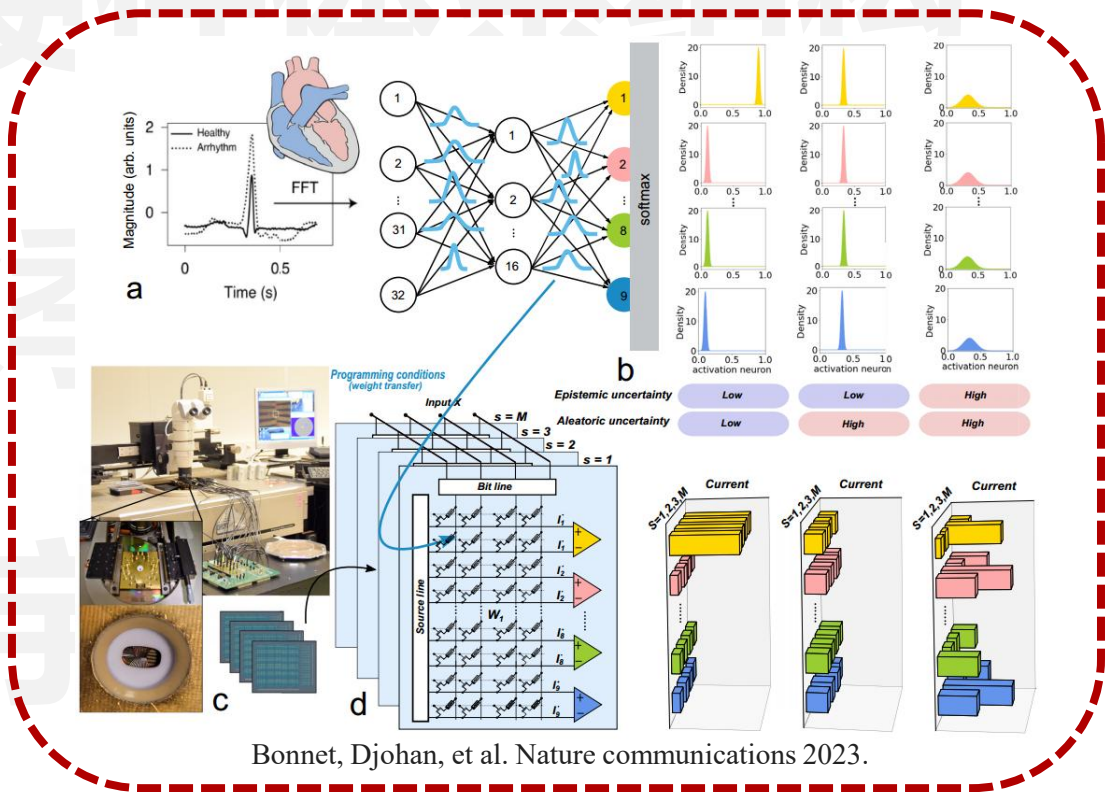
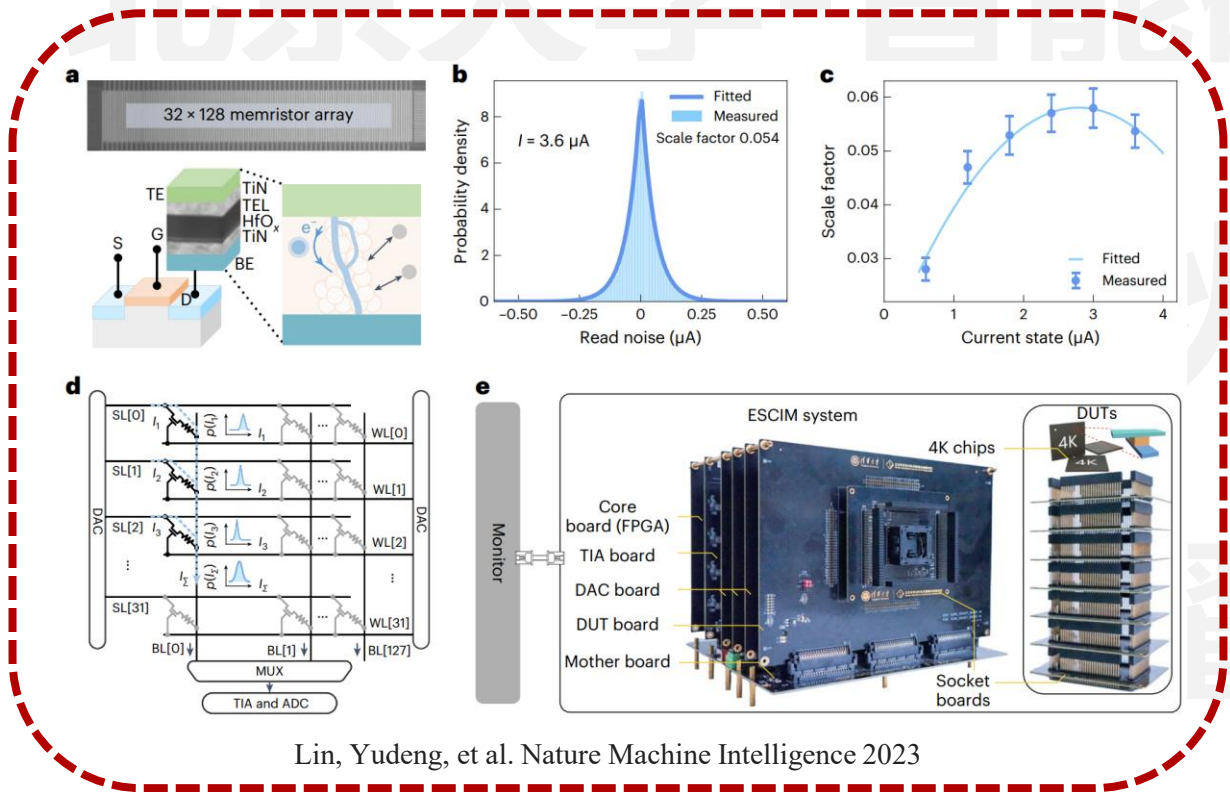
□ 点云切割 (排序+矩阵向量乘法)

- 对点云进行划分，使得同一划分内的点云拥有相似的特征，可用于分类、部件分割等
- 使用忆阻器阵列实现排序和矩阵向量乘法，用于剪枝和神经网络推理



贝叶斯神经网络（正态分布生成+矩阵向量乘法）

- 权重是一个分布而非定值，可以通过估计合理预测值的区间来表达预测的不确定性
- 使用忆阻器阵列实现正态分布权重和矩阵向量乘法，用于贝叶斯神经网络推理



- 在传统的神经网络模型基础上增加存储模块，用于模拟外部记忆
- 使用忆阻器阵列实现矩阵向量乘法和汉明距离，用于神经网络推理和TCAM



